

VECTOR

More eXtreme APL ...

• the VierNeun Puzzle	38
• Barman on Dyalog 9 and XML	48
• Forests in Finland V	66
• John Scholes on "D"	93
• Anne Wilson on Celtic Knotwork	101
• McDonnell's Erdős Number	113



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.17 No.4 April 2001

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2000-2001

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com

Contents

Editorial	Stefano Lanzavecchia	Page 3
APL NEWS		
Quick Reference Diary		5
APL2001: Information and Abstracts		7
News from Sustaining Members		8
APL Product Guide	Gill Smith	11
The Education Vector		
Zark Newsletter Extracts	edited by Jon Sandles	26
Crossword		28
J-ottings 28: Just say it in J	Norman Thomson	30
Puzzle Corner	Stefano Lanzavecchia	38
REVIEW		
XML in Dyalog APL Version 9	Jonathan Barman	48
J - the Natural Language for Analytical Computing		65
RECENT MEETINGS		
Forests in Finland V	Ray Cannon	66
The Development and Q/A Process at Microsoft, US	Kari Höijärvi	78
Array-based Logic	Klaus Klug Christiansen	85
GENERAL ARTICLES		
Dynamic Functions Masterclass	John Scholes	93
Over and Under: Celtic Knotwork Patterns	Anne Wilson	101
TECHNICAL SECTION		
Technical Correspondence		112
At Play with J: Erdős Numbers and Pierce and Engel Expansions	Gene McDonnell	113
A New Life For V: Structure Control	F.H.D. van Batenburg	125
J Rational Numbers....	Gordon Sutcliffe	131
Index to Advertisers		143

SHARP APL for Linux

SHARP APL 6.0 for Linux

- Full implementation of SHARP APL for UNIX
- Based on Iverson's APL Dictionary
- Rich in functionality
- Comprehensive documentation
- Free for personal use

SHARP APL Java Interface

- Available for SHARP APL 6.0 for Linux
- Platform independent Graphical User Interfaces
- Access from APL to Java, including Java services such as JDBC
- Access from Java to APL
- Free with personal use version of SHARP APL for Linux

Groot Blankenberg 53
1082 AC Amsterdam
The Netherlands
Tel: **+31.20.646.4475**
Fax: **+31.20.644.1206**

44 Victoria Street,
Suite 2100
Toronto, Ontario,
Canada M5C 1Y2
Tel: **416.364.9355**
Fax: **416.364.6159**

1100 University Avenue,
Suite 111
Rochester, NY, USA 14607
Tel: **716.256.6466**
Fax: **716.256.6469**

For information on the Personal Edition of SHARP APL for Linux, see www.soliton.com/Linux.

For information on the Commercial Edition of SHARP APL for Linux, contact sales@soliton.com or call Laurie Howard at **+31.20.646.4475** (for Europe) or Nancy Lamb at **716.256.6466** (for North America)



**SOLITON
ASSOCIATES**

Enterprise Software Solutions

Editorial

by Stefano Lanzavecchia

"And I think that I still can much better than most
produce blood curdling noises to bring on the ghost"

T.S. Eliot – Cats

APLers meet once a year, to paraphrase T.S. Eliot's beautiful Cats. They do enjoy singing and dancing and also various kinds of conversations under the sunlight as well as the moonlight, under the skies of a European capital, or of an American venue. In addition to the glorious annual APL conference, more and more geared towards Array based languages in general, there are the user group meetings like the one organised every year in Orlando by APL2000. And if that wasn't enough, several more local meetings take place, like the popular Scandinavian seminars, which attract people from all over Europe. While APLers are traditionally regarded as lone wolves who prefer the company of a single-malt whisky and would rather code the Chrysler payroll system in what they consider the most effective way of working, that is the one-man team, nevertheless they seem to love the sharing of knowledge, techniques, discoveries, at least with the other fellow APLers, who are more likely to understand them.

Certainly the pace of changes has so drastically increased that, sometimes, a yearly conference seems not to be enough for a deep and profitable exchange to happen. This is where, once again, the Internet can be helpful. On the 'net, many discussion groups devoted to the most exotic languages exist where people can give and ask for opinions, present a difficult problem hoping to find help, or simply idly wonder about the beauty of a tricky piece of code. J's discussion forum is a mail-list, monitored but not moderated by the guys at JSoftware, very alive and rich of detailed exchanges about interesting topics. K's is also a mail-list with a somewhat smaller participation, proportioned to the minor spread of the language. APL's official discussion forum is a UUNet newsgroup called comp.lang.apl, available on most news servers and, I believe, also replicated as a mail-list.

Notwithstanding the worldwide spread of APL, it's somewhat disappointing to see that comp.lang.apl (C.L.A. for the intimate) is almost a wasteland where the occasional post on-topic only generates a couple of replies before the silence falls again. Large newsgroups like comp.lang.perl can be so noisy and crowded that they very quickly become impossible to stand: 500 and more messages per day are hard to follow for all but the most dedicated souls and the natural consequence is to consider them a waste of time. comp.lang.apl is at the other side of the

spectrum. That's the one thing I don't quite understand: accepting the fact that even APLers consider valuable the exchange of information, then why don't they make more use of the newsgroup? I agree that at times UUNet newsgroups can be intimidating: there exist barons who think they are in charge of the group and would censor the occasional newcomer, verbally when lacking most powerful means, for any however trivial disrespect of the so called rules of the forums, scaring them away. Experience taught me that it's easier to ignore those barons and give a hand to the unlucky newcomer so that he feels at ease and remains: kindness is stronger than the sword. I also agree that most APLers are so busy with their business that they don't find it's worth spending the time even only to read what other people have to say. After all, somebody already said before me: APL is such a powerful tool and weapon that we should keep it secret and be the only ones to benefit. But somehow I think that real occasions are being missed here.

When the J mail-list was created, because somebody felt it was inappropriate and off topic to discuss J in the official APL forum, `comp.lang.apl` almost died because the most active people all followed the J crowd. Somebody will also be a bit upset by the heavy J content of this issue of Vector. There is good news and bad news: the good news is that next issue should be much better balanced, but the bad news is also that the most prolific authors are J minds and it's hard to maintain a high APL content if the only submissions we receive are about J. Even before becoming the editor of Vector (a letter I sent that was published years ago it's the proof), I've always supported J because I thought it was a real mind opener, which would improve my APL awareness. Array thinking is a state-of-mind that can be stimulated by different sources. I hope you agree with me. Since Vector remains the journal of the British APL Association, though, it would be nice to see a bit more APL content...

Last time, I announced that A+ was going to be released to the public in an Open Source version. Just to remind you, A+ is an APL dialect developed at Morgan Stanley with a Sharp APL flavour to it; Open Source means that not only the executable is available for free, but the complete source code is at your disposal, whether you want to learn about the dirty inner workings of an APL interpreter or you intend to customise it for a special need you might have. If you are interested, the official Web page of the project is <http://www.aplusdev.org/>. The documentation by itself is well worth a visit. The only drawback for me is the absence of a Windows version: only Unix is supported, especially as far as the GUI front-end is concerned. The BAA is debating the feasibility and the need of an at least partial port of the interpreter to the Windows platform. We will keep you informed.

Quick Reference Diary

Date	Venue	Event
18th May 2001	RSS, London	BAA Spring meeting and AGM
25th-28th June	Yale, USA	APL2001 Conference
23rd July 2002	Madrid, Spain	APL2002 Conference

Dates for Future Issues of VECTOR

	Vol.18 No.1	Vol.18 No.2	Vol.18 No.3
Copy date	8th June	7th September	7th December
Ad booking	15th June	14th September	14th December
Ad Copy	22nd June	21st September	21st December
Distribution	July 2001	October 2001	January 2002

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

**Download
from the
web for
free!**

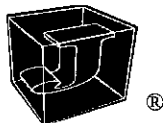
Announcing JsoftwareTM Version 4.06

- ♦ New and Improved Documentation
- ♦ New Data Types, Symbol and Unicode
- ♦ DLL Callbacks Supported
- ♦ New Language Features
- ♦ Improved Locales
- ♦ And more!

This release will make you more productive. We've listened to our users requests, and implemented many technical changes and improvements. We also added new features, for example, significant new data types.

Make sure to watch our web site for the new release. Professional Edition subscribers will receive their copies in the mail after the edition is released on the web.

Jsoftware Inc.
19235 Covington Ct.
Shorewood, MN 55331
Info@jsoftware.com
Tel (952) 470-7345
Fax (952) 470-9202



www.jsoftware.com

APL2001: Information and Abstracts

Conference Site

The conference will be held at Yale University, which is celebrating its 300th anniversary. Yale has made important contributions to the development of APL, both directly and indirectly through its graduates and faculty. In addition Yale is an important educational and cultural center with a number of important museums on campus. In many ways it is an ideal place to host our conference.

Invited Speakers

Paul Hudak, Chairman of the Yale Computer Science Department: *Computer Science Research at Yale*.

Judson Rosebush, renowned Computer Graphics Expert: *Digital Fusion*

Richard Krafchin, APLer par excellence: *Animating Email*

Workshops/Tutorials

A number of workshops and/or tutorials will be presented on such topics as Mathematical play with APL; J for Educators; APL and Excel; APL and the WWW; An Introduction to MATLAB; Utility functions in APL; Fundamentals of Objects; APL under Linux

Abstracts of Papers

A Canon for the Representation of Multivariate Arrays in the APL2 Computing Language, Richard Drexel Stockbridge. A uniform structure for multivariate data constructs that describes its own contents. A library of APL2 functions perform analytical, archival, and graphical operations on these structures.

Adage: A Dynamic APL GUI Equation Solver, Stephen M. Mansour ADAGE, (A Dynamic APL GUI Equation solver), allows the user to solve a single multivariate equation when values are supplied for all but one of the variables. Examples from the mortgage industry are explored.

Embedding a Tcl Web Server into APL, Dr. Tilman P. Otto. Tcl, Tool Command Language, is a script language and an extensible interpreter. The many available Tcl extensions for GUI, network, database, HTML and XML support, make the combination of APL and Tcl very attractive.

Good Taste in Color and Soft Palettes, Devon McCormick. Contrasts the exploration of a problem in a non-array-processing language, Java, and 2 array-processing languages, APL and J. The task is to generate a 256-color palette for a bitmap where the colors cover a full range of available hues evenly.

Operation Arrays in APL, Jacob Brickman. Explores the many ramifications of allowing arrays of APL functions, operators, and future syntactic classes. How operation arrays relate to Object-Oriented Programming, the implications of side effects, scoping, memory models and synchronization.

Suggestions for a Method of Analyzing Binary Images Using Langlet's Parity Logic, Stuart Smith. Describes an extension to the system of parity logic operations developed by Gerard Langlet and subsequently elaborated by Michael Zaus. The algorithms and the programming examples are coded in Q'Nial.

Is There a Way to Combine Array-processing and Object-Oriented Programming? Georg Reichard. It is shown that, with today's APL interpreters, it is possible to combine array-processing and object-oriented programming on a very high level of abstraction.

Replacing an Explicit Loop by the Each Operator, Joseph Burchfield, Samir Lipovaca. A comparison of explicit loops and array operations focusing on the each operator.

J and Sports Betting, Pablo Landherr. This paper discusses how J was used to describe the math in one area of sports betting to develop an automatic odds generator and to test this generator using robots.

Correctly Detecting Intrinsic Type Errors in Typeless Languages such as MATLAB, G. Pramod, V. Joisha. This paper presents a methodology for inferring a variable's type in languages which do not explicitly declare type. The techniques are demonstrated in MATLAB.

Array Form Representation of Idiom Recognition System for Numerical Programs, Hiroyuki Sato. PSE (problem solving environment) is an approach to providing high-level abstract functions. Term rewriting theory is applied to the problem in Fortran 90/95 matrix manipulations.

Graph Coloring Algorithms in J, Howard A. Peelle. Four basic algorithms for coloring non-directed graphs are described in J, along with tools for generating adequacy matrices for such graphs and testing them

D2Cell Data Mining Utility, John Manges. D2Cell is a Data Mining Utility for Windows based PCs which downloads custom Mainframe DB2 data directly into Microsoft Excel. A Wizard will automatically generate SQL for processing or the user may create his own SQL statements

An Array-Oriented (APL) Wish List, Jim Lucas. Several proposals for extensions to APL are proposed, including event objects, ragged arrays, enumerated classes, rank, pad and fill operators, and 'where' a new control structure.

Contacts

If you are unable to view the conference website, please contact Ed Shaw, Program Chair, at eshaw@eventra.com or Devon McCormick, Publicity Chair, at devon@acm.org or (212) 529-6139 by phone.

Sustaining Members' News

HMW Systems Ltd.

It's a while since anybody from HMW got around to writing something for the Sustaining Members News. It's not because we have nothing to say, just that we have been busy.

Partially, the cause of our state of rushing about (wearing our "Blue-Arsed Fly suits" as Marc Griffiths used to say) is mildly embarrassing – we have actually been working on and installing new users for our DOS based system. Yes folks, I did

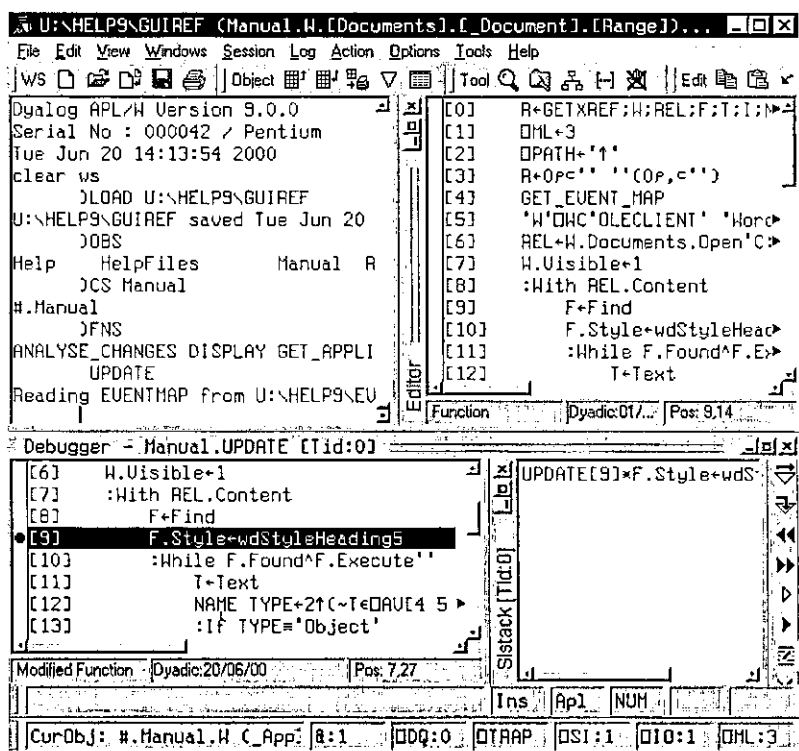
say DOS. A client (who shall remain discretely nameless) has spent 3 years attempting to replace an extremely old APL*PLUS DOS system running on 486 PCs with a state of the art C++, running under Unix on Sun Sparc machines with an Ingress Database. The week before going live they finally let the users test it – who simply pointed out one or two shortcomings (it may have been more), including the fact that the APL system managed to run about 50 times faster (and that bit is not a joke). Combined with the need to replace a couple of other systems due to Y2K problems, this lead to not only the continuation of our DOS system, but an expansion of our user base.

We have not entirely neglected the 21st century. Our other products have been developed in slightly more modern versions of APL (not that I am criticising DOS based APLs – anything that earns that much money for our company is all right by me). We have had our first sales of Inca (Software Change Management for APL systems) and Maya (a development and coding environment for APL) in the last year. Completing our South / Meso-American theme we are finalising Aztec, an APL task environment capable of running real-time jobs, scheduling batch tasks, automatic restart and recovery, etc., all those things normally associated with the less exciting, but still vital end of software development – namely making it run smoothly in a live situation. I'm sure that when we started naming this range of products we had grand visions of the significance of the names. Think of all that cyclopean Inca masonry fitted so neatly together – what an image for software management, piecing together all those little changes, or an Aztec pyramid lifting its temple out of the jungle, just like a task scheduler would raise our (small but elegant) application from the liana shrouded hinterlands of the operating system. However I'm equally sure that a certain peculiar sense of humour is resident at "HMW Towers" (as one client nicknamed our modest offices) – things like "May 'Ah show you something I've been working on" probably had as much to do with the names as a did a coherent marketing strategy.

As to the future: We hope to persuade our clients to move completely to an Aztec-run version of 4XTRA (how much longer can DOS go on?). We have become members of the Dynamic Systems Development Corporation, an organisation with many members much larger than us (IBM, ICL, BT to name but 3) dedicated to promoting a RAD methodology – we see a lot of synergy with both APL and our current work (but that could take quite a while to explain and could well be the topic of a talk to the APL group, if anyone is interested). We have a lot of ideas which are now coming to fruition under the Aztec environment – so you might not have to wait so long to here more news from us next time!

dyalog APL

The Definitive APL for Windows™



Version 9 - an even better IDE(A)

<http://www.dyadic.com>

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to: Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.

DynArray	DICE for Windows	poa	Software development kit which includes an APL Interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APs; TIME facility
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Systems Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
Iverson Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$95	
	J Standard Edition	\$295	
	J Professional	\$895	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	
	J User Manual	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Concrete Math	\$40	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads \$10 each		
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also

	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL.68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL.68000 Level II Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
Oasis	Dyalog APL	poa	Dyalog Systems
	APL*PLUS	poa	Manugistics
	APL.68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyalog and ISI products	poa	
	USA		
	Dyalog and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.
APL Group (see Eventra)			

APL Software/Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.holt@juno.com.
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability. Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win.
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+WIN version and later a LINUX version.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/1 users. It comes with RO/LAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
HMW	4XTRA	poa	Front End and Middle Office Foreign Exchange and Money Market Dealing System
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.

I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson. Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
INFOSTROY	APL*PLUS/Xbase Interface (11/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives you r Windows applications all advantages of DLLs.
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark <i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>
JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	AP127/PC	poa	ODBC interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP Interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
	AP747	poa	ODBC Interface for Dyalog APL/W
	QWIN	poa	Legacy DOS Windowing support for APL+Win
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +I v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.

	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL*Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer; APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job

Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
George MacLeod	Consultancy	poa	Design and programming of new APL applications. Enhancing and maintaining existing APL applications. Porting existing APL applications from one APL system to another. Supporting users of APL applications. Experienced on both mainframe, UNIX and PC APL interpreters.

Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Elitis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling - mainframe to PC, performance tuning.
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to APL-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Weighahead Systems	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees

APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL	FF350 (private) FF2800 (Company)	
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qtlly SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Marienthoe 88, 25451 Quickborn, Germany. Tel: +49 4106 60977 Fax: +49 4106 67889 Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
Adaytum Limited	Douglas Rowley	Castlegate, Tower Hill, BRISTOL BS12 0JA. Tel: 0117 921 5555 Fax: 0117 922 7749. Email: sales@adaytum.co.uk
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788365 Email: apl385@compuserve.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnengeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Software Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cais.com
Ajay Askoolum	Ajay Askoolum	42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737-771643 Email: ajay@askoolum.freeserve.co.uk
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interlog.com
BACUS	Joseph De Kerf	Rooienberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2836; Fax: +1 (215) 886-4888 Email: BeautifulSystems@goffs.net
Carnacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@tesco.net
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
Ian Clark	Ian Clark	1, Heifer Mill Cottages, Mosterton, Beaminster, Dorset DT8 3HG, England. Tel: +44 (0)7931 370304. Email: earthspot@aol.com
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@codework- it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark

Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 7002820 Fax: +358 9 7002824 Email: dinosoft@dinosoft.fi
Oitrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Evestic AB	Olle Evero	Bertellusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mailbox.swipnet.se
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwala	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwala@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749328 Fax: 01622 749365 E-mail: martin@gssoft.freeserve.co.uk
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-6290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email: HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028 Email: bob.hoekstra@khamlin.demon.co.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept REN/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 +1 (408) 463-2752 Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexei Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: ailm@infostroy.spb.su
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559 Email: info@jsoftware.com
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buld.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Hochsteingasse 13, 8010 Graz, Austria. Email: joachim.hoffmann@gmx.at
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.nfl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo_allegro@mail.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.1v., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: akinloch@globalnet.co.uk
George MacLeod	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ, UK. Tel: 01442-831385 Fax: 01442-831388 Email: george.macleod@simcorp.com
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX, UK. Tel: 0121-359 5096. Fax: 0121-359 0375

MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	4215 St-Andre, Montreal, CANADA H2J 2Z3. Tel: +1 (514) 529-1434 Email: danb@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01790-263843 Email: Ellis@mtfirm.demon.co.uk
NY Sig	David Siegel	PO Box 2697; New York, NY10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Theo Zwart, Louis Rijkse	Lokstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: info@oasis.nl or rijkse@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grendel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	1st Floor, 10 Raleigh Court, Priestley Way, Crawley, West Sussex, RH10 2PD, UK. Tel: 01293 562700 Fax: 01293 562699 Email: mailbox@optima-systems.co.uk
Qualedi Inc.	Ann Wilkie (Marketing) Ed Shaw (Technical)	Merritt Crossing, 440 Wheelers Farms Road, Milford, CT 06460, USA. Tel: +1 (203) 882-9988. Fax: +1 (203) 882-9891 Email: awilkie@eventra.com; eshaw@eventra.com
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@aplbooks.cnhost.com or orders@aplbooks.cnhost.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Laurie Howard	Soliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@soliton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5081, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (952) 470-7345 Email: sales@software.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulfhielm	Novator Consulting Group AB, Svårdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org

Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK. Tel +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
Adaytum Limited	www.adaytum.com/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/rb/apljourn.htm
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
COSY (Bob Armstrong)	www.cosy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de/ ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Hoekstra Systems	www.khamsin.demon.co.uk/about/hsl/noframe.html
IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.insight.dk/infostroy/
Insight Systems ApS	www.insight.dk/
Iverson Software Inc	www.jsoftware.com/
Japan APL Association	www.naska.co.jp/JAPLA/
KJK-tieto Oy	www.kjk-tieto.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	www.mackaykinlochltd.co.uk/~admk/
MicroAPL Ltd	www.microapl.co.uk/
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk

Qualedi, Inc.	www.qualifiedi.com
Renaissance Data	www.aplbooks.cnchost.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.jssoftware.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

ZARK Newsletter Extracts

introduced by Jon Sandles

Limbering Up: Combinations

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

The combination function `!` is one you probably don't use often. It tells you the number of different ways you can select `M` coloured jelly beans from a set of `N` jelly beans. (And it even works on other objects!) Each jelly bean has a different colour.

For example, suppose you have 5 beans, coloured Black, Green, Purple, Red, and Yellow. How many different combinations can you make?

```
3!5
10
```

Ten combinations. Here they are:

```
BGP  BPR  GPR  PRY
BGR  BPY  GPY
BGY  BRY  GRY
```

```
B=Black  R=Red
G=Green  Y=Yellow
P=Purple
```

Your task is to write a function that allows you to construct a set of different combinations. Call it *COMBIN*, and give it the same syntax as the `!` function. Given the left argument `M` and the right argument `N`, it returns a matrix of indices with `MIN` rows and `M` columns.

```
3 COMBIN 5
1 2 3
1 2 4
1 2 5
1 3 4
1 3 5
1 4 5
2 3 4
2 3 5
2 4 5
3 4 5
```

To get the answer to the jelly bean problem, use the expression,

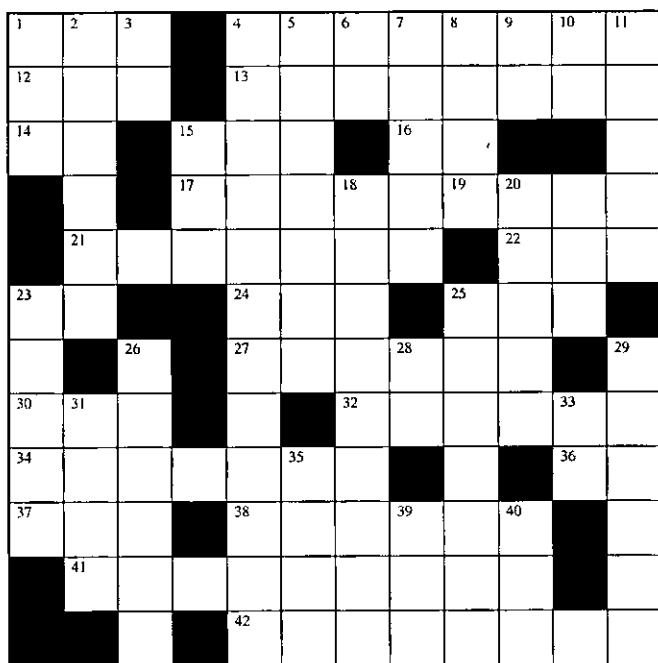
```
'BGPRY'[3 COMBIN 5]
BGP
BGR
BGY
BPR
BPY
BRY
GPR
GPY
GRY
PRY
```

Seem simple enough? Try your hand at writing this function. Naturally, efficiency is important. However, elegance (whatever that is) and originality will also be acknowledged.

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA

Solution and New Crossword

Crossword



Across

1. $(\sim V \in N) / V$
4. The elements of vector C that satisfy condition A1 or condition B
12. The characters used to represent the number KC
13. How many of the leading numbers in the list V are greater than 99?
14. $\square_3$ returns the names of all existing functions
15. Alternating sum of B
16. The larger of S or ...
17. Index of the first row of M that contains vector V
21. $T[I] - + / I \uparrow mrv$ for I from 1 to ρT (origin 1)
22. $S \times (7 \div S) - 17 \div S$
23. $3 \ 4 + . * 2$

24. $AA[(1 + \rho AA) - 1 \rho AA]$ in origin 1
25. The 256 different characters
27. How many elements in the same-length vectors VT and V are the same?
30. Pick a number from 1 to 10
32. $((E \times E) \times \neg 1) + V5$
34. $(A \times X \times 2) + (B \times X) + C$, all scalars
36. $(5 \ 10 + . \times 3 \ 2) \times .5$
37. The remainder of M when dividing by S
38. One of the two system commands that can increase $\square WA$ when the state indicator is empty
41. Rotate vector D to the left P places; stick R on the front of it; grab the first M elements; and reassign R
42. A constant is not _____

Down

1. Format N to V decimal places
2. $(T5 \neg KL) = \square IO + \rho T5$
3. A has a value if $2 = \square _ 'A'$
4. Drop the trailing elements of vector V that equal B
5. $(B \setminus A) / MAT$
6. Lawrence Wall: "A __ a two and a ..."
7. The bits used to drop from V (via /) the characters before substring S
8. The greater of BV or N
9. In addition to $\} \geq \} \supset$ are __
10. ... replicates 9
11. 17's, C9 of them
15. $\& ' - (+ '$
18. Attach vector VCR to the left of matrix R, and add VRA to the result
19. The APL developer's second-to-last system command at the end of the day
20. $(\square IO - 1) ++ \setminus ((V \times V) \times .5) \rho 1$
23. 20 different random numbers from the set $\neg XS$
25. Does the number entered equal the alternating sum of the vector $(S + A)$, $S \uparrow A$ (for positive S)
26. Subtract the array $T0AM$ from itself, reshape the result to have shape $\neg 0$, and reassign it
28. $++ / 6 \ 9 \ 3$
29. All of the vector ZE but its first 125 numbers
31. The sum of the magnitude of the numbers in the vector R
33. e
35. $(\times / \rho ERA) \rho ERA$
39. Application program interface (abbr.)
40. $(\neg ' ') \rho '' 'EASY' 'DOOR' 'BOLT'$

Solution to Crossword in 17.3

+	≠	V	o	.	=	ι	Γ	/	V		Φ
/		ι	.	5	+	N		ι	×	/	S
v	/	B	≤	5			A	ρ	B	S	
/		+		ε		T	~			-	N
φ	ρ	M		V	-	ι	V		(		E
0	ρ	A			N	N	V	ε	<	ι	G
v		T	?	1	2	5		A	ι		A
.	2		2				0		0	1	T
≠	/	M		→	(	A	ε	V	)	↓	I
M	ι	ι	/	M		φ	U	S	ρ	φ	V
	1	3		×	V		+	/	..	N	E
1	0	0	↑	T	1	↓	V	E	C	T	

J-ottings 28: Just say it in J

by Norman Thomson

An early claim for APL was that the exactness demanded by expressing mathematics in an executable language could lead to significantly greater understanding of underlying algebra and algorithms than was achievable by conventional mathematical notation. Subsequent experience diluted this claim somewhat since the obscurity which even a few lines of code could generate outweighed the inherent discipline of using a programming language as a learning tool. Nevertheless there are some topics in which the expository power of APL, and now even more strongly J, trounces traditional methods. One of the most conspicuous examples of this is Analysis of Variance.

The acronym ANOVA, which is at least as well known by its abbreviation as in its full form, is one of the curiosities of statistical terminology, since it is not a single technique, it is not primarily about variance, and by no stretch of the imagination can it be called a process of analysis. I surmise that many generations of students have tackled ANOVA as an arcane piece of algebra involving mysterious "correction factors", seemingly arbitrary rules concerning denominators, and mantras about "degrees of freedom", "interactions" and "error sums of squares". Yet if ANOVA is seen as series of transformations to numbers which are then later formalised in algebra, then both its rationale and the algebra itself become plain. I am not trying to underestimate the vast scope of the intellectually demanding science of Design of Experiments, but rather to suggest that its foundations might be more clearly presented to beginners through the medium of J. I imagine that if ANOVA were to be reinvented today it would probably be called something like SSP (standing for Sum of Squares Partitioning).

The insight to be obtained from using J does of course assume competence in that language, so that as a starting point the following basic verbs together with the adverb **each=. &>** can be taken for granted:

<code>mean=.+/ % #</code>	NB. Mean of vector
<code>dev=. -mean</code>	NB. Deviations from mean
<code>ss=.+/@*:</code>	NB. Sum of squares of a vector
<code>ssm=.ss@dev</code>	NB. Sum of squares about the mean

First tackle one-way ANOVA which is relevant where there are groups of numbers, not necessarily of equal size, but of sufficient general similarity for it to be interesting to speculate on whether the groups are distinguishably different, or whether any variability present is scattered across all the groups. In order to keep

the calculations checkable by mental arithmetic the values used in the following examples are integers. A set of groups of numbers is modelled by a list of lists such as

```
data1=.1 2 3 6 8;4 8 3;3 4 9 5
mean each data1
4 5 5.25
```

A common sense way of assessing whether this result shows real differences between groups is to hypothesise no variability within each group, and smooth out any such variability by replacing every value by its group mean. J obliges with the primitive verb *copy* which, operating as the centre tine of a fork, makes just as many copies of the numbers as are necessary:

```
((#each) # (mean each))data1
4 4 4 4 4 5 5 5 5.25 5.25 5.25 5.25
```

(There are redundant parentheses in the above expression in order to make the fork structure stand out.) The sum of squares of these values about the overall mean is totally free of any within-group variation and so, rationally, is a sound measure of the between-group variation:

```
bss=.ssm@(#each # mean each) NB. Between group sum of sqq
bss data1
3.91667
```

For variability *within* groups, a corresponding procedure is to sum the individual sums of squares about the mean within each group:

```
wss=.,/ @:(ssm each) NB. Within group sum of sqq
wss data1
68.75
```

Given the above results, any subsequent analysis is clear cut, viz. the within group variability overwhelms the between group variability. As a consequence of algebra, which can be investigated separately later, it happens that the sum of *bss* and *wss* necessarily equals the sum of squares about the mean which would have arisen with the same set of numbers if there had been no group divisions in the first place. Using *raise in*, this is:

```
tss=.ssm@; NB. Total ssq
tss data1
72.6667
```

Hence

```
aov1=.bss;wss;tss
aov1 data1
```

NB. one-way ANOVA

3.91667	68.75	72.6667
---------	-------	---------

At this point it is useful to consolidate the verbs which describe this first stage in understanding ANOVA :

```
bss=.ssm@(#each # mean each)  NB. Between sum of sqq
wss=.+/@:(ssm each)           NB. Within sum of sqq
tss=.ssm@;                     NB. Total sum of sqq
aov1=.bss;wss;tss              NB. 1-way ANOVA
```

Try them out again with another example:

```
d2=.6 2 3;2 8 5;5 6 8
aov1 d2
```

10.6667	31.3333	42
---------	---------	----

In this case there are the same numbers of items within each group, and so items could also be classified by position within group. One way to do this is to define markers for rows and cols (so named because `>d2` is a matrix):

```
rows=.3#i.3
cols=.9$i.3
rows
0 0 0 1 1 2 2 2
cols
0 1 2 0 1 2 0 1 2
```

In deciding to proceed in this way, the model has subtly changed its character to one where there is on the one hand a single list of values with no inherent defined structure, and on the other a collection of value-free data which defines possible structures within the data proper. (Anyone who has heard a statistician talking about a "design matrix" will see this concept beginning to emerge.)

Using row markers, the result of `aov1` is obtained by using the *key* adverb qualifying the *from* verb for the purposes of selection, together with a redefinition of the data in a structure-free form :

```
data2=.;d2
group=.{/.
rows group data2
```

6	2	3	2	8	5	5	6	8
cols group data2								
6	2	5	2	8	6	3	5	8

The between groups sums of squares can be found exactly as before:

```
bss rows group data2
10.6667
bss cols group data2
2
```

For convenience the verbs `bss` and `group` can be combined as

```
sq=.bss@group NB. sum of sqq for single design vector
rows sq data2
10.6667
```

A further extension to this verb allows the sums of squares to be adjusted simultaneously for any number of design vector(s) as left argument:

```
sqq=.(sq each)< NB. sum of sqq for several design vectors
(rows;cols)sqq data2
10.6667 2
```

The means of these subgroups can be obtained conveniently by a similar small extension to the `mean` verb:

```
means=.mean each@group
rows means data2
3.66667 5 6.33333
cols means data2
4.33333 5.33333 5.33333
```

By developing `dev` each data-item can be transformed into a deviation from the mean of the group to which it belongs, with *ravel* applied in order to restore it to single vector form:

```
deveach=.,@(group(-each)means)
rows deveach data2
2.33333 _1.66667 _0.666667 _3 3 0 _1.33333 _0.333333 1.6666
7
```

The definition of `deveach` is not quite complete since it is desirable that the deviations are restored to their positions in the original data vector. Restoring the

right order involves using an inverse permutation obtainable through two applications of the *grade-up* verb :

```
restore=.:@/:@[ { ]
deve=.[ restore,@(group -&> means)
cols deve data2
1.66667 _3.33333 _2.33333 _2.33333 2.66667 _0.3333333 0.66666
67 0.6666667 2.66667
```

The total sum of squares is

```
sq data2
42
```

A useful check can be made here since the sums of squares of the row-mean adjusted data:

```
sq rows deve data2
31.3333
```

is just the difference between the total sum of squares and the between-rows sum of squares.

Suppose now that this data had a further "hidden" classification based on a so-called Latin square design whereby a third factor (treatment) was incorporated using the matrix

```
0 1 2
2 0 1
1 2 0
```

to overlay the data in a row by column layout.

```
latin=.0 1 2 2 0 1 1 2 0
latin sq data2
24.6667
latin means data2
7.33333 4 3.66667
(rows;cols;latin) sqq data2
10.6667 2 24.6667
```

So far 37.333 out of the total of 42 sums of squares have been accounted for. What remains is the sum of squares of the residuals, often called the error sum of squares, which is the sum of squares after the various adjustments have been made in succession:

```
sq latin deve cols deve rows deve data2
4.66667
```

The form of the above line invites generalisation to an *insert* adverb:

```
>(deve&.>)/(latin;cols;rows;data2)
0.6666667 _1 0.3333333 _1 0.3333333 0.6666667 0.3333333 0.666
6667 _1
```

The items in the above vector are the result of successive row, column and treatment smoothing; squaring and summing them leads to a definition in J of the residual sum of squares:

```
rss=.sq@>@(deve&.>/@(, <))
(latin;cols;rows) rss data2
4.66667
```

and so the complete partitioning, including the total sum of squares, is given by

```
aov2=.sqq,rss,sq@]
(latin;cols;rows)aov2 data2
24.6667 2 10.6667 4.66667 42
```

The order in which the items in the left argument appear is, of course, immaterial.

The property that the *tail* of the result is the sum of the *curtail* (or in plain English the last value is the sum of all the others!) depends on **orthogonality** in the design. For example a design vector which consists of an arbitrary arrangement of three 0s, three 1s, and three 2s such as

```
x1=.1 0 2 2 0 1 0 1 2
```

does not possess this property:

```
(x1;latin;rows)aov2 data2
2.66667 24.6667 10.6667 4.37037 42
```

nor, more generally, do designs which, although balanced, are not orthogonal. However, such considerations go far beyond the scope of this elementary article which seeks to do no more than break the ground for on the one hand *analysis* of what such partitioned sums of squares can say about variance, and on the other, extension to increasingly complex structures. This is an appropriate point to consolidate the verbs which are part of the second stage of expounding ANOVA :

group={/.	NB. .. by design vector
sqq=(sq each)<	NB. Between sum of sqq
sq=.bss@group	
means=.mean each@group	NB. Group means
rss=.sq@>@(deve&.>/@(<,>))	NB. Residual sum of sqq
deve=[restore,@(group(-each)means)	
restore=.:@/:@[{]	
aov2=.sqq,rss,sq@]	NB. Multi-way ANOVA

The above development involves many J primitives – what is striking is that each of them corresponds to an operation which is a logically necessary step for understanding the underlying process of ANOVA in an abstract way. It could also be argued that for someone who already has a clear understanding of ANOVA, the foregoing discussion might lead towards a fuller appreciation of J!



Software * Support * Education

WWW.APLBOREALIS.COM

Canada's APL Products and Services Provider!

- Distributors for *Dyalog APL* and *APL+Win*
- Expert *APL* Software Development and Support, all platforms -- Specialists since 1973
- *APL* Training – Customized hands-on courses; introductory to advanced levels including *APL* system design and Windows *APL*

381 Manor Road East
 Toronto Ontario M4S 1S7 Canada
 tel: 416.457.7828 fax: 416.482.6582
 email: info@aplborealis.com

The Puzzle Corner

*by Stefano Lanzavecchia (stf@apl.it)
and Jonathan Manktelow (jonathan@causeway.co.uk)*

Introduction

Learning by playing, something kids do all the time and adults often dismiss as an inefficient way of tackling new challenges. I would love to have material for a puzzle corner to be run regularly, because I truly believe puzzles are a great way to enjoy oneself while exploring and learning. Instead of rewriting the Centigrade to Fahrenheit converter for the hundredth time, when trying a new feature of a known language or a new language altogether, why not take a simple puzzle and solve it in the new environment? The blinking cursor beside the correct solution is quite a rewarding sight and the techniques so learned are then ready in our brains to be applied to more serious matters.

For this issue I'll present a new puzzle that recently swept onto the UUNet newsgroups and an article with a new solution to the Funny Cube of Mr Legrand. While the language used for the latter isn't exactly an Array Programming Language, the author, who is also an APLer, raises a few interesting points.

The Vier-Neun Puzzle

On March the first, Steve Graham posted the following message in a few discussion groups related to programming languages:

This puzzle was originally posted on a mailing list for the Icon programming language. Thought members of this group might also want to give it a shot.

VIER and NEUN represent 4-digit squares, each letter denoting a distinct digit. You are asked to find the value of each, given the further requirement that each uniquely determines the other.

The "further requirement" means that of the numerous pairs of answers, choose the one in which each number only appears once in all of the pairs.

Before you read more I suggest you spend a minute figuring out a possible solution strategy, even if you don't have the time to write the code: it will make what follows clearer.

First of all it's easy to see that a brute force approach is viable since the pairs of perfect squares to be examined are less than 5000.

At the time of my writing this summary, on the web page where Mr Graham is collecting the solutions to the problem [members.home.net/js.graham/vierneun.html], almost 40 entries are listed, in a wide range of languages (APL, Dylan, Forth, Haskell, Icon, J, K, LISP, MUMPS, OCaml, Perl, Prolog, Python, Smalltalk, SNOBOL4, SPITBOL, Tcl) and I am aware of other unpublished solutions, like at least one in A+.

It would not be easy for me to review all the solutions, because I don't understand all the languages that were used, nor would it be appropriate since Vector is devoted in particular to APL and friends. But I still think it's interesting to spend some time looking at different worlds because there's always a new trick to be learned.

Let's start with APL. The first APL solution to appear was provided by Morten Kromberg, who has since revised his solution multiple times, thanks to an inspiring discussion with Roger Hui, until it reached this final version:

```
r←NS2;m
[1] a New Scientist problem, Dyalog APL with Rogers improvements
[2]
[3] a 4624x8-column char matrix NEUNVIER
[4] r←+,r←.,r←+4 0*(31+168)*.*,2
[5] a Must have same pattern as NEUNVIER or 12315628
[6] r+({(w1w)''+r)εc{w1w}'neunvier')+r
[7] a Unique entries in each group of 4 columns
[8] r+(>^/({(w1w)=(1+pW)-(qW)1W+1w)''(8p4+1)<r)≠r
```

Next comes Roger Hui's submission, at the same time elegant and efficient. Let's read his post, where the strategy is clearly explained step by step:

The 4-digit squares are (>. %:1000) squared to (<. %:9999) squared, or, as a character matrix,

```
s=: 4" : , . *:32+i.68
4{.s
1024
1089
1156
1225
_4{.s
9216
9409
9604
9801
```

To form all pairs of such squares, catenate each row to all other rows, and make into a matrix, thus:

```
p=: ,/ , "1 "1 _ ~ s
$ p
4624 8
4{.p
10241024
10241089
10241156
10241225
_4{.p
98019216
98019409
98019604
98019801
```

The pattern of digit assignments must be the same as the pattern of letters in 'vierneun', and so:

```
a=: ((i.~'vierneun')-:"1 i.~"1 p)#p
$a
9 8
a
13694624
13695625
17645625
43561521
47615625
62419409
70561521
75691681
75694624
```

Finally, choose numbers that are unique in all the pairs. An item is unique if its index of first occurrence is the same as its index of last occurrence. Thus:

```
((4{."1 a} *.&(i.~ = i:~) (4)."1 a))#a
62419409
```

Collecting together the essential lines:

```
s=: 4":,.*:32+i.68
p=: ,/ , "1 "1 _ ~ s
a=: ((i.~'vierneun')-:"1 i.~"1 p)#p
((4{."1 a} *.&(i.~ = i:~) (4)."1 a))#a
62419409
```

Forming all pairs is linear in the size of the result; all other operations are linear in the size of the argument(s).

It was easy for me to translate Roger's idea in rough K:

```
s:4$(32+!68)^2
p:./s,\:/:s
a: p @ & ({x?/:x}"vierneun")~/{x?/:x}'p
k:{*:'x@&1=#:'x:=x}'+{(4#x;4_ x)}'a
a k[0] @ &k[0] _in\: k[1]
, "62419409"
```

David Ness quickly provided a generalization to try other pairs of German numerals that have four letters, thanks to the fact that the condition which asks for each letter in the words "vier" and "neun" to be assigned to one digit is so cleanly expressed. In K:

```
skp: z, 'z:{"eins";"zwei";"drei";"vier";"funf";"acht";"neun"}
num: num @ &(#z)=skp ?/: num:./z ,\:/:z

dj:([arg] `0:"\n",arg,"\n"
a: p @ & ({x?/:x}arg)~/{x?/:x}'p:./s,\:/:s:4$(32+!68)^2
if[0=#k:{*:'x@&1=#:'x:=x}'+{(4#x;4_ x)}'a;:{arg;-1}]
`0:t:"\t",'a k[0] @ &k[0] _in\: k[1]
{arg;#t})
```

Typing

```
dj'num
```

will result, among other things, in a list:

```
{("zweieins";2)
("dreieins";2)
("viereins";6)
("funfeins";2)
("achteins";0)
("neuneins";-1)
...
("zweineun";1)
("dreineun";1)
("vierneun";1)
("funfneun";-1)
("achtneun";1)}
```

which indicates each "problem" and the number of solutions found for that particular problem. The difference between '0' and '-1' is that 0 represents places where there are solutions but none of them satisfy the 'single/single' restriction. '-1' occurs when there are no solutions even relaxing this assumption.

John R. Clark provided a more traditional APL solution:

Since we know they are 4 digit squares we can generate all possible values and start to do an elimination.

Let V be the universe and A be the possible values for NEUN

```
A←((V[;2]≠V[;3])∧(V[;1]=V[;4]))#V+⌈((ρV),1)ρV+(33+199)*2
```

By removing duplicates from V we can obtain possible VIER values

```
V←(NODUPS"=[2]V)#V
```

If we cut out the values of V that fail we have a solution

```
A,CUT"=[2]A
```

```
1521 4356
```

```
7056
```

```
1681 7569
```

```
4624 1369
```

```
7569
```

```
5625 1369
```

```
1764
```

```
4761
```

```
9409 6241
```

```
Z←NODUPS X
```

```
[1] Z←1=⌈/+/X°. =X
```

```
Z←CUT X
```

```
[1] Z←((V[;1]≠X[1])∧(V[;2]≠X[2])∧(V[;3]≠X[3])∧(V[;4]≠X[4]))#V
```

```
[2] Z←(1+⌈/+/Z°.X)#Z
```

Koji Kawakami has submitted this function, to be run on Sharp APL. Please notice line [6] which is the direct equivalent of Roger's assignment to p.

```
r←spuzzle;t
```

```
[1] r all possible 4 digit combinations
```

```
[2] r←'214' ⌈fmt (4624 2)ρr(∘.,{rank}0)r←(31+168)*2
```

```
[3] r a rough sieve
```

```
[4] r←((r[;3]=r[;6])∧r[;5]=r[;8])#r
```

```
[5] r pattern matching
```

```
[6] r←('vierneun'ε{on})#(rank)1 r)[;1]#r
```

```
[7] r unique pair only
```

```
[8] r←((v+{1=+/t})#t+←r[;1 2 3 4])∧v+{1=+/t})#t+←r[;5 6 7 8])#r
```

```
[9] r←1 1 1 1 0 1 1 1 1\ r
```

Just for fun, I tried to craft a one-liner in the functional style John Scholes from Dyadic uses to build his exemplar dynamic functions. More an exercise in style than in code elegance or efficiency, here it is:

```

vierneun+{ a Solve the generalised New Scientist's Vier-Neun problem
  { (4+[2]w){ a enforce uniqueness
    (▷^/{(w!w)=1+(pw)-(φw)w}•+"α w)/[1]α,w
  }~4+[2]w
  }{ a remove the debris
    †(x▷•p~w)/w
  }(α,w){ a test the combination for the proper pattern
    ,w•.({(w!w)α}{αα={w!w}α,w)/α,w})w
  }{ a generate all the squares and convert them to text
    +4 0†,[0](32+168)•2+□10+0
  }□ML □10+1
}

```

The left and right arguments are the two patterns, in our case 'vier' and 'neun'. The solution is again pretty much a rephrasing of Roger's J solution. It is more easily understood if read from bottom to top.

We can readily calculate the solutions to the generalised problem (forgive the missing umlaut on "funf" which allows for more solutions to be generated):

```

gn+'eins' 'zwei' 'drei' 'vier' 'funf' 'acht' 'neun'
all+gn •.vierneun gn

```

Then we can count them:

```

▷~p~all
36  2  2  6  2  0  0
2 36 10  3  1  0  1
2 10 36  2  1  0  1
6 10  2 36  1  0  1
2  1  1  1  5  1  0
0  0  0  0  1 36  1
0  1  1  1  0  0  5

```

... verify that the solution to the "vier-neun" case is the one expected:

```

all[←gn:'vier' 'neun']
62419409

```

... and finally extract all the pairs for which the problem has only one matching pair of squares:

```

↑[0.5](,1=>p"all")*/"(,all){,gn .. gn)
37219409 zweifunf
62419409 zweineun
37219409 dreifunf
62419409 dreineun
[...]
62419409 neunvier
94093721 neunacht

```

For those of you who can never get enough, here's the table built from the French numerals:

```

fn+'zero' 'deux' 'cinq' 'sept' 'huit' 'neuf'
=>p"allf+fn ..vierneun fn
36 2 0 2 0 2
2 36 0 2 5 8
0 0 36 0 5 6
2 2 0 36 0 2
0 5 5 0 36 5
2 8 6 2 5 36

```

... and the Italian ones:

```

=>p"alli+in ..vierneun in+'zero' 'otto' 'nove'
36 0 4
0 0 0
4 0 36

```

I will spare you the Danish, the Swedish and the Japanese, but before we give up, let's have a look at solutions in other (programming!) languages.

First, the cryptic Perl solution due to Bruce Hoult for which I have no comments:

```

for $a(32..99){b:for $b(32..99){
    @cnt=();
    for(1..8){$cnt[substr($a*$a.$b*$b,$_-1,1)].=$_]
    for(0..9){next b if(sort{$b<=>$a}@cnt){$_}!=(58,36,7,4,2,1){$_]}
    $a{$a}++;$b{$b}++;$p{$a}=$b
}}
while(($a,$b)=each%p){print$a*$a," ",$b*$b,"\n"if$a{$a}*$b{$b}==1}

```

Then the revealing Smalltalk solution, courtesy of Reinout Heeck:

```
| squares neuns pairs tallies results |

squares := (1000 sqrt ceiling to: 9999 sqrt truncated)
           collect: [ :n | n squared printString ].
neuns := squares select: [ :string |
                        string first == string last
                        and: [string asSet size==3]].
pairs := OrderedCollection new.
tallies := Bag new.
squares do: [ :square |
  neuns do: [ :neun |
    ((square at: 3 )==(neun at: 2)
    and: [(square,neun) asSet size = 6])
    ifTrue: [
      pairs add: square -> neun.
      tallies add: square; add: neun ]]].
results := pairs select: [ :pair |
  (tallies occurrencesOf: pair key) == 1
  and: [(tallies occurrencesOf: pair value) == 1]]
```

The code yields:

```
results =
  OrderedCollection ('6241'-'>'9409')
```

This solution has many features in common with the Dylan and the Python solutions: they make heavy use of standard library collection objects. While in APL and J (and to a certain extent K) there is only the *array* (with the powerful *dyadic iota*, *shape*, *reduction primitives*), Smalltalk's (as well as Dylan's or Python's) library provides the developer with objects such as the *Set* (an unordered collection of unique objects), or the key-value *pair*. The authors decided to make use of these features and, at the same time, hard-code the pattern "vier-neun" (the first digit of "neun" must match the last, "vierneun" has only 6 unique letters and so on) in the search code and not to extract the signature of the pattern and apply it in a more general way. Anyway, with the help of the library objects, many submissions while not as compact as the APL, J or K entries, are still very compact even for languages traditionally considered verbose.

Solutions where matching against a general pattern is tried can be found on the reference web page in the Icon, LISP and, if I interpret the source code correctly, in the SNOBOL section. I wish I could read the solutions in Forth.

One thing that many authors did not try was to provide a completely coded solution, that is a solution that took into account the further requirement of uniqueness of both elements of the pair. After having generated the possible pairs matching the pattern, they left the last task of extracting the items uniquely identifying the pairs to the human.

Another interesting point is that the large majority of the languages listed earlier are interpreters with an interactive environment in which the solutions can be built step by step by trial-and-error or scripting languages that don't need compilation.

Finally, from Oleg Kobchenko, a little pearl "to restore the decency to *conventional languages*" (his words). A direct translation in C of Roger's algorithm, which shows "how a language like J can serve as a great prototyping tool".

```
#include <stdio.h>
#include <string.h>

void main(void)
{ int i, j, k, n=0; wchar_t p[1000], v[]="vierneun", s[9];

  for (i=32;i<=99;i++) for (j=32;j<=99;j++) {
    swprintf(s,L"%04d%04d",i*i,j*j);
    for (k=0;k<8;k++) if (wcschr(s,s[k])-s != wcschr(v,v[k])-v) break;
    if (k==8) { p[n++] = i*i; p[n++] = j*j; } }
  for (i=0;i<n;i+=2) if (wcschr(p,p[i]) == wcsrchr(p,p[i])
    && wcschr(p,p[i+1]) == wcsrchr(p,p[i+1])) break;
  printf("%04d%04d\n",p[i],p[i+1]);
}
```

The Funny Cube in Delphi

[by Jonathan Manktelow]

But Delphi isn't an Array Language!

Whilst Delphi (or to be more accurate Object Pascal) is an Object Orientated/Procedural language it can handle arrays of arbitrary dimensions and sizes. It does lack a lot of the array manipulation primitives in APL etc., however it is a very expressive and powerful general-purpose language. It is also fast. Very fast. Both in terms of compilation and execution speed.

So why try the funny cube in Delphi – surely APL would be better?

After seeing the funny cube puzzle in Vector Vol.16 No.3 I started to wonder if the problem was really an array manipulation problem or not. Obviously the pieces of the puzzle can be represented as arrays, but it turns out that the most intuitive approach to solving the puzzle (to me at least) is to recursively pick up a piece, put it into the next position [A,B,C etc] and see if it fits. If it does try the next piece,

otherwise try a different piece. In other-words this is a recursive problem. Now Object Pascal (the language of Delphi) is an Object Orientated procedural language, and well suited to recursive problems. So I thought I would put together a Delphi solution to be compared to the APL ones already published in Vector. [Please note – I did not read any of the solutions until after writing my code.]

Solving the puzzle

Since this is the Journal of the British APL Association, and not a Delphi magazine, I will just compare my solution to those in APL, rather than trying to explain how the code works. The source code and executable are both available on the Vector website.

Development speed. This took one very relaxed evening in Delphi, and would have taken me about the same in APL. This reaffirms my belief that developing programs in a specialist language such as APL is no faster than developing in a good general-purpose language.

Execution speed. For the simple case (no flipping allowed) the execution is 3ms. If you allow the pieces to be flipped then it takes 37ms on my 266Mhz Pentium laptop. These times are averaged over 1000 consecutive runs, as the basic Windows timer only has a resolution of 10ms. These results are nothing short of spectacular when compared with the fastest APL solutions from Vector Vol.17 No.2 which were 100ms for the non-flipping and 734ms for the flipping case on a PII 233Mhz machine.

Conclusions

Having solved the problem in Delphi and read the various APL solutions, I am certain that the fastest way to solve the problem would be using the 6 bits of foam that made up the original puzzle (suitably labelled so that you can track which piece is which, and their rotations).

[ed] You can find the full text of Jonathan's Delphi solution on the Vector website.

XML in Dyalog APL Version 9

by Jonathan Barman

Introduction

There has been a tremendous amount of hype about XML being the “the ASCII of the future”. Initially I was suspicious as all seemed rather pointless. Having seen how and where XML is used the penny has finally dropped. Clearly separating the data, using XML, from the display, using HTML, seems to be a very sensible idea. Another of the ways in which XML is used is to transfer of data between completely independent programs and machines. The transfer is normally on the Web but can be between any machines networked together. Having a clear, self-documenting, medium when one is trying to write a program to pick up data from another source is obviously a great help. I was interested in the Web site www.xmlrpc.com which contains a specification for using XML for remote procedure calls and which appears to be implemented on several platforms.

This article is about the way in which XML can be manipulated using Dyalog APL version 9 with the new-style object syntax.

XML Basics

XML is a simple way of structuring data. It looks very like HTML but the tag names are whatever you need to define the data. The general idea is to make it self documenting. Tag names should describe the data so that anyone looking at the XML can easily see what is going on.

XML is fussier than HTML in that all tags must be closed and the nesting must be correct. If you are familiar with HTML then XML is easy to create. For example, an APL array could be defined as follows:

```

<array>
  <rank>
    <dimension>2</dimension>
    <dimension>3</dimension>
  </rank>
  <data>
    <number length="3">0 1 2</number>
    <array>
      <rank>
        <dimension>2</dimension>
      </rank>
      <data>
        <char length="2">AB</char>
      </data>
    </array>
    <number length="2">4 5</number>
  </data>
</array>

```

The `<array>` element has two elements `<rank>` and `<data>`. The `<rank>` element contains `<dimension>` elements which specify the shape of the array. The `<data>` element contains elements which specify the data type of the value. A data type can be `<number>`, `<char>` or an `<array>` for a nested array. The `<number>` and `<char>` elements have an attribute `length` specifying the length of the data each contains.

Whilst this simple scheme shows how easy it is to define data structures in XML it is not adequate to fully describe an APL array. For example, the character data element may need to cater for any character out of `⎕AV`. Also it might be necessary to precisely define empty arrays with full prototype information.

If you have never seen XML before the tutorial site www.xml101.com contains some good examples and clear explanations.

XML Document Object Model

Microsoft distribute an XML parser with Windows, `MSXML.DLL`. This has been upgraded for Windows 2000 to version 2.6 which is in `MSXML2.DLL`. If you have installed `.NET` there is yet a further upgrade to `MSXML3.DLL`. I have used `MSXML` version 2.6 for all the examples because I am running under Windows 2000 but, hopefully, I have not used any of the new features.

The base object of the parser is the Document Object Model (DOM). The main thing that this contains is the Root Element of the XML. The Root Element contains collections of Nodes. A Node within the collection may be one of several types which include Attributes and Elements.

Dyalog APL and MSXML

The latest version of Dyalog APL, 9.0, includes a major improvement in the way COM objects such as MSXML can be accessed. In order to show off the power of the new version code will be developed in both 8.2 and 9.0 so that a direct comparison can be made between the two version.

Both versions access the basic DOM object in the same way:

```
'xDOM'⊂WC 'OLEClient' 'MSXML2.DOMDocument.2.6'
```

The above is only valid if you are running under Windows 2000. Otherwise the second argument would be 'MSXML.DOMDocument'. I find that it is always best to check the exact spelling by looking at the array produced by

```
a←Ⓜ''.⊂WG'OLEServers'
```

There are two methods for loading XML into the DOM; *load* which loads from a file and *loadXML* which loads a character vector. Again, both 9.0 and 8.2 have the same syntax:

```
xDOM.load 'Example1.xml'
```

1

The 1 returned indicates that the XML has loaded without error. If an error occurs then the error information is held in the *parseError* property. In this case versions 9.0 and 8.2 are different:

Dyalog 8.2

```
xDOM.loadXML xmlbad
0
'xPer'⊂WC xDOM.parseError
xPer.line,xPer.reason,xPer.srcText
7 The character '<' cannot be used in an attribute value.

<number length="3">0 1 2</number>
```

Dyalog 9.0

```
xDOM.loadXML xmlbad
0
xPer←xDOM.parseError
xPer.line,xPer.reason,xPer.srcText
7 The character '<' cannot be used in an attribute value.

<number length="3">0 1 2</number>
```

In Dyalog 9.0 you can assign objects to variables, so you don't have to use `⎕WC`. This facility is fully extended throughout the language so that `:with` arguments can also be an object rather than a character vector giving a namespace path. It also allows the dot syntax that is standard in other programming languages, so it is possible to access the reason for the error with `xDOM.parseError.reason`.

The following functions illustrate the way in which XML might be loaded and the return code checked.

Dyalog 8.2

```

v r←LoadXML xml;xp
[1]   :If ~r←xDOM.loadXML xml
[2]       :With 'xp'⎕WC xDOM.parseError
[3]       #.MsgBox'XML Error'((⍶line),' ',reason,srcText)
[4]       :EndWith
[5]   :EndIf
v

```

Dyalog 9.0

```

v r←xd LoadXML xml
[1]   :If ~r←xd.loadXML xml
[2]       :With xd.parseError
[3]       #.MsgBox'XML Error'((⍶line),' ',reason,srcText)
[4]       :EndWith
[5]   :EndIf
v

```

In version 8.2 you have to refer to a global DOM object whereas in version 9.0 it can be passed into the function. Also, in version 9.0 the `:with` creates an anonymous namespace from the object.

Convert XML to an Array in Dyalog 9.0

Converting the example XML into an APL array is relatively straightforward in Dyalog 9.0. With help from the Editor, who made some very clever suggestions, I created a single function with all sub-functions defined using direct definition.

It needs to be emphasised that this function will only work using Version 9.0 release 2 which allows arrays of references. The version I used was dated 7 March 2001.

```

v r=XMLToArray xml;Array;Length;Load;Shape;Value;xDOM; IO;XML
[1]  IO=0 o XML+1
[2]  a Convert xml into an APL array.
[3]  a Returns 2 element vector (0 array) or (n errmessage)
[4]  Load+{      a Load xml (w) into DOM (a). Return 0 for OK
[5]      r=a.loadXML w a loadXML returns 1 for OK
[6]      o=r:1,c.a.parseError.('XML Error on line ',(i line),'. ',reason,srcText)
[7]      'array'#a.documentElement.nodeName:2,c'XML does not
          contain an <array> root element'
[8]      0}
[9]  Array+{      a Get array from root node w
[10]      s+Shape w
[11]      o=vl+w.selectNodes'data/number|data/char|data/array':0
[12]      o=vl.length:0
[13]      sp>Value/vl.(item`phi length),c0}
[14]  Shape+{      a Get shape from root node w
[15]      o=dm+w.selectNodes'rank/dimension':0
[16]      o=dm.length:0
[17]      dm.(item`phi length).(=>(//)VFI text))
[18]  Value+{      a Value node a catenated to vector in w
[19]      s+Length a
[20]      'number'#a.nodeName:w,s+(//)VFI a.text
[21]      'char'#a.nodeName:w,s+a.text
[22]      'array'#a.nodeName:w,cArray a
[23]      w}
[24]  Length+{      a Length attributed in value node w
[25]      o=lg+w.selectSingleNode'@length':0
[26]      o=0,(//)VFI lg.text}
[27]  'xDOM'OWC'OLEClient' 'MSXML2.DOMDocument.2.6'
[28]  :If 0=>r=xDOM Load xml o :Return o :EndIf
[29]  r+0,cArray xDOM.documentElement
v

```

The *Load* function, defined on line [4] and called on line [28], is much the same as defined above but I wanted a zero return code to indicate success. It checks that the XML has loaded correctly and also checks that the root node has the correct name. Line [6] shows off the way in which the new namespace syntax can now include arbitrarily complex statements enclosed in parentheses between the dots.

The DOM object actually contains a collection of child nodes, which will be explained later on. The *documentElement* property (referenced on line [7] and [29]) contains the root node. There can only be one of these otherwise the XML would not have loaded correctly. *Array* is therefore called with the root object as its right argument.

The *Shape* function, called on line [10] and defined on line [14], uses the *selectNodes* method on line [15] to get the dimension nodes. where the argument '*rank/dimension*' is an XPath expression which specifies the path to the required nodes. If something goes wrong then the method returns NULL so the variable *dm* is tested to see if it contains a scalar zero. *dm* is not an array, it is a

Selection Object containing the nodes found. Line [16] checks the *length* property to see if there are any dimensions and then line [17] uses yet more complex expressions to convert the *text* property of each node into numbers. Before the Editor pointed out this new feature I was much mystified why statements such as *xRoot.childNodes.(item i).nodeName* gave a *VALUE ERROR*. I correctly guessed that the variable *i* was being masked on the way down the object hierarchy, but did not realise that this was deliberate or how useful it could be.

Back to the *Array* function on line [11] the nodes containing data are selected with a set of paths separated by an OR (|) symbol. The XPath documentation in MSDN is listed under XSLT which I did not find until I had virtually finished this article; it is well worth careful study if you are going to manipulate XML. Line [13] is thanks to John Scholes where the reduction operator is used to place the *Value* function between each pair of nodes to catenate all the results together. Because reduction operates from right to left, but we want to work from left to right, the indices are reversed.

The *Value* function is therefore dyadic taking a node as its left argument and the data to be catenated as its right argument. The *Length* function selects the length attribute by putting an AT (@) sign in front of the name. Lines [20] to [22] convert the data and catenate it to the right argument. Line [23] is not really necessary as the function can only be called with nodes of the correct names.

Running *XMLToArray* produces the following results.

```

SE.DISPLAY XMLToArray xml
┌───┐
│ 0 ─┐
│ │ 0 1 2 │
│ │ │ │ │
│ │ ─┐ ─┐
│ │ │ AB │ 4 5 │
│ │ │ │ │
│ │ ─┐ ─┐
│ │ │ │ │
└───┘
ε

SE.DISPLAY 2>XMLToArray '<array></array>'
.0.
|0|
|~|

```

```

x+ '<array><data><number length="1">23.1'
x,+ '</number></data></array>'
□SE.DISPLAY 2>XMLToArray x

```

23.1

```

x+ '<array><rank><dimension>3</dimension></rank>'
x,+ '<data><array></array></data></array>'
□SE.DISPLAY 2>XMLToArray x

```

```

┌-----┐
│ .0. .0. .0. │
│ |0| |0| |0| │
│ └─┘ └─┘ └─┘ │
└-----┘

```

Whilst developing this code I came across another change in Version 9.0 that caused me some puzzlement. `)FNS` and `)VARS` no longer report the COM object's methods and properties as they do in Version 8.2. Stopping the code on line [17], for example, to check what is going on gives the following:

```

)cs dm
#.xDOM.[IXMLDOMNode].[IXMLDOMSelection]
)FNS
)VARS
)METHODS
clone    getProperty    matches    nextNode    peekNode
removeAll    removeNext
reset    setProperty
)PROPS
context    expr    item    length    AutoBrowse    ChildList
ClassID    ClassName
Data    Event    EventList    Handle    HelpFile    Instance
Mode    KeepOnClose
LastError    Locale    MethodList    PropList    QueueEvents
nts    Type    TypeList
NODE_DOCUMENT
9
NODE_DOCUMENT+32
SYNTAX ERROR
NODE_DOCUMENT+32
^
□NC'NODE_DOCUMENT'
0

```

As can be seen the enumeration variables such as `NODE_DOCUMENT` are not listed but are available and can't be assigned. The values can be displayed in APL Explorer where one can find the name of the property that contains them.

Convert XML to Array in Dyalog 8.2

It is more difficult to organise the recursion in version 8.2 as a character argument of the namespace name has to be passed around and the recursion has to rely on the namespaces created by `WC`. Also, because all the namespaces have to be explicitly created it is not so easy to use direct definition for all the sub-functions as in version 9.0, and everything is much more verbose.

```

▽ r←XMLToArray xml;xDOM;ns;IO;ML
[1]   IO←0 ∘ ML←1
[2]   ns←'xDOM'WC'OLEClient' 'MSXML2.DOMDocument.2.6'
[3]   :If 0≠ns Load xml ∘ :Return ∘ :EndIf
[4]   :With ns
[5]       :With ns+'xRoot'WC documentElement
[6]           :If 'array'≠nodeName
[7]               r←0,c#.GetArray ns
[8]           :EndIf
[9]       :EndWith
[10]  :EndWith
▽

```

The `Load` function does not check the name of the root array as a root namespace would have to be created twice. Otherwise it is the same as Version 9.0 with the addition of an explicit namespace to generate the error message.

```

▽ r←ns Load xml;xP;IO;ML
[1]   IO←0 ∘ ML←1
[2]   :With ns
[3]       :If r←loadXML xml
[4]           :With 'xP'WC xDOM.parseError
[5]               r←1,c'XML Error ',(▼line),' ',reason,srcText
[6]           :Return
[7]       :EndWith
[8]   :EndIf
[9]   :EndWith
▽

```

The `GetArray` function first gets the required shape and then gets the data. The processing is really exactly the same as in Version 9.0, but namespaces have to be created and explicit loops have to be coded.

```

v r←GetArray ns;i:len;shape;[]IO;[]ML
[1] []IO←0 o []ML+1
[2] r←0
[3] shape←GetShape ns
[4] :With ns
[5] 'vls'selectNodes'data/number'data/char'data/array'
[6] :If 9≠[]NC'vls' o :Return o :EndIf
[7] :With 'vls'
[8] :For i :In vlength
[9] 'vl'item i
[10] :With 'vl'
[11] len←0
[12] 'lg'selectSingleNode'@length'
[13] :If 9≠[]NC'lg'
[14] len←len+ε0,~(//)[]VFI lg.text
[15] :EndIf
[16] :Select nodeName
[17] :Case 'number'
[18] r,←len+ε(//)[]VFI text
[19] :Case 'char'
[20] r,←len+text
[21] :Case 'array'
[22] r,←c#.GetArray'[]NS''
[23] :EndSelect
[24] :EndWith
[25] :EndFor
[26] :EndWith
[27] :EndWith
[28] r←shapepr
v

```

To check if the select methods return a NULL I have used []NC on lines [6] and [13] to test if an object was created.

The *GetShape* function contains very similar code.

```

v r←GetShape ns;i;[]IO;[]ML
[1] []IO←0 o []ML+1
[2] r←0
[3] :With ns
[4] 'dms'selectNodes'rank/dimension'
[5] :If 9≠[]NC'dms' o :Return o :EndIf
[6] :With 'dms'
[7] :For i :In vlength
[8] 'dm'item i
[9] r,←ε(//)[]VFI dm.text
[10] :EndFor
[11] :EndWith
[12] :EndWith
v

```


and no other data. The ? following the names means that there may be zero or one occurrences of each.

A child entity name may be followed by nothing, which means that it must occur exactly once, a query (?) if it is to occur zero or once only, a plus (+) if there must be one or more occurrences, or a star (*) if it can occur zero or more times. Putting the decoration after the enclosing parentheses means that it applies to all of them.

The declaration dimension (#PCDATA) means that there may be data between the start and end dimension tags.

Specifying child entities separated with vertical bars (|) means that any may occur in any sequence. The data element has been defined in this way.

Attributes are defined with !ATTLIST element-name attribute-name attribute-type default-value. The attribute types and attribute defaults may be as follows:

Attribute Type	Meaning
CDATA	Character data
(value value ...)	One of the given values must be included
ID	An unique Id
IDREF	The Id of another element
IDREFS	A list of other Ids
NMTOKEN	A valid XML name
NMTOKENS	A list of valid XML names
ENTITY	An entity
ENTITIES	A list of entities
NOTATION	A name of a notation
Xml :	Predefined

Attribute Default	Meaning
#DEFAULT value	If the attribute is not present then the value is assumed
#REQUIRED	An attribute value must be included
#IMPLIED	The attribute is optional, but no default is available
#FIXED value	The attribute always has the same value regardless of the value specified in the XML

The example DTD has defined the default attribute values in double quotes rather than using #DEFAULT. For example, the number attribute could have been defined as !ATTLIST number length CDATA #DEFAULT 1.

It is a good idea to include a DTD because otherwise you have to carry out much more checking in the code. For example, without a DTD it is permissible to include data between the array tags.

```
1 xDOM.loadXML '<array>data</array>'
```

But, with a DTD the parser recognises that this is not permitted.

```
0 xDOM.loadXML dtd, '<array>data</array>'
```

```
0 xDOM.parseError.reason
```

Text is not allowed in this context according to DTD/Schema.

Expecting: rank, data.

Another useful facility is to control white space. The XML parser normally removes white space which means you don't actually get what is in the XML. Defining character arrays can be quite difficult if spaces are removed in some arbitrary way. The `xml:space` attribute ensures that spaces are preserved for character data, but this can be overridden if you really don't need it. The following shows the effect on spaces.

```
x+ '<array><rank><dimension>20</dimension></rank>'
```

```
x, + '<data><char length="20">      A      B  </char>'
```

```
x, + '</data></array>'
```

```
[]SE.DISPLAY 2>XMLToArray x
```

```
-----
```

```
| A      B |
```

```
-----
```

```
[]se.DISPLAY 2>XMLToArray dtd, x
```

```
-----
```

```
|      A      B |
```

```
-----
```

Another way of dealing with spaces is to use the CDATA definition which can be used anywhere PCDATA is expected.

```
x+ '<array><rank><dimension>20</dimension></rank><data>'
```

```
x, + '<char length="20"><![CDATA[      A <> B  ]]>'
```

```
x, + '</char></data></array>'
```

```
[]SE.DISPLAY 2>XMLToArray x
```

```
-----
```

```
|      A <> B |
```

```
-----
```

Writing XML

I don't actually like using the MSXML parser to create XML. The XML is not formatted and is spewed out on a single line. The object hierarchy tends to force one into loops which can be difficult to replace with APL arrays. As far as I can see the main advantage in using MSXML is where existing XML needs to be edited and then saved. MSXML is full of useful methods such as `replaceChild`, `removeChild`, `insertBefore` etc. etc.

First of all then is a function to convert an array into XML without using MSXML. The function is identical in versions 8.2 and 9.0.

```

VArrayToXML←{⎕I0←0 ⋄ ⎕ML+1 ⋄ ⎕PP+16
[1]      cr←2>⎕TC
[2]      array←{ a Generate array
[3]          ind←ap' ' a Indent
[4]          top←ind,'<array>',(ind rankpw),cr,ind,'<data>',cr
[5]          tail←ind,'</data>',cr,ind,'</array>',cr
[6]          (w)←0 1:top,(ind values,w),tail
[7]          a Mark contiguous groups of scalars
[8]          b←(1,(1+w)*~1+w)a+0=,w
[9]          top,(ε((b/a),,a)data"bc,w),tail}
[10]     rank←{ a Generate rank information
[11]         θ←w:''
[12]         cr,α,'<rank>',cr,α,' ',(εdim" w),cr,α,'</rank>'
[13]     dim←{ a Format a dimension
[14]         '<dimension>',(w), '</dimension>'
[15]     data←{ a Append values or array
[16]         r i←α a Recurse flag and indent
[17]         r:(ip' ')values w
[18]         (i+2)array" w
[19]     values←{ a Deal with heterogeneous data
[20]         {w^,=w}⎕DR" w:α, ' ',(value w),cr
[21]         ε(α){α, ' ',(value w),cr}" w
[22]     value←{ a Format values
[23]         82=⎕DR w:'<char length="',(wρ,w),'">',(trans w), '</char>'
[24]         '<number length="',(wρ,w),'">',(w), '</number>'
[25]     trans←{ a Translate reserved characters
[26]         c←5×b+<>'"'⌈a←,w
[27]         a[c/1ρb]+<lt;' '>' '&' '&quot;' '&apos;' [c/b]
[28]         ea}
[29]     a Return result. dtd is global
[30]     dtd,0 array w
[31] }
```

v

I would not normally include `dtd` as a global, referenced on line [30], but it makes the function listing shorter and the DTD has already been listed above.

Running the function on our sample XML produces the following:

```
(pdttd)+ArrayToXML XMLToArray xml
<array>
  <rank>
    <dimension>2</dimension><dimension>3</dimension>
  </rank>
  <data>
    <number length="3">0 1 2</number>
    <array>
      <rank>
        <dimension>2</dimension>
      </rank>
      <data>
        <char length="2">AB</char>
      </data>
    </array>
    <number length="2">4 5</number>
  </data>
</array>
```

Creating XML using MSXML

If you need to use MSXML then the following functions show how to set about the task. Also, there are several characters in `<math>\text{XML}</math> which seem to give MSXML a bit of a headache when reading XML so it is interesting to see what MSXML does when generating the XML.`

The easiest way to start is to load a bit of XML into an empty DOM object, in fact this appears to be the only way to define a DTD.

```
'xDOM'<math>\text{WC}</math>'OLEClient' 'MSXML2.DOMDocument.2.6'
xDOM.loadXML dtd,'<array></array>'
1
```

This actually loads 3 nodes, the first being the Processing Information, the second the DTD and the third the root node.

```
xDOM.childNodes.length
3
xDOM.childNodes.(item"(\length)-<math>\text{IO}</math>).(nodeName nodeType)
xml 7 array 10 array 1
```

The `nodeType` numbers can be checked against the property that contains the enumerated variables.

<i>xDOM.tagDOMNodeType</i>	
<i>NODE_INVALID</i>	0
<i>NODE_ELEMENT</i>	1
<i>NODE_ATTRIBUTE</i>	2
<i>NODE_TEXT</i>	3
<i>NODE_CDATA_SECTION</i>	4
<i>NODE_ENTITY_REFERENCE</i>	5
<i>NODE_ENTITY</i>	6
<i>NODE_PROCESSING_INSTRUCTION</i>	7
<i>NODE_COMMENT</i>	8
<i>NODE_DOCUMENT</i>	9
<i>NODE_DOCUMENT_TYPE</i>	10
<i>NODE_DOCUMENT_FRAGMENT</i>	11
<i>NODE_NOTATION</i>	12

If you don't load the initial XML in this way then you have to set the root node explicitly. This is easy in Dyalog version 9.0, but quite nasty in version 8.2.

Dyalog 9.0

```
'xDOM'[WC'OLEClient' 'MSXM      xRoot+xDOM.createElement 'array'
xDOM.documentElement+xRoot
```

Dyalog 8.2

```
'xDOM'[WC'OLEClient' 'MSXML2.DOMDocument.2.6'
'xRoot'[WC xDOM.createElement 'array'
2 [NQ'xDOM' 'SetPropertyInfo' 'documentElement' 'VT_DISPATCH'
'xDOM'[WS'documentElement' 'xRoot'
```

Pete Donnelly kindly told me how to get this to work in version 8.2 and explained that 8.2 does not know anything about the *IXMLDOMElement* which the *documentElement* property is expecting, so you have to tell it using 2 [NQ.

Creating XML using MSXML in Dyalog version 9.0

The following function creates XML using MSXML in Dyalog version 9.0. For a change of scenery the sub-functions are direct definition operators. In all of them the left function is the DOM object's *createElement* method and the right function is the node to be processed.

```
v r←ArrayToXMLms a;x;rt:sink;Array;Data;DataMixed;Dim; Rank;Recurse;
Value;Values;[IO;[ML;[PP
[1] [IO←0 ∘ [ML←1 ∘ [PP←16
[2] Array←{      a Append array node to ww
[3]     sink+(aa Rank ww)pw
[4]     x+aa'data'
[5]     sink+(aa Data x).w
[6]     x+ww.appendChild x
[7]     0}
[8] Rank←{      a Create rank node
```

```

[9]      @=pw:0 % not required for a scalar
[10]     x+aa'rank'
[11]     sink+(aa Dim x)"w
[12]     x+ww.appendChild x
[13]     0}
[14]   Dim+{      % Create dimension node
[15]     x+aa'dimension'
[16]     x.text+vw
[17]     x+ww.appendChild x
[18]     0}
[19]   Data+{      % Append values & arrays to data node
[20]     (w)ε0 1:=(aa Values ww)w
[21]     % Mark contiguous groups of scalars and arrays
[22]     b+{1,(1+w)ε-1+w}a+0="w
[23]     =>(b/a)(aa DataMixed ww)"b=w}
[24]   Values+{    % Deal with heterogeneous data
[25]     {w^.=w}82=□DR" w:(aa Value ww)w
[26]     =>(aa Value ww)"w}
[27]   Value+{     % Append values to data
[28]     x+aa(82=□DR>w)>'number' 'char'
[29]     sink+x.setAttribute'length'({p,w)
[30]     x.text+vw
[31]     x+ww.appendChild x
[32]     0}
[33]   DataMixed+{ % Data is simple or array
[34]     a:(aa Values ww)w % simple
[35]     (aa Recurse ww)"w} % array
[36]   Recurse+{   % Recurse on an array
[37]     x+aa'array'
[38]     sink+(aa Array x)w
[39]     x+ww.appendChild x
[40]     0}
[41]   % Create DOM and load initial node. dtd is global
[42]   'x'□WC'OLEClient' 'MSXML2.DOMDocument.2.6'
[43]   sink+x.loadXML dtd,'<array></array>'
[44]   sink+(x.createElement Array x.documentElement)a
[45]   r+x.xml

```

▼

I don't want *ArrayToXMLms* itself be a direct definition function because on line [42] there is a call to □WC which in direct definition mode results in a global variable being created. If there happened to be a variable *x* lying around in the Workspace then the call to □WC would result in a *DOMAIN ERROR*. Also, you would have to remember to)ERASE *x* when the function finished.

The calls to the *appendChild* method (line [6] is the first) return the address of the child node, which has been updated to include a reference to its parent. The documentation states that if something goes wrong a NULL is returned rather than an address. Ideally one should check the result to see if the operation has been successful. Setting attribute values seems too easy, as line [29] demonstrates. The results are not particularly pleasing as the XML is not formatted into lines, but it

all seems to work correctly. It is interesting to see how MSXML copes with the full `⌈AV` and exactly which characters it finds difficult.

```
XMLToArray a←ArrayToXMLms ⌈AV
0
  {(\\' <array>' ⍉w)/w)a
<array><rank><dimension>256</dimension></rank><data><char length="256"></char></data></array>
```

This shows that MSXML ignores everything after `⌈AV`. Using the character version shows that it is no good putting `⌈AV` into the XML either.

```
XMLToArray ArrayToXML ⌈AV
1 XML Error on line 0. The following tags were not closed: array, data, char.
```

Removing the first element from `⌈AV` at least processes everything but there are still some characters it finds difficult.

```
>b+XMLToArray a←ArrayToXMLms 1+⌈AV
0
  ((2>b)≠1+⌈AV)/1+⌈p⌈AV
4 12 220 236 239
```

Creating XML using MSXML in Dyalog version 8.2

I have not attempted to create the equivalent function using MSXML in Dyalog 8.2 because the character version is perfectly adequate and version 8.2 is quite tricky. Direct definition would not be applicable, but otherwise the general scheme would be much the same as the 9.0 version. The only thing that might not be immediately obvious is how to call the *appendChild* method. The following shows the general approach.

```
'xDOM'⌈WC'OLEClient' 'MSXML2.DOMDocument.2.6'
xDOM.loadXML dtd,'<array></array>'
1
  'xRoot'⌈WC xDOM.documentElement
  'xRank'⌈WC xDOM.createElement 'rank'
  3 ⌈NQ'xRoot' 'appendChild' 'xRank'
#.appendChild #.newChild
  'xrc'⌈WC xRoot.childNodes
  'c0'xrc.item 0
  xrc.c0.nodeName
rank
```

Conclusion

XML seems to be quite simple to manipulate with MSXML and Dyalog APL. Version 9.0 is a great improvement over 8.2, but while more tedious, 8.2 is perfectly capable of carrying out the task.

J – the Natural Language for Analytical Computing

As this edition goes to press, we learn that Norman Thomson, who has been a regular contributor to Vector through his J-ottings articles, is about to publish a book with the above title, which should be available at or shortly after the time of APL2001.

There are four parts to the book. The first part aims to present a gentle but persuasive lead-in to the serious scientific reader, and make the case for why he/she should adopt J as a basis for solving computational problems. The emphasis in this part is on parts of speech and the relationships with how these are used in natural languages, leading on to the importance of the various forms of composition. The second part, Advanced J, carries on from there, and is intended for those who have started in J but would like both to feel more secure in it, and also better able to take advantage of the full range of facilities. Separate chapters are devoted to control words, gerunds and their application, and to the foreign, the dot, power conjunction and rank conjunctions.

The final two parts are addressed to the two broad classes of people for whom J offers both practical advantages and appeal, namely those who carry out data manipulation and those who apply mathematics. The chapter headings in part 3, which is called Data Processing with J, are Selection, Data Structuring, Ranking and ordering, Searching, finding and updating, Partitioning, Opposites (that is inverse, obverse, etc.), and Programming Principles. Those in part 4, Mathematics with J, are Complex Numbers, Number bases and Polynomials, Series, Calculus and the D adverb, Numerical Analysis, Randomness, Permutations and Combinations, Groups and Symmetries, and Logic.

Each chapter concludes with a set of examples to which full solutions are given, and those phrases in the text and exercises which have possible general usefulness are consolidated in an Appendix.

While this is a beginners book in the sense that is intended to try and attract the attention of those of scientific or mathematical inclination but with no prior J knowledge, it is by no means a "dummies" book, since it also aims to demonstrate how quickly progress can be towards achieving significant computations through the variety and richness of the high quality algorithms which are provided in the J interpreters.

The publishers are Research Studies Press who recently formed an association with American Technical Publishers Ltd. RSP specialise in producing academic and professional monographs in growing areas of science, technology and computing, and it is hoped through this publication in one of their series, J may achieve wider recognition in the scientific community as a versatile tool for personal computing.

A full review will appear in the next edition of Vector.

Finland Has It All

Observations on the FinnAPL "Forest Seminar" held in Malvik, Kirkkonumi near Helsinki 8th and 9th February 2001 by Ray Cannon.

"You're so near to Russia,
So far from Japan,
Quite a long way from Cairo
A lot of miles from Vietnam"

(Quote taken from the song "Finland, Finland" available on "Monty Python's Contractual Obligation Album" CAS 1152, 1980 Full lyrics:
www.tm.informatik.unifrankfurt.de/~andreas/finland/finland_song.html)



Wednesday 7/2/2001

As I was preparing to fly to Finland for the seminar, I read that the temperature two days before had been minus 26C with a wind chill factor of minus 40C! But Tomas (Tomas Gustafsson of FinnAPL) reassured me with "Current temperature around zero degrees centigrade, 10 centimetres of snow...but we will stay in civilised areas".

The flight from Gatwick to Helsinki was uneventful, other than that I saw (for the first time, although I remember reading about it as a child) a completely circular rainbow, with the shadow of the aircraft in its middle cast on the clouds below.

Helsinki airport was pleasant, and efficient. My complete lack of any knowledge of the Finnish language was not a problem. Within no time I was in a taxi going round "Ring III" on my way to Majvik. All the roads were clear, that is until we turned off the road into the conference centre, where for the last kilometre, we drove on snow.

Tomas had made all the arrangements, and I booked into the hotel with the very minimum of fuss. I was staying in one of the outer buildings, next to the shore of what I took to be a lake. I was later informed it was actually the sea. It's the first time I have seen reeds growing in the sea, but if the salt content is low enough then maybe! The room was warm, well equipped, and if it were not for the mist which restricted the vision to about 500m would have had a great view.

That evening, I met up with Gitte Christensen, Morten ("may the force be with you") Kromberg and Tomas in the bar. Later, after supper, Klaus Klug Christiansen joined us. Since he had missed the meal he was supplied with two bananas which had been liberated from somewhere. I stayed in the bar, (which was very lively) until about midnight, hoping to see John Daintree, but somehow I missed his arrival.

Thursday 8/2/2001

Up for breakfast at 8:30, porridge, juice and cold meat. Met up with John who was slightly unnerved as he was the first speaker of the day.

At 9am, the seminar was opened promptly by Arto Juvonen – FinnAPL chairman. Following a (very) few opening remarks he introduced the first speaker John Daintree, chief Windows/GUI architect of Dyadic Systems Ltd.

John's first (of two) talks was an introduction to what had been called Dyalog version 10, but was currently being called Dyalog.NET. (Who knows what it will be called by the time it is released.) We are hoping John will be writing up his talk for Vector.

After coffee and sandwiches, Arto introduced the next speaker Richard Eller, of RE Time Tracker Oy.

Richard gave a talk (in Finnish and English) on a case study of a large APL application for handling orders, sales and accounting at a major Finnish energy and fuel supplier. The system, written in APL2000+WIN, handles a network of trucks, keeping track of petrol (gas) deliveries to petrol stations, optimising the routes, with full regards to security, and guaranteed time scales. It handles over one billion Finn Mark (£1,000,000 approx) a year.

If the system receives a request in the morning, it would hope to provide a petrol delivery that same day, and that night receive payment via a direct debit (DD). Problems covered include that petrol prices vary during the day and petrol is bought and sold by volume, which varies with temperature. The APL system does it all including the submission of DD requests over the internet.

The system is maintained by 10 people, who are able to fix most problems over the phone. Due to the very large amount of money/petrol being traded, the system is held on a mirrored data-base with "LOTS" of back-ups!

(Richard did say he would try to write up his talk in more detail for Vector.)

The last speaker before lunch was Morten Kromberg of Insight Systems ApS. The APL supermarket in Denmark!

Morten talked about a system call SOFIA (used by most Italian insurance companies). First written in 1985 (for 16-bit APL), rewritten in 1990 in 32-bit APL+PLUS II and transferred to Dyalog APL/W in 1998 (last DOS system due to be phased out in 2002). GAMA the Russian cousin of SOFIA was written in 1995 under Dyalog APL.

SOFIA has become the dominant product in the Italian insurance market! All the largest insurance companies are customers.

Morten provided two candidates for "quote of the day":

"Insurance companies are in business to take money from people, keeping it as long as possible, and giving back as little as possible";

and

"There is always a bigger fish" (Star Wars, Episode 1);

But I quite like this one (albeit taken out of context):

"... allows operator .. to kill users...." <grin>

Lunch was excellent. The only problem was that the buffet-style starter salad was so good, (and they had not put out the main course when we first arrived) some of us (i.e. me) over-indulged, thinking that it was the main course, there being at least four types of herring. However, when the main course arrived (I had the "Trout Loaf") it was delicious, leaving no room for the sweet or coffee.



After lunch Anssi Seppälä (of Enease Oy) started giving the first part of a talk in Finnish. (Tomas gave the second half.)

Unfortunately, since Finnish (according to Klaus) is "Chinese spoken in Italian" all I had to go on was the supplied note (Attacking the big ones. Penetrating the electricity distribution markets with APL software. The "Electricity Quality and Load" concept) and the overhead foils (also almost all in Finnish).

Anssi talked (I think) about taking the first steps in developing a new application "METERING". This project would include both the hardware and software.

The concept of "Electricity Quality and Load" (EQL) was demonstrated with a slide showing two images, the first of a happy meter plugged into a wall socket, with a smiling PC; the second showed a meter running a temperature with an ice-pack on its head, and a red-faced PC looking very sick "ejecting" a floppy disk.



Tomas Gustafsson (Weis Impactor Oy) then took over and then continued in English. Tomas is working with Anssi on the EQL project, Anssi writing in J and Tomas in APL.

Tomas showed a Windows scrolling bitmap graph. Since Windows has limits of 32k in its pixel resolution, in any dimension,

objects larger than this 32k limit cannot (using standard Windows API calls) be displayed without loss of resolution. Tomas showed that it could be done using their technique which involves trapping the "Expose Event".

Tomas then went on to tell us about how to do "Property Sheets in Seconds". Tomas showed how he has automated property sheets to a great extent, using data structures with default values and a common set of functions to manipulate property sheets. This resulted in getting a consistent look and feel, data-aware controls, pre-set and pre-filled controls with default values, pre-set controls with stored values, and "real time" updates from GUI to data.

At 3 p.m. we stopped for "Coffee and Cake". The cake, being a doughnut with pink icing, tasted much better than its colour suggested!



Olli Paavola (Pakkausalan Ympäristörekisteri PYR Oy) was next up with another talk in Finnish. He described "an APL-powered intranet/internet-enabled information system, with an approved SFS-EN ISO 9001 quality certificate (you what John?), for handling Finland's job in the packaging area within the European Union."

The system used Dyalog APL/W, dynamically generating HTML. Under the latest European Packaging and Waste law, each member country is supposed to produce relevant data. However most member countries failed to do so, but Finland, using this system, did provide the data and was cited as having provided the best and most useful form.

(All the talk about waste and recycling begs for the response "What a load of rubbish." <grin>. However this talk did not fall into this category and despite my complete lack of understanding of Finnish, I was able to enjoy it.)

The final talk of the day was given by the invited speaker Kari Höijärvi from Vaisala Oy.

Kari, an ex-Microsoft programmer, gave a really excellent talk entitled "The development and quality assurance process at MS, US"

You can read Kari's own write-up of his talk immediately after this report.

In essence Kari stated that given a set of requested features, a realistic budget, and a deadline, you won't get a quality product with all the features if you meet the deadline. To write quality code and meet a deadline will normally mean that you have to drop features.

Kari gave some excellent examples including how MS's Internet Explorer version 1 was released on time, with very few of its required features. New releases adding extra features kept appearing every few months. On the other hand MS WIN2000 was released with almost all of its original design features, very very late!

Meanwhile, Word for Windows-1 had 4 lead programmers in 5 years, and exceeded its schedule by 400% mainly because it had impossible goals.

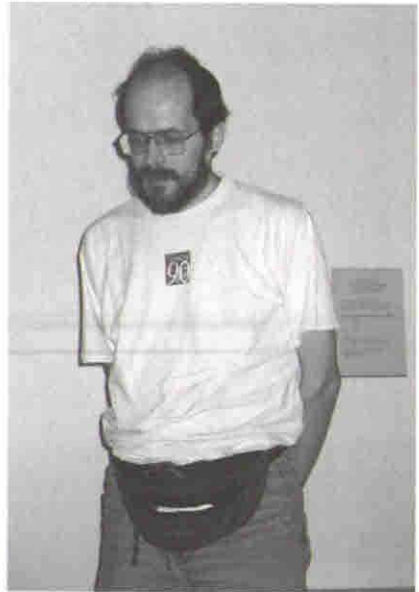
Tips

- Start a project small. For a ten man-year project, required in a year's time, start with 2 people NOT 10. Add more people after 10 weeks.
- Don't make the best developer the project manager.
- A feature is never "almost completed". It is either 0% or 100% done.
- Only release a system after "n" days of serious testing with no new bugs being found.
- Testing is a profession, and needs to be planned. Every line of code needs to be tested. (This may require tools to prove they have been tested.) The NASA style of 20 testers for 1 programmer is in the real world unthinkable, and a mere one tester per programmer is more realistic.
- Design for testability. Add code for data consistency checks, use backup algorithms. Bugs do NOT go away.

Kari then went on to tell us we were to find bugs in code:

- In Error recovery code;
- Functions that return special values in an error case;
- "Candy machine interface": is the 35 the code you enter or the price you pay?
- "Seattle Airport parking lot". Requires drivers to enter into the ticket machine, the bay "number" that their car is parked in. The bay number 41 requires the driver to enter "041" not "41".

Kari then mentioned two alternative approaches to writing quality software, the cleanroom way (<http://www.cleansoft.com>) and the Extreme Programming way which he went into in greater detail.





DINNER and SAUNA was followed by sauna barbecue and beer (Sing song beside a piano played by one of Finland's top professional J programmers).



Tuli saunasta tupahan, tuli tuntemattomana,
kasvot vallan kaunihina, poskipäät
punertavina.

When he came in from the sauna, they could
hardly recognise him,
With his face so shining bright and his cheeks
so flushing red.

Then bar, bar-pool (as in bar-billiards),
dancing, and more beer. (At some stage I was
chatted up by a very nice young lady, whose
boyfriend plays football for Luton Town.)

Friday 9/2/2001

Fog has lifted, clear view of the lake/sea.

Timo Teileri (of ABB Oy) kicked off the second day with a talk in Finnish on CoCoS and COPIS, two systems 95% written in APL.

(Sorry, no photo this morning, well there is one but it's all blurred due to an excess of beer the night before.)

CoCoS – Company COntrol System. This boasted a data-base with “modules” on component files.

COPIS – Customer Orientated Process Information System. This included a very powerful graphics editor (C.A.D.) and in addition printed output via an RTF file.

After a coffee break, a very enthusiastic Antti Korpela (Merita Henkivkuutus Oy) talked (in Finnish and English) about improving the efficiency at the largest Finnish bank conglomerate’s insurance marketing group by using APL as a modelling tool.

Antti is a mathematician, NOT a programmer, but produced some very impressive results from very simple code, including a live demo (using a loaned laptop, connected to the internet via a mobile phone) of a Dyalog workspace on a web server.

Gitte Christensen (of Insight Systems ApS) was on next, and gave a really splendid talk on the history of Insight Systems, and on the way gave some us all some tips on managing APL programmers.

She started out working for I.P. Sharp Associates in 1987, which she described as a “brainy flat organisation”. They did not bother with “internal bills” of less than \$10,000. They took pride in their product and service and strived to “Make the best product”. They had 600 employees in 50 offices.

Then, disaster struck, Reuters took over I.P. Sharp.

So in 1990 Gitte and Morten with 3 other former I.P. Sharp personnel from their Copenhagen office, formed Insight Systems where they attempted to “have fun and make money”.

There they developed “Client Server” products for their customers who were LARGE companies. They developed a no-nonsense culture, obsessed with quality.

By 1993, they were not earning quite enough money to have fun, so they merged with Adaytum, where they became Adaytum development department. Adaytum was sales and marketing focused, with mid-sized customers. A “swat” team approach, with a CMA (cover my ass) approach, “make money, and make it big!”

Adaytum grew from 8 APLers in house with 30 employees in 1995, in 1997 they moved the headquarters to the USA, and by 1998 they were heading for “IPO” (Initial Public Offering). In 1999 there were 14 APLers in-house, 27 of then in Denmark with 200 employees world-wide.

Big money, but no time to have fun. So in 1999 Insight Systems split from Adaytum, and are currently looking for "the Right Idea".

Gitte then went on to describe the life cycle of a new project. Here are a few thoughts taken out of context:

"Product Spec". The challenge is to Define the Problem, NOT the solution.

"Development Prototype". Purpose is to verify the function and its usability. The challenge is to solve the right problem.

"Documentation". If you don't want to get stuck with it, DOCUMENT it and always remember to define your ACRONYMS.

The guy who phones you needing your support, is the one who needs your product, and pays your bills. The guy who does not phone you, may not be your customer any more!

"APL is a tool of thought. If you can't think it, don't say it."

After another excellent lunch, John Daintree gave his second (and in his words, more juicy) talk, which I am hoping he will write up for Vector. In the meantime here are a few snippets.

Dyalog version 10 (as announced in Berlin) may now be called "Dyalog.NET" but this is still subject to change. With it you will be able to provide "Web Services".

"Web Services" under ".NET" can be accessed over the internet and are (.NET) language-independent. That is they can call and be called by any other ".NET" language.

- It's a SOAP/XML based protocol;
- It can be called from an Internet Browser (e.g. MS Internet Explorer 5.5);
- It can be called from an application;
- It's implemented in a ".ASPX" extension file;

The APL can be in the form of a script and can be written using Notepad.

John stated this as an aside, but we all caught on to it, and then we spent the next 10 minutes or so talking about APL fonts in non-APL programs.

What Dyadic has done is to implement an APL "Input Keyboard Method" and "Input Method Editor". This works in a Unicode environment such as NT4

Win2000 (and including Notepad under Win98SE, I know because I am using an MS supplied IKM to see Japanese, not that I can read Japanese, yet).

(For a good introduction to the topic see: http://nim.cit.cornell.edu/usr/share/man/info/C/a_doc_lib/aixprgdd/genprog/input_method.htm from which the following is an extract.)

Input Method Overview

For an application to run in the international environment for which National Language Support (NLS) provides a base, input methods are needed. The Input Method is an application programming interface (API) that allows you to develop applications independent of a particular language, keyboard, or code set. Each type of input method has the following features:

- | | |
|------------------|--|
| Keymaps | Set of input method keymaps (imkeymaps) that works with the input method and determines the supported locales. |
| Keysyms | Set of key symbols (keysyms) that the input method can handle. |
| Modifiers | Set of modifiers or states, each having a mask value, that the input method supports. |

John did give one warning of a problem he had experienced writing APL under Notepad, "You can't exit from notepad via ")OFF". <grin>

He then ran a demo of a Dyadic Web Service application, for booking rounds of golf. (Sorry no Ducks in the demo.)

At this point, my digital recorder became full, so I think we must ask John to write up his talk in full for Vector, as my hand written notes are quite frankly, unreadable! (Something about a web based spider service, Arachnid on the Net?)

After tea, (alas NO pink doughnuts) Klaus Klug Christiansen (of Array Technology ApS of Denmark) gave the final talk of the seminar, again in English.

Klaus started out by giving a potted history of the company Array Technology. Founded in 1996, got its first patent in 1997, and set up its first "array data-base" for a Swedish Electricity company. In 1998 it became a consultant to Scan Rail. In 2000, Klaus was hired as its 4th programmer.

Array Technology's main commercial product is the Array Data-base, which "sells well" ;-). This system handles "Logical Constraints", ensures consistency (that is, all rules are considered) and guarantees Completeness (all rules are obeyed).

The array data-bases are prototyped in APL. The product is then made in C and made available as either an EXE or +COM/ActiveX.

To see it in action see <http://recguide.chipshot.com/Cybrant/Velocity/> where you can use it to select a golf club!

"We created this Recommendation Guide so you can have woods that are perfectly suited to your body and game. Using your answers, we'll select clubs that match your goals, tendencies, and price range. Click the boxes at left, and your choices will appear"

Klaus then told us the secret(!) of how to build a model:

- Map overall bindings in the system
- Locate suitable primitives (building blocks)
- Implement bindings between primitives
- Build model(s) from primitives

This was followed by a case study of "Railway Security" with a small station example dealing with:

- Control of train passage
- Reservation of train routes
- Train routes excluding other train routes

This was supported with a slide showing first a pair of train routes that can exist, and then a pair that can not exist!

(I chatted to Klaus in the bar afterwards, and wondered if this system could be used to create time-tables for Schools and Colleges, but Klaus thought not. Anyway, he thought it would be far too expensive.) *You can read his own notes on the talk following this report.*

At 16:30 Arto Juvonen closed the seminar, and I retired to the bar.

That evening, Tomas took the "foreign" visitors off for a meal in a very nice restaurant (which was famous for having been the home of a very famous Finnish architect, whose name I can't remember!) I must say the food was superb. Thank you Tomas.

Saturday 10/2/2001

After breakfast, Klaus and I, who were both staying in Finland till Sunday afternoon, said our good-byes to Morten and Gitte, and left for Helsinki. Where we booked into the 5 star Hotel Kamp. (Tomas had made reservations for us at an extremely good weekend rate!)

"The original Hotel Kamp was built in 1887 in accordance with the architectural style of the grand continental hotels of its era – embodying the atmosphere of London, Paris and St. Petersburg. The original Hotel Kamp was a legendary venue for politicians, artists and economists at the turn of the century. It is an important monument for the Finns, a part of the national history and cultural life." And I was particularly taken by the heated marble floor in the bathroom!

Klaus and I walked around Helsinki seeing the sights, and did some shopping. We went for an evening meal in a Lapland restaurant where we both tried the reindeer (which was excellent) followed by a few beers in a Helsinki pub.

Sunday 11/2/2001

After breakfast (which was included in the weekend rate at the Hotel Kamp, if I plug it enough, they might give me an even better rate next time I am in Helsinki <grin>) Klaus and I walked to the large park on the southern tip of the city.

Here we watched a very colourful scene of brightly dressed children (and other young at hearts) tobogganing down a snow covered hill (the temperature being about minus 2 degrees). The bright primary colours against the white background looked like it was straight out of a picture by Lowry.

After a hot chocolate in a city centre café to warm up again, we took the FinnAir bus to the airport, did our last minute shopping and caught our respective planes home.

All in all an excellent seminar, and a FUN trip.

The Development and Q/A Process at Microsoft, US

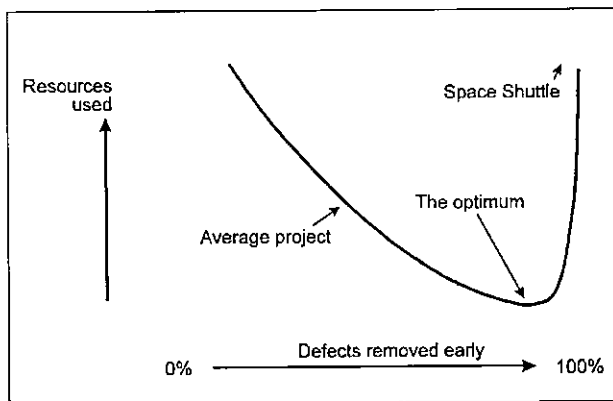
by Kari Höijärvi (hoijarvi@hotmail.com)

I worked for Microsoft Outlook and Exchange groups 1995-1998 and during those three years I learned more about teamwork than I had ever done before, and it sparked my interest in SW Project Management. Most of the knowledge in this article is at least 25 years old, but still not widely understood. While I'm not doing any creative work here, I hope that I can at least teach what others have found out.

No Industrial Spying, Sorry

Microsoft's internal development methods have been very well documented by Steve McConnell[1], Steve Maguire and others[2]. R. Shelby and M. Cusumano[3] wrote a detailed book about it. I was personally surprised how open the text was: just as I experienced myself.

Cost of Quality



Projects that have a tight deadline are completed late, over budget and bad quality. That was a well known fact. But IBM made a strange observation thirty years ago: projects that must have high quality are completed earlier and they cost less. That same was then stated in the classic book "Quality is Free"[4]; the title is

incorrect, but sounds more sexy than the precise "Quality requires initial investment but gives a quick bang for your buck."

The diagram shows the relationship between quality and software cost. Producing space shuttle onboard software has cost 25 hours of work per line. A major amount of shuttle SW development time is spent in inspections and testing. Less

critical software can be produced with much less work, simply dealing with the errors later, in the production, which is not an option for the Shuttle group.

When you move left in the diagram, at some point reducing quality becomes counter-productive since it's more expensive to fix damage later. Sadly, most of the software in the world is produced on the wrong side of the optimum: high cost and shoddy quality combined.

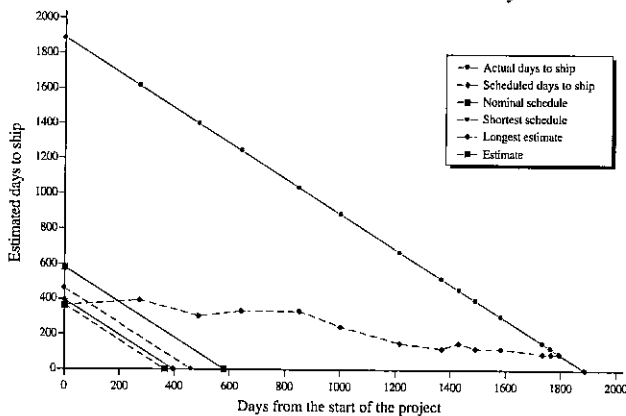
Word for Windows 1.0

In principle I don't like to show negative examples, since they teach you nothing you really need to know. But this project is a very good collection of bad project management practices.

Word for Windows in a nutshell: [5]

- Exceeded the schedule 400%
- Four development leads in five years
- Impossible goal
- Last twelve months was "stabilization", bug fixing and testing.
- After all, a great success
- 250,000 Lines of code

WinWord 1.0 Estimates and Reality



The original deadline was a year, or at worst thirteen months. Steve McConnell at his website has a free statistical estimation tool, which gives the optimal schedule 580 days and the absolute minimum schedule 460 days. The shortest schedule would cost a lot more, since you have to hire a lot of the very best developers, which then would produce less code per developer.

After a year or two, reality hit a little bit. The estimation error went from 400% to the classical π where it stayed for a long time. The last year the product was regularly shipping in three months, until after four years nine months the "90 days to ship" estimate held.

Other bad examples

Mac Excel 1.03 was killed due to a runaway bug list.

I have similar experiences from my projects outside MS. Once I was hired to work in a project, that was almost ready but had some bugs. I tested it for a day and found 50. My boss tested it for a day and found 35. The really worrying thing was, that none of the bugs were the same! Statistically I could estimate at least 2500 bugs, since then the chances that you pick all different bugs are 50%. Obviously, fixing thousands of bugs is not an option and the project was started from scratch.

Four dimensions of software development

- schedule
- resources
- features
- quality

You can set three and predict the last. The major problem with WinWord 1.0 was fixing all the four. You cannot do that. If you want to have **good quality** and **M features** with **N engineers**, then the **schedule** is like weather: you can **forecast** it, but you cannot set it. Similarly, if you have a deadline that is fixed, one of the other three must yield.

Changes to the Process, Birth of MSF

First, a small team with top-notch developers start the project. They create the core architecture and the core code. Testers come into picture right away and once this core part of the product works, more developers can be added to the team.

Features are added to the project. A little at a time, all the time focusing on quality. It's easier to estimate the schedule if 57 out of 83 features are ready and there are twenty known defects, compared to having all 83 features "done" with 1832 items in the bug list. And you can always ship a product without all the functionality. It's much more embarrassing to ship a buggy product.

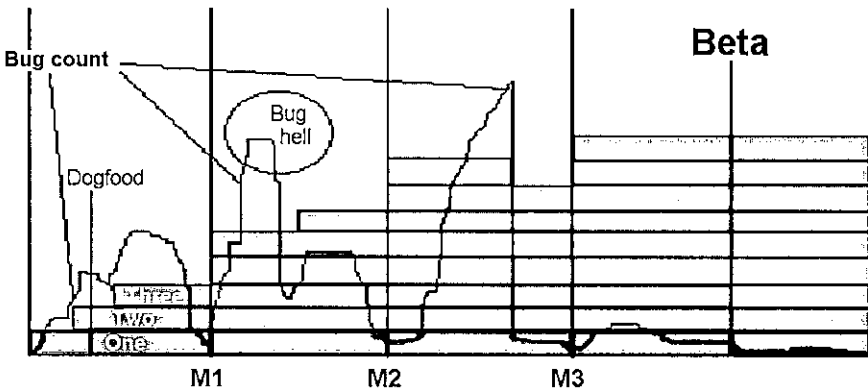
All the projects can be done incrementally. The waterfall method, requirements, design, code, test, integration test, production, all sequential, is rarely the best.

One of the things I like is that people at MS focus in their core competence and are not subject to Peter's principle, reaching their level of incompetence. The best developer is not "promoted" to middle management. Testers get better in testing and get promoted in that field. The unfortunate myth is that testing is a low-level job that you have to do until you get a programming job.

MSF, Microsoft Solutions Framework

The result is now documented at <http://www.microsoft.com/msf>. Since their documentation is extensive I'm not going to repeat it here. There are six equal teams: program management, product management, development, testing, user education and logistics. It's to be noticed, that "management" here refers managing code, not people. Program management communicates between the end user and developer, making sure that the product is useful. Product management communicates with the payer, making sure that the product sells.

Sample Development Process



Here is a simplified MSF project. The core functionality is designed up front. The schedule is split into three major milestones. First the core team puts two first features (One, Two) in and gets the product into "dogfood stage". For a dogfood manufacturer the cheapest way to test it is to feed it to the workers and MS do just the same: I have used MS Outlook as my main mail client since '95, two years before it came to market. It helps to keep developers honest to themselves.

After dogfood, a few features come in (**Three**) and the bug count goes up. Now, instead of fixing them later, they must be fixed now. Once the count is below a pre-agreed limit, Milestone 1 (**M1**) can be declared done. When new features are added the bug count jumps too high. This is unacceptable, everybody's job is to bring it down. Stop what you are doing, get quality back. Fixing it later would cost more. Milestone 2 is reached as soon as the features are in and the bug count again under the pre-agreed limit.

After **M2** a disaster strikes. Bug count goes incredibly high and one feature is identified as the source. The options are either fix it all or throw it away. In this case, the feature is thrown away and bug count is low again. Milestone 3 is reached. After this, get ready for Beta. There are two reasons for Beta. First, it's impossible to test all the possible hardware combinations for PC. For Macintosh, maybe, but PC, no. Second, you need to get real users' feedback. Real feedback beats laboratory tests. The quality of the product must be high to benefit from Beta release, otherwise it's just test user abuse.

Mini Milestone Process

There's no place for 90% done features. Features are either 0% done or 100% done. If you have over a hundred small tasks in your project, this granularity is just fine. Here's a way I currently do these mini milestones:

1. Get a bug fix or new feature to work on.
2. Review it with appropriate people: do we need to fix this, can we cut the feature, what is the simplest way that we can get away with this and keep the product quality good. If you need to write code, continue.
3. Implementation related activities, code reviews included.
4. Find a person to test your changes. Work, until he replies OK. If the feature takes many fixes before getting accepted, it's a sign of questionable quality and a good point for a formal review.
5. Get the latest version from the version control, merge your changes, run automated tests, have a code review. If the product is just going out of the door, get acceptance from the project management to include your work. Last minute fixes are always dangerous.
6. Send a check-in mail for the group.
7. Build lab, full automated tests, in-house testing.

This seems like a lot of bureaucracy, but after all this I seldom have to work with the same problem again. I'd rather have more process than less, for example mandatory pair programming[6].

Build Lab and Daily Build

The product is built every day. Every day, all the automated test scripts are run to catch possible quality regression. Every day, you're supposed to install your daily dogfood from the build lab.

Some people protest against this and say that once a week is enough. But the longer the interval you have between the builds, the further away good quality can go. If your daily build breaks, it's usually a matter of hours to get it fixed. If the daily build breaks twice in a row, warning signals go red. And three days with broken build is a sign to stop what you're doing and fix what you have done.

If a build is done weekly, fixing a broken build gets more and more difficult. I have heard of projects that went months without a working test version. Weekly build can take you to bug hell, daily build can take you back.

If you cannot do a full build with one command, write a script that does it.

Automated Testing

You need some test automation. You don't need 100%. [7] Read "Test Automation Snake Oil," it's excellent.

Two Main Things to Remember

- **Quality.** You have to set a measurable level and make sure it stays there. Teams that focus on quality will automatically focus on a lot of important things. If you have low-quality features, fix them or cut them.
- **Incremental Process.** Do your product with small steps and lots of feedback. measure steps in hours rather than days.

Recommended Reading...

This is about 1% of my link list, choosing is hard.

Spiral and Waterfall <http://www.sei.cmu.edu/cbs/spiral2000/Boehm/>

Cleanroom Software Engineering: Michael Deck has written excellent papers at http://www.cleansoft.com/cleansoft_library.html and I definitely recommend these for anyone who is working with mission critical systems.

eXtreme Programming: How to replace heavy process with discipline.
Numerous links, start with <http://www.xprogramming.com/>

CrossTalk: <http://www.stsc.hill.af.mil/crosstalk/> . Before you have read your monthly free Crosstalk, don't waste money on any other magazine.

Software Project Managers Network <http://www.spmn.com/> has more good stuff than your local bookstore, just a mouse-click away.

Free, short and 100% on-topic books:
http://www.spmn.com/products_guidebooks.html

...and References

- [1] Steve McConnell has written the best books published by Microsoft Press. I'd recommend "Software Project Survival Guide" ISBN: 1572316217 as a must read, I still skim it through regularly. "Rapid Development" has a lot more useful information, but is harder to read and understand.
- [2] Steve Maguire wrote "Writing Solid Code" Microsoft Press; ISBN: 1556155514 which describes ways to make your programs less prone to errors, examples are in C but the principles apply to any language. His "Debugging the Development Process" is similar to Jim McCarthy's "Dynamics of Software Development" ISBN: 1556158238, both telling how MS has done things with bad process and how they were improved.
- [3] R. Shelby and M. Cusumano: "Microsoft Secrets: How the World's Most Powerful Software Company Creates Technology, Shapes Markets, and Manages People" Simon & Schuster; ISBN: 0684855313
- [4] Philip B. Crosby: "Quality is Free" Mentor Books; ISBN: 0451625854
- [5] Steve McConnell, Rapid development. [1]
- [6] Pair programming: numerous studies and links, start from <http://www.pairprogramming.com/>
- [7] http://www.satisfice.com/articles/test_automation_snake_oil.pdf

Array-based Logic

by Klaus Klug Christiansen (kklug@worldonline.dk)

This article is a cut down version of the presentation I did at the FinnAPL Forest Seminar in February 2001. It gives a brief introduction to the basics of handling logical constraints by arrays, which in the world that I am currently part of, we like to refer to as Array-based Logic.

The Ph.D. thesis by my boss Gert Møller "On the Technology of Array-based Logic" covers all that is described in this short article, and conversely this article of course covers only the very fundamentals of Array-based Logic and the contents of the thesis. The thesis as a whole can be downloaded as a PDF-file from the Web: www.arraytechnology.com/technology.asp.

Basis

Array-based Logic is all about handling variables and relations between these variables. In the basic approach variables are represented by axes and the relations between the variables are represented by the content of the array that is spanned by the axes.

An example says more than many words. Here are some relations, expressed by arrays:

$$\begin{array}{cc}
 A \vee B & A \Rightarrow B \\
 \begin{array}{c|c} & B \\ \hline & 0 \ 1 \\ A & 1 \ 1 \end{array} & \begin{array}{c|c} & B \\ \hline & 1 \ 1 \\ A & 0 \ 1 \end{array}
 \end{array}
 \quad
 \begin{array}{cc}
 A \wedge B & A \Leftrightarrow B \\
 \begin{array}{c|c} & B \\ \hline & 0 \ 0 \\ A & 0 \ 1 \end{array} & \begin{array}{c|c} & B \\ \hline & 1 \ 0 \\ A & 0 \ 1 \end{array}
 \end{array}$$

$$\begin{array}{cc}
 A \Rightarrow (B \vee C) & (B \vee C) \Rightarrow D \\
 \begin{array}{c} \overline{\quad\quad\quad} A \\ \begin{array}{c|c} & C \\ \hline & 1 \ 1 \\ B & 1 \ 1 \end{array} \quad \begin{array}{c|c} & C \\ \hline & 0 \ 1 \\ B & 1 \ 1 \end{array} \end{array} & \begin{array}{c} \overline{\quad\quad\quad} C \\ \begin{array}{c|c} & D & D \\ \hline & 1 \ 1 & 0 \ 1 \\ B & \overline{\quad\quad\quad} D & \overline{\quad\quad\quad} D \\ & 0 \ 1 & 0 \ 1 \end{array} \end{array}
 \end{array}$$

Each axis expresses all possible values of the variable that it represents. The content of these arrays is strict binary, since it expresses whether a combination of

variable values is valid or not. 0 means that a particular combination is invalid, 1 means that the combination is valid.

Combining Relations

But these are all independent relations. Before you can start making conclusions on what will happen, when the same variables form part of more than one relation, you'll need to be able to combine relations, to put smaller relations together to make bigger relations, or in our terminology: to join relations.

A new array to represent a joined relation is made by combining the axes from the relations that are to be joined. If two relations (arrays) are being joined, and they respectively have 2 and 3 variables (axes), the joined relation will have 5 axes.

The content of the array still represents the relation.

In figuring out how to fill the new array (describe the relation), keep in mind what you are trying to achieve: combinations of variable values should be valid in the joined relation only if they are valid in each of the relations that are being joined. Or in other words: in the new array only combinations that are represented by a 1 in each of the initial arrays should result in a 1. This is easily done with the Cartesian product between the arrays:

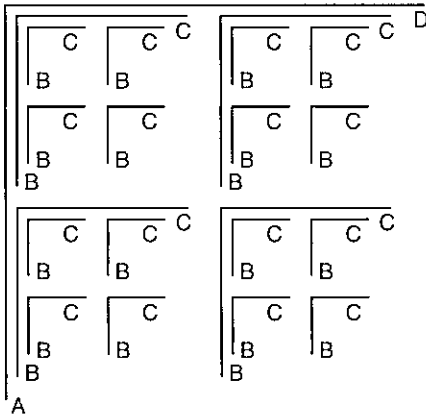
$$A \Rightarrow (B \vee C)$$

$$(B \vee C) \Rightarrow D$$



The result is as follows. The content of the array has been left out intentionally, in order to enhance clarity. Also, the order of axes might not be indicative of what the result would be, if this operation were performed in APL.

$$A \Rightarrow (B \vee C) \Rightarrow D$$

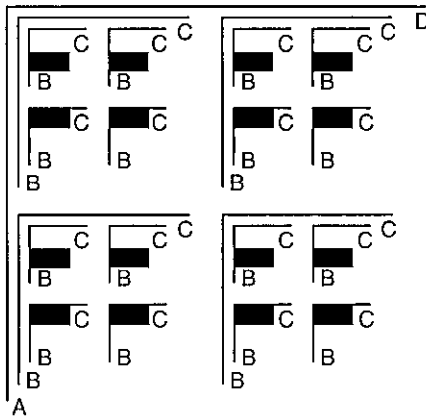


Now we have the joined relation, but the result is not ideal for practical use, since some of the variables in the relation are represented more than once, i.e. there is not a one-to-one mapping between the variables in the relation and the axes in the array. What you would want to do now is to reduce the number of axes in a way that the result has only one axis per variable. How can this be done?

Again you should keep in mind what the trouble is, and what the result is that you are trying to get at: a number of axes (more than one)

represent the same variable, and you only want one.

When a variable is represented by more than one axis, some of the content of the array spanned by these axes is superfluous, since it represents combinations of variable values that will *never* occur, which can be best illustrated by looking at it.

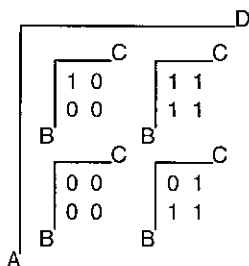


Here an array with six axes expresses the relation between four variables. If you were to look up if a combination of variable values is valid, you would make sure when doing the look-up in the array that you were looking from the same position on all the axes representing the same variable. The same variable can take only one value at a time!

The black boxes in the illustration then describe combinations of variable values that will never exist, with regards to the variable B. If you remove these combinations, the combinations left behind perfectly describe the diagonal between the two axes representing the variable B.

Reducing them by the diagonal thus combines the axes that represent the same variable, leaving this variable represented by only one axis. In Array-based Logic this is known as "colligation".

Here the array has been reduced by the diagonals between the pairs of axes representing the variables B and C:

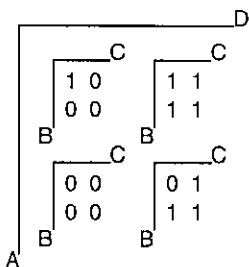


Deduction

Now that you're capable of combining relations and describing the relation between many variables, you would want to make use of it. One practical use could be deducing the impact that limiting some variables has on other variables.

Reducing the axis that represents a variable, and thus reducing the entire array that represents the relation, limits the number of valid values for the variable. The remaining array gives the relation between the remaining variables.

An example: what happens to B, C and D if A is bounded to true?



As illustrated, the answer to this question is found by reducing the array by the axis which represents the variable A. Only the value to which A has been bounded is left on the corresponding axis (the non-hatched area of the array).

This is basically an array representing the relation between variables B, C and D in the case where A is true. Locating the 1s, which (as mentioned earlier) represent valid value combinations, it becomes obvious that D is bounded to true. This is seen from the fact that there are no 1s in the part of the array representing value combinations, where D is false.

It can also be seen that neither B nor C are bounded to any particular value, except that they cannot both be false. In fact the part of the array, where variables A and D are bounded to true, represents the relation $B \vee C$.

In this small example one variable is bounded to only one value. From a more general point of view, it is possible to bound more variables in a deduction, and not all of these variables need be bounded to one element only.

Affirmative Representation

As you have probably figured out by now, representing variables as axes poses limitations. One is that APL allows a maximum of 15 axes in an array. And 15 variables in a relation is a very small number, if you want to get serious in constraint solving.

And even if APL didn't stop you, the size of a relation doubles with every variable you add, which quite quickly makes your relation grow out of proportion, even with today's reasonable density and price of RAM chips.

So far all combinations of variable values have been represented. What if you chose to represent only the valid combinations? If you did this by representing them by their addresses in the array, you'd have broken the unpleasant scaling of a relation and overcome the limitation of 15 variables at the same time.

Representing the relations in the affirmative form looks like this:

$$A \Rightarrow (B \vee C)$$

$$(B \vee C) \Rightarrow D$$

A	B	C		ABC
		1	1	
		1	1	
		0	1	
	B	C		
		1	1	
		0	1	
		1	1	

D	B	C		BCD
		1	0	
		0	0	
		1	1	
	B	C		
		1	1	
		1	1	
		1	1	

Looking at the addresses you'll notice that these are actually the values of the variables. This is a natural consequence, since the addresses describe the location on the axes, which again are representations of the variables. This also means that the content of the arrays is no longer limited to being binary, but closely follows the domains of the variables.

With this representation you can have more than 15 variables, and the more relations, you combine, the smaller your array will probably get, since you're storing only the valid combinations of variable values.

With this new representation, you'll have to redo the join and colligation operations. Take a minute to think about it, before you read on. Being an APL'er, you'll most probably be able to figure out how to do the Cartesian product between two relations in the affirmative representation, right?

You have two arrays that contain representations of valid combinations of variable values - combinations that were previously represented by 1s. Previously the array representing the joined relation would contain 1s only at the places that corresponded to 1s in both relations that were being joined. Each of the 1s in the one relation was being matched with each of the 1s in the other relation. Now you do the very same thing, just using the addresses of the 1s:

BCD		ABC
000	° . ,	000
100		001
101		010
110		011
111		101
		110
		111

The result will be an array that is too big to display in its entirety, here's just a small part of it:

ABCBCD

```
000000
000001
000011
000101
000111
001000
001001
001011
...
```

As before, joining the two relations leaves the variables B and C represented twice. Again you would want to colligate the axes, only now the axes have been converted to columns in the array. How do you reduce two axes by their diagonal, working with columns in the affirmative form?

In the diagonal spanned by axes that represent the same variable, the instances of the variable have the same value. Reducing by the diagonal an array in the affirmative form therefore is done by comparing the values in the columns and leaving behind only the rows where the values are equal. Colligation takes out the hatched rows, first with regards to variable B and then C:

ABCB	ABCB
00000	00000
00001	00001
00011	00011
00101	
00111	
00100	00100
00101	00101
...	...

The resulting array has 8 rows. Now that the columns representing the variables B and C are equal, one of each can be removed, leaving only one column per variable:

ABCB	AB
00000	0000
00001	0001
00101	0011
01010	0101
01111	0111
10101	1011
11010	1101
11111	1111

If you compare this result with the result from the join and colligation working with the full array, you'll see that the rows in the former correspond exactly to the 1's in the latter.

Deduction

Now that the building operations have been redone to work with relations in the affirmative form, the deduction needs to be made by the same process.

Taking the example from before, asking what happens to B, C and D if A is bounded to true, have a look at the array:

ABCD	
0000	A is bounded to true, meaning that only the rows where A is true represent possible variable combinations. These are the non-hatched part of the array.
0001	
0011	
0101	
0111	Looking at these rows only, it is fairly easy to see that as a consequence the variable D has been bounded to true, and that at least one of the variables B and C must be true.
1011	
1101	
1111	

Use of Array-based Logic

This concludes a short look at the fundamentals of Array-based Logic. Following the described guidelines, you're perfectly capable of building your own logic models and doing your own deductions. If you want to go further, the techniques of Array-based Logic are available as an all-in-one application (the Array Database™) that automatically will do all joining and colligation (plus a lot more) for you. The Array Database is a *configurator*, which can handle large-scale configuration problems in real time, due to the intrinsic properties of Array-based Logic.

The Array Database is used in practice as the underlying engine in many different surroundings. Some examples are: e-trade solutions, both Web and business-to-business, railway-interlocking systems, switching in power networks, and configuration in general.

References

- [1] Gert Møller: *On the Technology of Array-based Logic*. Ph.D. Thesis, Electric Power Engineering Department, Technical University of Denmark, January 1995.
- [2] Bo Grave: *Modelling Signal Interlocking Systems using Array-Based Logic and APL*. Paper at APL96, Lancaster, <http://www.vector.org.uk/apl96/bograve.htm>
- [3] Bo Grave: *Railways and Array-based Logic*. Ph.D. Thesis, ScanRail Consult and Department of Electric Power Engineering, Technical University of Denmark, February 1998.
- [4] Morten Christensen: *Array-based Configuration in Power-supply*. Ph.D. Thesis, Department of Electric Power Engineering, Technical University of Denmark, October 1999.

D: A Functional Subset of Dyalog APL

John Scholes (john@dyadic.com)

Dynamic functions and operators appeared in Dyalog APL/W Version 8.1 in early 1997. Since that time, experience has shown that their field of application is at least as wide as expected and "D-Fns" have started appearing in many operational workspaces. A number of applications [1] have been written entirely in D-Fns and D-Ops and so we can consider them to constitute a small, complete sub-language within Dyalog APL. As the letter is one of the remaining few, not currently assigned to a programming language, we could call it: **D**.

D functions and operators are lightweight, faster but richer alternatives to the traditional "header-style" container for the collection of APL expressions. **D** is described in detail in [2] but the following little "reference card" is a summary:

{ α Function w }	{ $\alpha\alpha$ Operator $w\alpha$ }	: Guard
α Left argument	$\alpha\alpha$ Left Operand	:: Error-Guard *
w Right argument	$w\alpha$ Right Operand	$\alpha+$ Default larg
v Self reference	$v\alpha$ Self Reference	1:s+ Shy result

* Error-Guards are due to appear as an enhancement to Dyalog Version 9 and will be released initially via the web-based Dyalog Subscription Service.

There is a (possibly apocryphal) story that a great advance in the field came when some computer scientists were trying to teach programming to a group of geography students. The students couldn't understand the meaning of: $1=1+1$, and on reflection, the teachers realized that they weren't too sure either!

Enumeration of all possible D functions

If we were to list all possible D functions in order of complexity, each of the first six would be useful:

<i>sink</i> +{ }	<i>a Discard unwanted value.</i>
<i>lev</i> +{ <i>α</i> }	<i>a Left argument.</i>
<i>dex</i> +{ <i>ω</i> }	<i>a Right argument.</i>
<i>const</i> +{ 0 }	<i>a Return constant.</i>
<i>dup</i> +{ <i>ω ω</i> }	<i>a Duplicate.</i>
<i>link</i> +{ <i>α ω</i> }	<i>a Left arg, right arg, pair.</i>

const provides the shortest (but not quickest) way to return an array of values, the same shape as an arbitrary array: `{ 'woof' }''102p'K'9`.

Tail recursion vs. Iteration

D's guard construct is sufficient to replace *all* the usual control structures. People find this unsurprising for *:If* *:Else* and *:Select*, but sometimes have difficulty with the iterative controls.

:For splits into two cases. If there is no communication among the passes of the loop, it maps immediately onto `{...ω}`. The construct is recognized by the interpreter and special-cased to re-use the D function stack frame, which gives it the same slick performance as a tail call. Otherwise, if there is some 'accumulation' of values by successive passes, it maps onto `+{...ω}/`.

:While and *:Until*, each contain an implicit test for termination. In D, this test maps onto a guard at the top or bottom of the function, where the 'loop' becomes a tail-recursive call on the function. In this context, tail recursion has been described as "looping with parameters". Look at the following equivalent versions of a function for the sum of the items of a vector. (There are of course, easier ways to achieve this.)

<i>Traditional APL</i>	<i>D</i>
<pre> ∇ rslt+sum nums [1] rslt←0 [2] :While 0≠pnums [3] rslt←+>nums [4] nums←1+nums [5] :EndWhile ∇ </pre>	<pre> sum←{ α←0 0=ρw:α (α+>w)∇ 1+w } </pre>

We sometimes seem reluctant to use recursion. To be fair, we may be worried by a *perceived* cost of stack allocation and at first find it difficult to distinguish between a 'tail call' and a 'stack call'. In Dyalog, a tail call is by a small margin, the fastest way of 'branching'! Look at the following two versions of factorial.

<i>Stack Call</i>	<i>Tail Call</i>
<pre> fact←{ w←1:1 w×∇ w-1 } </pre>	<pre> fact←{ α←1 w←1:α (w×α)∇ w-1 } </pre>

Techniques

The rest of this note is devoted to a variety of techniques that have been discovered while programming in D. References such as [*max parse*], show [workspace function], where examples of the technique can be found in sample workspaces supplied with Dyalog Version 9.

Avoiding Name Clashes

APL's 'name clash problem' can be largely avoided in D. Traditional functions that do workspace administration tasks such as function cross-referencing, must be careful that their own local names do not obscure the global names in which they are interested. This leads to the choosing of 'nasty' local names:

`Δ_Δ_fns` `__cr_list`, etc. (The author has been tempted to use obscene words for such names in locked functions, reasoning that if a name clash did occur, the victim would be unlikely to call to complain - but decency prevailed).

Names in **D** are localized dynamically. In the following function, *names* and *source* are localized *after* the evaluation of $\square CR$ and $\square NL$ and so do not obscure global homonyms.

```
wsdoc+{
  names source++q+{w( $\square CR$  w)}''+ $\square NL$  w
  ...
}
```

[dfns attrib]

This technique is fine for simple tasks such as the above, but can become unwieldy for more complex ones, as it requires that all 'external work' be done *before* the first assignment. More generally, we can preserve an outer name-free environment as a function operand to an inner operator. This is one advantage of **D**'s *static* scope rules.

```
find+{
  {±w}{
    local names+... A 'external' operand.
    local names
    ...
    vars+αα' $\square nl$  2' A name list via operand.
    vals+αα''+vars A values via operand.
    ...
  }w
}
```

[dfns find]

Function Pipelines

It is sometimes convenient to break up a large function into a 'pipeline' of smaller ones, where the result of each is passed as argument to the next. In particular, this gives fine control over the amount of workspace used, as large temporary definitions may be localized more accurately. In some cases, the indentation in a 'raw' pipeline of functions shows the structure of the code better than does a succession of function names: *foo goo ...*.

```
{
  ... ▽ ...
}•{
  ... ▽ ...
}'' ...
```

[dfns factors] [dfns stamps] [dfns wsdiff]

Initial left argument

One way to provide an initial (as opposed to default) left argument, as an accumulator for a function, is to bind it at definition time using composition. This delivers a slight performance benefit, particularly in a recursive function, as the body of the code no longer needs to test $(\alpha \leftarrow \dots)$ for the existence of the left argument. The *sum* example above would become:

```
sum←0∘{
    0←p w:α
    (α+1+w)∇ 1+w
}
```

```
[max.parse]
```

'Call me'

The ∇ or 'self' token refers to its nearest enclosing brace pair. If we want a recursive call that 'reaches' out more than one level, it seems we are obliged to name the target function, solely in order to use its name for the call:

```
foo←{
    {
        foo w a recursive call on outer function.
    } w
}
```

An alternative is for the target function to pass a reference to itself as an *operand* to the inner function, which then becomes an operator:

```
{
    ∇{
        αα w a recursive call on outer function.
    }w
}
```

The technique can be used to pass the function reference through an arbitrary level of nesting. This means that we are not forced to invent an artificial name for a function just in order to recur. We name things only for elucidation and to avoid repeated evaluation. Perhaps for this reason, defined operators seem to occur more frequently in D than in traditional APL. [*min parse*]

Complex guards

Finally, an obvious point: there is no limit to the complexity of the guard expression to the left of the ':'. In particular it can include a D function call.

```
{
  ...
}w:{
  ...
} w

[max unify]
```

References

- [1] *min* and *max* are sample workspaces supplied with Dyalog Version 9.0.
 - [2] Vector Vol. 13.2, 88 "Dynamic Functions in Dyalog APL" Available from <http://www.dyadic.com/download/dfns.zip>. This rich text format file uses font "Dyalog Std TT", from <http://www.dyadic.com/download/dlogttst.zip>.
 - [3] The Computer Journal, Vol. 32.2, April 1989.
 - [4] Bird, R. and Wadler, P. "Introduction to Functional Programming" Prentice Hall ISBN 0-13-484197-2
-

Renaissance Data Systems

All APL Books in Print



We are pleased to announce that RDS is now on the web at www.aplbooks.cnchost.com with a display of our entire catalog of APL Books in Print. There are even some classic publications which we have received permission to reproduce. Subjects include APL and Mathematics, Learning APL, and APL for the Professional Programmer. We also carry a few books of interest from the rest of the world!

You may contact us using email at rendata@aplbooks.cnchost.com and place orders at orders@aplbooks.cnchost.com. Credit card payment is available for international customers.

Please note our new mailing address:

Renaissance Data Systems
P. O. Box 313
Newtown, CT 06470.

Over and Under, Over and Under: Celtic Knotwork Patterns

by Anne D. Wilson

Introduction

Like many mathematicians I am intrigued with pattern, and the weaving of Celtic designs has been a favourite of mine since I was a child. So when recently the words "computer" and "Celtic design" were uttered together it triggered an urge to write my own system.

Most published work describes existing patterns, but does not give methods of drawing patterns. However, George Bain [1] attacked it from the artist's viewpoint in the mid 20th century. Later his son, Iain Bain [2], described and extended his father's work in a more scientific manner. Although their method produces correct patterns the procedures followed are more complicated than is appropriate for the complexity of the problem.

Thus I felt free to follow my own path to simplicity. And so I started drawing simple patterns over every spare piece of paper – over and under, over and under – until the algorithm emerged.

In this article I first describe Celtic patterns, then enumerate rules for defining *rectangular* patterns. Comments on evolving the design of programs show why I work as I do. Next I describe the construction I place over a Celtic pattern to enable me to describe an entire design in a simple matrix. Then the ways of proceeding from a solid mass of pattern to a designed Celtic pattern, and a method of separating into *bands*. Lastly, but not crucial to this article, I indicate how I have developed this as an APL system.

Subsets of Celtic Patterns

There are several different types of Celtic design. This article describes the easiest which are the *rectangular* patterns. *Circular* patterns share many of the same characteristics, and will be described in a later article. *Zoomorphics* are composed of intertwined birds, beasts and reptiles.

What is a Celtic pattern?

Two rules are sufficient to describe Celtic patterns, *viz.*

Rule 1 Celtic patterns are composed of closed lines (curves) known as bands

Rule 2 Lines which cross alternately weave over and under

These rules apply to all Celtic patterns except zoomorphics.

With such simple rules the algorithm required to draw Celtic patterns should be simple, for no algorithm should be more complicated than the problem it solves.

Simple Programs

The process of developing algorithms intrigues me. Why is one approach better or worse than another? How do some programmers produce short, efficient and correct programs with ease, while others produce long, slow and difficult to maintain ones? The intelligent fly on the wall will notice that the good programs are those where characteristics of the problem become the characteristics of the program. It will also notice that the bad programs are usually overly complicated and the good ones relatively simple.

What are these characteristics? Whole methodologies have been devised in the nineteen seventies which concentrate on different aspects [4]): Grindley [5] time stamps every piece of data; Jackson [6] makes tree structures of input and output data, and combines these to form the program structure; Myers [7] and Constantine [8] look at the in-transform-out flow of data; no one apparently starts with the problem being solved.

The intelligent fly on the wall also notices that a macro methodology would guide the developer to see which aspect should be considered, *viz.* data structure, data flow or the problem itself. The fly will realise that no program should be more complex than the problem it is solving [8], and that the most crucial time in any system development is spent in describing data and problem in different ways until the simplest solution emerges. Complexity in a system should be the result of a real unresolvable complexity in the problem.

The method devised by the Bains [1] [2], and used by Glassner [9], involves two sets of grid lines. The intersections occur on the points in the grid (see below for my intersections). The resulting process is more complex than mine.

Defining A Celtic Pattern

Starting with a rectangular solidly filled pattern the shape is described as $m \times n$ where there are m cusps at the top (and bottom) and n cusps along each side. An arbitrary decision is made whether the top crossing row is right over left or left over right. Note the irrelevant fact that we are into the realms of reef knots and macramé!

Interestingly in alternate rows the lines go from (top) left over right on one row and right over left on the next; the columns alternate in the same way.

This follows from the Rule 2, that lines alternately go over and under. I will not give the proof here, but it is an elementary topology.

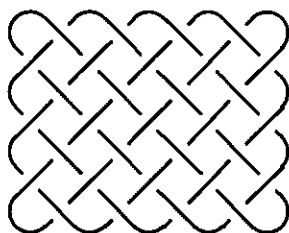


Figure 1: 5×4 solid pattern

The Grid

Many pieces of paper passed under my pencil before it emerged that the following method of describing a pattern yields a simple method of describing the data, and allows easy manipulation when developing patterns.

With an $m \times n$ pattern the grid is composed of $(2m+1) \times (2n+1)$ cells. It is therefore a sparse matrix, and these initially empty cells always remain empty; justification for their existence is that they simplify the processes of drawing and manipulating patterns.

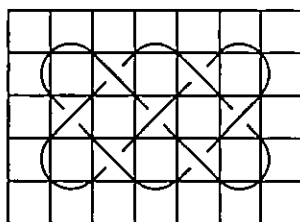


Figure 2: 3×2 solid pattern with grid overlaid

Rules of the grid:

1. Lines enter cells at corners (never cutting the sides)
2. Lines go diagonally through corners of cells
3. Either 0 or 1 line goes through a corner
4. Each cell contains 0, 1 or 2 lines
5. Lines enter and leave a cell at either adjacent or opposite corners
6. Intersections occur inside cells

Several of these rules can be deduced.

Rule 2 follows from the sparsity of the cells, as only diagonally adjacent cells can be non-empty.

Rule 4 follows because the curves are closed (definition of Celtic patterns) combined with grid rule 3.

Rule 5 follows from grid rule 3.

Rule 6 follows from grid rules 3 and 4.

The Cell-trix Matrix

The data is now ready for processing, and the next step is to devise a method of representing it. The method seemed so obvious that only my subconscious can claim credit for it!

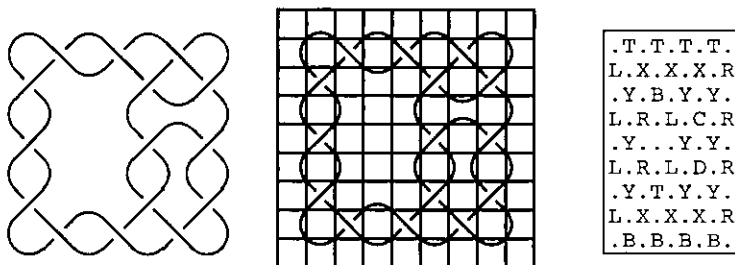


Figure 3a: pattern grid 3b: same pattern with grid 3c: the cell-trix matrix

When considering the lines contained within a cell there are very few cell types, and to each I assign a letter, as in Figure 3:

empty	blank
T	top cusp of band
B	bottom cusp of band
L	left cusp of band
R	right cusp of band
X	intersection – left over right
Y	intersection – right over left
C	across – composite T + B
D	down – composite L + R
I	needed for splitting into individual bands
	– line from top right to bottom left
J	needed for splitting into individual bands
	– line from top left to bottom right

The matrix describing the 4×4 pattern in Figure 3a is the 9×9 matrix in Figure 3c. And that is all that is needed to reproduce the pattern. Simple!

The decision to have both X and Y cells is an important design consideration. They both represent an intersecting pair of lines, so could be represented by a single cell, say Z. But do they represent 2 lines or 3 lines (one diagonal and 2 shortened halves)? They contain more information than Z would, for they say which line goes on top. If Z were used this data would have to be held separately. The design decision was to hold all the data in the one matrix, and keep this extra information within the cell type – a principal behind object oriented design.

I pondered long over the need for I and J types. They do not have a similar relationship to X or Y that T and B have to C, because they lose the left over right information.

I use the words cross and uncross to describe these operations performed on intersections. The Bains [1] [2] [9] use breaklines. Their grids are drawn through points where intersections can occur, so that their breaklines are along edges of the grid cells. These differences arise from our different descriptions of the data, and makes comparison of methods difficult.

Start with the Solid Pattern

There are at least two approaches to starting – everything or nothing. Intuitively I found it easier to start with everything and remove bits. The steps in producing the cell-trix matrix for an $m \times n$ pattern are:

- [1] Create a $(2m + 1)$ by $(2n + 1)$ matrix of blanks
- [2] Put T in columns 2 4 6 ... $(2m)$ in row 1
- [3] Put B in columns 2 4 6 ... $(2m)$ in row $(2n+1)$
- [4] Put L in rows 2 4 6 ... $(2n)$ in column 1
- [5] Put R in rows 2 4 6 ... $(2n)$ in column $(2m+1)$
- [6] Put X in columns 3 5 7 ... $(2m-1)$ in rows 2 4 6.. $(2n)$
- [7] Put Y in columns 2 4 6 ... $(2m)$ in rows 3 5 7.. $(2n-1)$

This process is obviously stored in a subroutine, and invoked with parameters m and n . What could be simpler?

Next Comes an Interesting Pattern

Figure 3 shows the result of the 2 design processes I have included in my system. **Uncross** – changes an intersection into a pair of non-intersecting lines. **Hole** – removes lines to create a space in the centre of the design. These operations can be reversed. Disclaimer – these are not necessarily exclusive.

Process 1 – Uncross

In Figure 3 there are 2 examples of this, replacing X's with C and with D

either X or Y is replaced by:

either C – split across into a composite T and B

or D – split downwards into a composite L and R

Why can this be done? Do Rules 1 and 2 of the Celtic pattern still hold?

Rule 1 All lines entering and leaving the cell are continuous, the lines are cut and spliced.

Rule 2 I will illustrate this by the example of a line entering an X cell from the top left corner with a LoverR, i.e. it is in an over state. See Figure 4.

The line must therefore go over at its next intersection.

Having changed direction it will enter the next cell from the top right.

Having changed rows it is now in a Y row, so it will become RoverL.

Thus there are 2 changes due to its change in direction and of row, which (binary) process brings the line back to its correct over state. This reasoning can be repeated for the other 3 cases or extended into a proof.

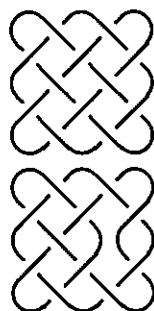


Figure 4: "Over" line is still over after removing intersection

	Before	Inter	After
	.T.T.T. L.X.X.R .Y.Y.Y. L.X.X.R .Y.Y.Y. L.X.X.R .B.B.B.	.T.T.T. D.X.X.R .Y.C.Y. D.D.D.R .Y.C.Y. D.X.X.R .B.B.B.	.T.T.T. L.X.X.R .Y.B.Y. L.R.L.R .Y.T.Y. L.X.X.R .B.B.B.

Figure 5: Pattern before, after and partly processed with cell-matrix at each stage

Process 2 – Make a Hole

When making a hole in the pattern I am always surprised at the space resulting from an apparently small alteration.

A hole is centred on an empty cell, in figure 5 this is at position (4,4).

Any surrounding X and Y are uncrossed. At this stage a circle is centred on the cell. This circle is then removed and hey presto the space is empty.

The hole can only be made if the circle can be drawn.

Do Rules 1 and 2 of the Celtic Pattern still hold? The processing to produce the circle obeys the rules of uncrossing, so at this stage the rules have not been violated, and we only need consider the removal of the circle itself.

Rule 1 One closed line is removed, but all other lines are unaltered and therefore remain closed.

Rule 2 There is no weaving with the circle, and therefore all intersections remain unaltered.

Bands

Perhaps because I use a black and white printer the necessity to show different bands is of low priority in my system. It is, however, a useful tool when investigating the geometry of bands in a Celtic pattern. Iain Bains reputedly states that in a solid pattern of shape $m \times n$ there is one band if m and n are prime to each other, otherwise there is more than one band. This seems intuitively correct, and is true for all the examples I have looked at, but...? When the pattern is altered in any way the number of bands may well change radically.

When working on this algorithm I decided to split up the pattern cell-trix into a set of band cell-trices, one for each band. This keeps the consistency of the data formats and does not introduce any new concepts. Combined together the band cell-trices gives the complete pattern; except for the over-under information, which is only available from the pattern cell-trix.

Glassner's method [9], based on the Bain's [1] [2] ideas is band oriented, with plenty of looping. In APL using a frontal approach on all the data can reduce looping. However, I do find it easier to loop through the consecutive nodes on each band or part band, (see below). I may rewrite it later and choose a different approach without loops? Because bands are closed lines they must contain at least one each of top, bottom, left and right cusps, or else they would escape like a molecule out of a gas. I use this fact to give an initial, maximum, estimate of the number of bands i.e. equal to the number of T cells.

Data held in cell-trix format contains information about the position of the lines, including their behaviour at intersections. But when splitting the pattern into bands we need to separate out individual lines in a cell as they may belong to different bands. At this stage we drop the over-under property in the band cell-trix. Now we need to use the I and J type cells.

To recap, and expand:

- input to this part of the system is the pattern cell-trix;
- output is a set of identically shaped cell-trices, one per band;
- each T cusp is used as a starting place for a band;
- the band is completely "visited" when the next cell is its starting cell;
- if the next cell is the starting point of another band the two bands are combined;
- when all the bands are complete every line of the pattern will have been incorporated into bands;
- over-under characteristics are still available from the program cell-trix.

The steps in producing the band matrices are:

- [1] Find all T cells. To each assign a band number and set up an empty cell-trix the size of the pattern cell-trix. Keep a last-cell-visited value which is artificially set as a sentinel for the first cell.
- [2] Copy the cell type letter to the band matrix. This will result in a composite X, Y, C or D if the cell has previously had a value assigned.
- [3] Move to the next cell for the band, which is determined from the cell type and the last-cell-visited, and loop to [3] until another T is visited.
- [4] This new T is either the start of this band, signalling that this band cell-trix is complete, or
- [5] the start of another band; that is actually another part of this band.
Combine the two part bands, by "adding" the band information in the two band cell-trices.
Replace starting reference to the second part band with that of the first.
Remove all references to the first part band.
- [6] If the bands are processed sequentially return to [3] until all bands are processed.
- [7] Each of these bands is a stand alone cell-trix, but does not hold all the over-under information.

The problem, encountered here, of not attaching over-under information to the band cell-trix is the price paid for simplicity in the rest of the system. It is a small price to pay when compared.

Double and Skew

Celtic patterns are often used as borders, or repeated with rotation through 90 or 180 degrees. The program has facilities to perform these operations, which it does by manipulating the cell-trix with a suitable translation of cell types.

While the current program only understands a square grid it is but a simple step to include rotation and translation at the display/printing step.

The Program

Eventually it all has to come out, and the program is the proof of the pudding. I have written this as a Windows program in APL, using the same infrastructure as my other systems for use in craft design. The user can set up, and keep, their own environment, including line width, gap size at intersections; scaling for printed outputs, headings for prints.

There are filing facilities for saving the program cell-trices and retrieving them. The cell pattern can be output as a printed pattern, scaled to a requested size. It can be output as a metafile or sent (as another metafile) to the clipboard, with or without grid lines shown. Using the "RealSize" property ensures it is a sensible size to use in a Word program, as I have done in this article.

The code consists of two parts; firstly maintaining and manipulating the data, and secondly presenting the output efficiently. To start the user either picks an existing pattern, or selects the size of a new pattern using spinner objects. To manipulate the data the user clicks the place on the pattern where he wants alterations made, and then selects the type of alteration from a drop down menu. The code consists of checking that the requested alteration is valid for the clicked point, followed by really noddy, and rather boring, alterations to the cell-trix.

Displaying the pattern is simple in principal, but oh I wish I found putting data into nested arrays more intuitive. The centre of each cell uses its row and column values as its co-ordinates. The straight lines, from X, Y, I and J cells have their end points calculated, noting that intersected lines are actually composed of two lines. Some optimising is performed to combine straight runs of lines into single lines. If the grid lines are required their endpoints are calculated and appended. Offsets and scaling are calculated for the particular output and size required, and applied. The lines are then all combined into a single nested array.

I have used two methods to display the curves for T, B, L, R, C and D cells – simple arcs of circles and Bezier curves. It is important that the join of line and curve should be seamless, which necessitates having the same gradient; in Bezier

curves the starting gradient is specified by the user, so can be the same as the line to which it is attached. The same condition is met with segments of circles if the radius is the length of the diagonal of a cell and the centre is the midpoint of the opposite edge of the adjacent cell, so that the arc subtends 90 degrees at the centre.

A run of T and B, or L and R give a wavy surface, which could/should be replaced by straight lines. In Celtic designs corners can be treated in various ways, and options should be given to do this. But these are all cosmetic changes with subjective decisions.

In Conclusion

The representation of Celtic patterns is very simple using the cell-trix matrix. Describing the data in this way leads to simple processing both for data manipulation and pictorial output. Complex patterns and designs can be quickly produced, and a record stored for future use. There is scope for prettifying the designs, but that is subjective and complicated to code; if the patterns are to be used by craftsmen they will make changes anyway.

Future developments include rotation and transformation in a non-linear way to give non-rectangular shapes. The analysis of circular patterns requires further investigation, and may require a different algorithm and method of storing the data.

Bibliography

- [1] Bain, George: *Celtic Knotwork*. Wllm.McLellan. 1951
- [2] Bain, Iain: *Celtic Knotwork*. Constable & Co., 1986
- [3] Wilson, A D: *Tree Algorithms* (D Phil thesis). York University, 1986
- [4] Grindley, K: *Systematics*. McGraw Hill, 1975
- [5] Jackson: *Principles of Program Design*. Academic Press, 1975
- [6] Myers, G J: *Reliable Software Through Composite Design*. Petrocelli/Charter, 1975
- [7] Constantine, L L: *Fundamentals of Program Design*. Yourdon Press, 1974
- [8] Wilson, A D: "Mix and Match". Datalink, 20/11/1978
- [9] Andrew Glassner: "A.G.'s Notebook". Graphics and Applications. IEEE Computer, Sept.1999

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Technical Correspondence		112
At Play with J: Erdős Numbers and Pierce and Engel Expansions	Gene McDonnell	113
A New Life For ∇: Structure Control	F.H.D. van Batenburg	125
J Rational Numbers....	Gordon Sutcliffe	131

TECHNICAL CORRESPONDENCE

From: Jonathan Barman

22 March 2001

One aspect of the style of coding in the Zark extracts that I find difficult is the way functions are defined which return a result and also update one or more global variables. In Vol. 17 No 3 there was a typical example where the function *COMMAΔMAT* both returns a result and assigns a global variable *fieldwidths*. Of course, with a nested array APL one just returns the two items wrapped up as a two element vector and the function becomes 'pure' without any side effects. I think my displeasure is due to the fact that I have used nested arrays for such a long time that seeing a function with side effects is instantly categorised as being poorly coded. I find it quite difficult to remember how I attempted to solve the dilemma of multiple results with a non-nested APL, but I think I tried to make a clear separation between functions returning results and those which did not return a result. Only the non result returning functions were allowed to update or assign global variables. I have thrown away all my old code now, so I can't tell if my memory is correct!

From: Jonathan Barman

22 March 2001

In J-ottings No. 26 Norman Thomson gives an interesting solution to the problem of distributed rounding, but only for positive numbers. Accountants have a little problem, which I have never been able to solve totally satisfactorily, where there is a mixture of positive and negative items which sum to zero. A Trial Balance is effectively a vector of detail items which sum to zero. Rounding the Trial Balance usually, but not always, enables a rounded version of the Balance Sheet and Profit & Loss Account to be prepared with all the intermediate totals which are near the correct values. One way of rounding the Trial Balance is to apply distributed rounding independently to the two sets of positive and negative items, but sometimes this rounds items the 'wrong' way unnecessarily. Does Norman know of a better method?

At Play With J: Erdős Numbers and Pierce and Engel Expansions

by Eugene McDonnell

If you wish in the world to advance,
Your merits you're bound to enhance,
You must stir it and stump it,
And blow your own trumpet,
Or, trust me, you haven't a chance!
W. S. Gilbert, *Ruddigore*

Introduction

This paper discusses a way in which mathematicians are connected to each other, much like the *six degrees of separation* of the play and film of that title by John Guare, or the *Bacon numbers* associated with the film actor Kevin Bacon. It then discusses a paper written by two of these interconnected mathematicians that gives two ways of representing a rational number that were new to me. The paper also had some personal relevance. In discussing the subject matter of the paper I'll define a number of adverbs and a conjunction, the first serious use I've made of these J possibilities. Finally, I'll apply the verbs I define in connection with the almost 4000-year old Egyptian mathematics found on the Rhind papyrus.

Erdős Numbers

I heard a talk a few years ago given by the mathematician Ronald Graham, of the Bell Laboratories, in the course of which he discussed Erdős numbers. Graham had been a frequent collaborator with the mathematician Paul Erdős and even had a room set aside in his home for him. This was to facilitate visits by this eccentric nomad, who, in his later life, drifted from one collaborator to another, with only his suitcase and his mind, greeting his next host with the words, "My brain is open!"[1]. The web site:

<http://www.oakland.edu/~grossman/erdoshp.html>

is devoted to Erdős numbers. The next paragraph is copied from that source:

Most practicing mathematicians are familiar with the definition of one's Erdős number ... Paul Erdős (1913–1996), the widely-traveled and incredibly prolific Hungarian mathematician of the highest caliber, wrote hundreds of

mathematical research papers in many different areas, many in collaboration with others. His Erdős number is 0. Erdős's co-authors have Erdős number 1. People other than Erdős who have written a joint paper with someone with Erdős number 1 but not with Erdős have Erdős number 2, and so on. If there is no chain of co-authorships connecting someone with Erdős, then that person's Erdős number is said to be infinite.

Here are some data that I gathered from this web site, giving the number of people known to have each of the first several Erdős numbers (the site is updated periodically so the data will change from time to time):

Erdős number	0	1	2	3	4	5
1st kind	1	502	5713	26422	62136	66158
2nd kind	1	229	1969	8602	22668	36112

The row labelled "2nd kind" refers to a more stringent classification, described in the web site as follows:

The entire discussion so far has been based on linking two mathematicians if they have written a joint paper, whether or not other authors were involved. A purer definition of the collaboration graph (in fact, the one that Paul Erdős himself seemed to favor) would put an edge between two vertices if the mathematicians have a joint paper by themselves, with no other authors ... Let C' denote the collaboration graph under this more restrictive definition, and let us call the associated path lengths "Erdős numbers of the second kind" (and therefore call traditional Erdős numbers "Erdős numbers of the first kind" when we need to make a distinction).

Since those with Erdős number 2 got their numbers from writing with someone with Erdős number 1, we can get the average *promiscuity number* of the Erdős 1 authors by dividing their total into the total of Erdős 2 authors. The average number of co-authors for those with Erdős number 1 of the 1st kind is 11.4 and for those of the 2nd kind is a more selective 8.6. All sorts of people have Erdős numbers. You may be surprised to learn that Bill Gates has Erdős number 4, that Sir Francis Crick has Erdős number 7 and that his double helix collaborator Jim Watson has Erdős number 8.

This is the background you need in order to make sense of this message I received from Roger Hui about a year ago:

Subj: Erdos Number
 Date: 12/18/99 6:36:06 AM Pacific Standard Time
 From: RHui@Interlog.Com (Roger Hui)

To: EEMcD@AOL.Com (E.E. McDonnell)
CC: KEI@Interlog.Com (Kenneth E. Iverson)

Apparently my Erdos number is 2, having co-authored a paper^[2] with Shlomo Moran (during my grad school days in the early 1980's), who co-authored a paper with Erdos himself^[3]... Neither you nor Ken are in [the] list [of people with Erdős numbers 0, 1, or 2]. Therefore,... both you and Ken have a Erdos number of 3 or less, having co-authored a paper with me.^[4]

This was a surprise, since, although I have written a lot of semi-mathematical papers, I am not really a mathematician. Since there were three co-authors of the paper I wrote with Roger, I had an Erdős number 3 of the first kind, but not of the second kind. Similarly Shlomo Moran has an Erdős number 1 of the first kind, but not of the second, since there were three co-authors of his paper with Erdős.

Thus matters stood until a few months ago, when Roger sent me another message on the same subject:

Subj: Erdos Number
Date: 11/15/00 9:57:21 AM Pacific Standard Time
From: rhui@ADAYTUM-CAN.COM (Roger Hui)
To: EEMcD@AOL.Com (E.E. McDonnell)

According to <http://www.acs.oakland.edu/~grossman/erdoshp.html>, Jeffrey Outlaw Shallit wrote a joint paper^[5] with Erdos in 1991. Since you wrote at least one paper with Shallit ("Extending APL to Infinity")^[6], that makes you an Erdos 2.

Since Erdős died in 1996, I had now reached the highest I can get on the Erdős graph. What is more, since my paper with Shallit had just the two of *us* as authors, and Shallit's with Erdős had just the two of *them*, I now had Erdős number 2 of both the 1st and 2nd kind, and Shallit's Erdős number 1 is also of the 1st and 2nd kind. Among my fellow number twos are such luminaries as Albert Einstein, G. H. Hardy and Donald Knuth. As I chat with my fellow 2's, I appreciate the rather better class of thinkers they are, and wonder now that I was able at all to tolerate those arriviste 3's with whom I rubbed shoulders. I know personally only one other number two, and that is my friend Charles Brenner, about whom you may have read in my letter in the *Technical Correspondence* of the January 2001 *Vector*, vol. 17, No. 3, p. 112. He is today best known as *the forensic mathematician*. See his very interesting website at www.dna-view.com. Charles was just sixteen in 1961 when he and his father, the mathematician Joel Lee Brenner, collaborated on a paper^[7]. In 1987 the elder Brenner collaborated on a paper with Erdős^[8], thereby promoting his son Charles's Erdős number from infinite to 2. This raises a question. His father's collaboration with Erdős involved a total of six co-authors, so his father's Erdős number 1 is of the first kind only. Charles and his father were the *only* co-authors of their paper. How does this rank Charles? I'd say that the sins of the fathers should *not* be visited on the sons, so that Charles's number 2 is,

like mine, of the first *and* second kind. The promiscuity number of Joel Lee Brenner is 37; of Jeffrey Outlaw Shallit is 47; and of Shlomo Moran is 54, all three of them well above average.

I got to know Jeffrey Shallit when he was a young teenager with a gift for mathematics. He was a regular visitor to the IBM Philadelphia Scientific Centre when I worked there. When he went to college at Princeton University, he wrote his bachelor's paper on the mathematical aspects of my design of the complex floor function[9]. Jeff came to work for me at I. P. Sharp Associates in Palo Alto in the summer of 1979, after graduating from Princeton and just before beginning graduate work at the University of California at Berkeley. It was then that he and I wrote our joint paper on APL and the infinite. I wrote the part on infinite values, and Jeff worked out the details of infinite-sized arrays, including a very nice way of displaying them using a diagonal transformation. Upon getting Roger Hui's second message I emailed Jeffrey, who is now a full professor at Waterloo University in Ontario, Canada, saying how grateful I was to him for making this possible, but said that I didn't feel worthy of the honour. He answered

Subj: Re: thanks for raising my erdos number
Date: 11/19/00 5:35:42 AM Pacific Standard Time
From: shallit@graceland.uwaterloo.ca (Jeffrey Shallit)
To: Bemcd@aol.com

Au contraire, it's I who should be grateful to you. The paper that I wrote with Erdos was on expansions of the form

$$1/a - 1/(ab) + 1/(abc) - \dots$$

which I learned about from you in your APL article on "Spirals and Time"^[10]. If I hadn't had the opportunity to learn from you in Philadelphia and later in Palo Alto, I would never have explored this interesting topic!

Best, Jeff

This, unlike the essentially shallow glamour of my Erdős number, was something that I felt I could take legitimate pride in. The "Spirals and Time" article was a very early one of my articles. It gives me no little gratification when someone tells me they've enjoyed one. This message gave me an exceptionally large boost because Jeff had actually learned from it something useful to him professionally, and may even, as he suggests, have led to his involvement with Erdős. By the way, it had another satisfying repercussion, which I heard about from someone in Denmark who had shown my article to his fiancée. The article noted that the Gregorian calendar intercalation scheme had leap years in a 4-100-400 year cycle, but could be made exactly accurate if it were extended to a 4-100-400-3200-86400 cycle. The fiancée told Henry that on reaching the conclusion, where I noted that 86400 was also the number of seconds in a day, she "had an intellectual orgasm."

Notice that successive terms in the Gregorian sequence are multiples of the preceding term. That is, 100 is 25×4 and 400 is 4×100 . In the extension suggested in my article, 3200 is 8×400 and 86400 is 27×3200 . The sequence of multipliers 4 25 4 for the Gregorian sequence and 4 25 4 8 27 can each be used to determine the average length of the year in each system. In the next section I'll discuss how this may be done. For now, I'll only point out that the current method of adding leap seconds to a year is sufficient to make it unnecessary to do any extensions to the Gregorian sequence, so discussions (like this) concerning changing or extending the current Gregorian scheme are purely academic.

Pierce and Engel Expansions

That brings us, at last, to the *At Play With J* part of this paper.

The expansion in Shallit's message lies behind the design of the Gregorian calendar.

$$1/a - 1/(ab) + 1/(abc) - \dots$$

In the case of the Gregorian calendar the values a, b, and c are 4, 25, and 4.

$$'a \ b \ c' =: 4 \ 25 \ 4$$

The product scan ($\star/\backslash$) of this list gives the cumulative products, in this case defining the intervals in years when to intercalate and when not: every fourth year but not every hundredth year unless it is also a four-hundredth year.

$$\begin{array}{l}] \ m =: \star/\backslash \ a,b,c \\ 4 \ 100 \ 400 \end{array}$$

We reciprocate these values:

$$\begin{array}{l}] \ n =: \% \ x: \ m \\ 1r4 \ 1r100 \ 1r400 \end{array}$$

The alternating sum of this gives the part of a day p which, when added to 365, gives the average length of the year in calendar days in the Gregorian calendar.

$$\begin{array}{l}] \ p =: -/ \ n \\ 97r400 \end{array}$$

It also gives the number of leap years (97) in the cycle (400). One gets 97 this way:

$$\begin{array}{r} 400 \% 4 = 100 \\ 100 \% 4 = 1 \\ - / 100 = 1 \\ \hline 97 \end{array}$$

The number of days in 400 years gives the length of the Gregorian cycle in days, which keeps repeating as the millennia roll on.

$$400 * 365 + 97 * 400 = 146097$$

Shallit's *curriculum vitae* lists three papers, each bearing on the topic of *Pierce expansions*. These are not widely known, but they and the companion *Engel expansions* are the topics of the rest of this article. What are they? They are algorithms for converting a rational number into a series of integers, which, much like a continued fraction, give a way of representing a rational. The algorithm for Pierce expansions is described by Shallit in his paper on their metric theory as follows:

[Pierce expansion algorithm]: Given a real number x in $(0, 1]$, this algorithm produces the sequence of a_i such that $x = \{a_1, a_2, \dots\}$.

P1. [Initialize]. Set x_0 to x , set i to 1.

P2. [Iterate]. Set a_i to floor $(1/x_{i-1})$; set x_i to $1 - a_i x_{i-1}$.

P3. [All done?]. If $x_i = 0$, stop. Otherwise set i to $i + 1$ and return to P2.

Jeff points out that if x is a rational p/q , step P2 replaces p by $q \bmod p$, that this is less than p , and so eventually x will become 0, and the algorithm will terminate.

Several friends of mine asked if I could find out what Shallit's joint paper with Erdős was about. It dealt with Pierce expansions, but also with something called *Engel expansions*. All I could find out about them was that they involved sums of reciprocals, as opposed to alternating sums, but I was unable to work out the details on my own, so asked Jeffrey for help. His reply:

From: shallit@graceland.uwaterloo.ca (Jeffrey Shallit)
To: Eemcd@aol.com

Do you read Maple? Here is a maple program to compute the engel expansion of x up to the first n terms:

```
engel := proc(x,n) local xp, z, k;
xp := x;
z := [];
```

```

k := 0;
while ((k <= n) and xp <> 0) do
k := k+1;
y := ceil(1/xp);
z := [op(z),y];
xp := y*xp - 1;
od;
z;
end;

```

Basically you take ceiling of $1/x$, and that's the next output. Then you multiply that by your current x and subtract 1. Then continue.

Jeff

I'm not familiar with Maple, but I did manage to feel my way through this code. Having digested it, I concluded that the Pierce and Engel expansions were closely related. The Pierce list of integers implies an alternating sum, and the Engel list of integers implies a direct sum. This simplified my work in turning them into J , since one pattern would do for both.

To begin with, I changed Jeff's approach in two important ways: first, to simplify termination control, I would work only with rational arguments; second, instead of having essentially two main variables, a continually modified rational and a continually lengthening list of integers, I would combine the two, beginning with a scalar rational, and successively modifying this in two ways: first appending a tail, and then modifying the head. The tail extension would be obtained by applying an integer function to the reciprocal of the current head value, and appending this; the *floor* ($<.$) for Pierce expansions, and the *ceiling* ($>.$) for Engel expansions. The head modification would be obtained by applying a subtracting function to the product of the head and the tail, and replacing the head with this; the *one minus* ($-.$) function for Pierce expansions, and the *minus one* ($<:$) function for Engel expansions. Similarly, in computing the inverses to these functions, obtaining the rational from a list of integers, the same pattern would be used, with minus ($-$) for Pierce contractions, and plus ($+$) for Engel contractions. This is summarized in the following table:

	Pierce	Engel
tail	$<.$	$>.$
head	$-.$	$<:$
inverse	$-$	$+$

First I defined an adverb which would serve for both tail extensions:

```
BT =: 1 : '(, u. @ % @ {.) y.'
```

where *u.* stands for the appropriate integer function to be used in its place, floor or ceiling. It works by applying the appropriate integer function (*u.*) to the reciprocal (%) of the head ({.), then appending it (,). Here's how this works with each form of expansion:

```
<. BT 97r400
97r400 4
>. BT 97r400
97r400 5
```

Next comes an adverb for both head modifications:

```
BH =: 1 : '((0 } ~)u. @ ({. * {:))y.'
```

This replaces the head (0 } ~) with the appropriate subtracting function (*u.*) applied to the product of the tail and the head ({. * {:). For example:

```
-. BH <. BT 97r400
3r100 4
<: BH >. BT 97r400
17r80 5
```

Next, these are combined in a conjunction that will allow a step function to be defined for both expansions:

```
BS =: 2 : '(u. BH)@(v. BT)y.'
```

Here the *u.* and *v.* stand for the left and right function arguments to be used. For example:

```
-. BS <. 97r400
3r100 4
<: BS >. 97r400
17r80 5
```

A control structure is needed to allow the steps to be applied as often as necessary. This requires a sequence of two uses of the power conjunction; the first to control termination, with a right argument which gives the signum of the head (* @ {.). This will be *one* for any nonzero head value (I assume the argument is always positive), which allows the function to be applied; when the head eventually becomes *zero*, as it must since it is continually being reduced, the function will not

be applied, and the result will be the same as the argument. The second use of the function power conjunction will cause the steps to be applied to the limit, that is, until two successive results are equal. The convention proposed by Iverson[11] is that positive infinity (∞) be used to describe application of a function to the limit. Nicely enough, this proposal was seconded by Shallit and me in our paper on infinities, and it is now part of J.

```
CS =: (^:(* @ {.)) ^: _
```

We can use CS with both steps:

```
(-. BS <.)CS 97r400
0 4 33 100
(<: BS >.)CS 97r400
0 5 5 16
```

These results show that the head is indeed zero. The zero is extraneous, so now we define two functions that yield just the needed Pierce and Engel expansion from a rational. We only have to behead ($\{ \}$) the results we just got:

```
PR =: 3 : '}'. @((- . BS <.)CS)y.'
ER =: 3 : '}'. @(<: BS >.)CS)y.'

PR 97r400
4 33 100
ER 97r400
5 5 16
```

Let's define the functions inverse to PR and ER and check whether each expansion contracts to 97r400. The method is essentially the same for both, so again we define an adverb that applies the appropriate subtraction function to insert ($u. /$) between the result of reciprocating ($\%$) and product scanning ($* / \backslash$) our lists of integers:

```
RB =: 1 : 'u. / * / \ % y.'
```

and this makes the inverses easy to define:

```
RP =: - RB
RE =: + RB
```

So now we use each of them:

```

      RP PR 97r400
97r400
      RE ER 97r400
97r400

```

So the expansions contract properly. We saw above that the list 4 25 4 contracted to 97r400, and now have verified that the list 4 33 100 does as well. In other words, both intercalation schemes will give an exact Gregorian year. The difference is that the cycle for the present Gregorian year is 400 years; for a 4 33 100 calendar the cycle is 13,200 years. For each, the resulting average year length is 365.2425 days.

By the way, the result 97r400 is given rather than the decimal equivalent 0.2425 because the results of PR and ER are both rationals, the same type as their arguments. See what happens if one just types in the numbers:

```

      RP 4 33 100
0.2425
      RE 5 5 16
0.2425

```

The functions PR and ER will work properly only when applied to rational arguments.

Engel Expansions and Gypsy Math

The very ancient document called the Rhind papyrus includes a table of all fractions of the form $2/n$ from $2/3$ through $2/101$, and for each gives a list of from two to four unit fractions that sum to it. For example,

$$\begin{aligned}
 2r3 &= 1r2 + 1r6 \\
 2r61 &= 1r40 + 1r244 + 1r488 + 1r610
 \end{aligned}$$

The Engel expansions of rationals of the form $2/p$ for the first four odd primes give the same results as those listed in the Rhind papyrus:

```

BE =: [: % */\
      (BE @ ER)"0 [ 2r3 2r5 2r7 2r11
1r2 1r6
1r3 1r15
1r4 1r28
1r6 1r66

```

This is not generally true, however. For example, the Rhind values for $2r21$ are $1r14$ and $1r42$, whereas the rationals given by the Engel expansion are $1r11$ and $1r231$. However, I am now in a position to bring some unfinished business to a close, that I've left in abeyance since June, 1981. In my last column as Recreational APL editor for *APL Quote Quad*, in a section called "Gypsy Math" I wrote:

The Rhind papyrus ... shows, for each odd integer from 3 to 101, several integers whose reciprocals sum to $2 \div w$. For example,
 $(2 \div 17) = + / \div 9 \ 153$. Write a function F such that, for odd positive argument w ,

```
(2 ÷ w) = + / F w
2 = ρ F w
~ w ∈ F w
```

Thus, $F \ 17 \leftrightarrow 9 \ 153$.

I'm impressed by the fact that twenty years ago I knew how to give unit fraction results for Rhind fractions such as $2r17$ without knowing anything about Engel expansions. However, the Engel expansion is completely general, and will handle any positive rational less than 1. Our Egyptian predecessors probably used a simpler formula which we would write in J notation as:

```
Egypt =: [: */\ (,~ -:@>:)
Egypt"0 [ 3 5 7 11
2 6
3 15
4 28
6 66
```

Postscript

Erdős, by the bye, has a *Bacon number* of 4. Schechter explains how this came about in his book on Erdős:

A mathematician and sometime actor named Gene Patterson appeared briefly in the 1993 documentary about Erdős, *N is a Number*. Patterson also had a role in *Box of Moonlight* with John Turturro, who was in *The Color of Money* with Tom Cruise, who appeared in *A Few Good Men* with Kevin Bacon.

And Jeffrey Shallit is the proud possessor, in addition to his Erdős number of 1, of an *Elvis number* of 3.

If you can't guess what this is, you can read all about it at his web site:

<http://www.math.uwaterloo.ca/~shallit>.

References

- [1] Schechter, Bruce, *My Brain is Open, the mathematical journeys of Paul Erdős*. Simon & Schuster, New York, 1998
- [2] O. Ibarra, S. Moran and R. Hui, A Generalization of the Fast LUP Matrix Decomposition Algorithm and Applications, *Journal of Algorithms* 3 (1982), 45-56.
- [3] P. Erdős, N. Linial and S. Moran, Extremal Problems on Permutations under Cyclic Equivalence, *Discrete Mathematics* 64 (1987), 1-11.
- [4] Hui, Roger K. W., Kenneth E. Iverson, E. E. McDonnell, Arthur T. Whitney, APL\?, *APL90 Conference Proceedings*, 192-200
- [5] Erdős, Paul, and Jeffrey Shallit, New bounds on the length of finite Pierce and Engel series, *Séminaire de Théorie des Nombres de Bordeaux* 3 (1991), 43-53.
- [6] McDonnell, Eugene E. and Jeffrey Shallit, Extending APL to Infinity, *APL80*, Noordwijkerhout, 123-132
- [7] Brenner C. H. & J. L. The popularity of small integers as primitive roots, *Numer. Math.* 4 (1962), 336-342
- [8] Brenner, J. L., Beasley, L. P., Erdős, Paul, Szalay, M., Williamson, Alan Glynne; Generation of alternating groups by pairs of conjugates, *Period. Math. Hungar.* 18 (1987), 259-269.
- [9] McDonnell, E. E., Complex Floor, *APL Congress* 73, Copenhagen
- [10] McDonnell, E. E., Spirals and Time, *APL Quote Quad*, 7, (Winter 1977), 4 20-22
- [11] Iverson, K. E., *Operators and Functions*, RC 7091, IBM Corp. Yorktown Heights, NY (1978)

A New Life For ∇ : Structure Control

F.H.D. van Batenburg (Batenburg@rulsfb.LeidenUniv.nl)

1. Introduction

In the years after Edgar Dijkstra wrote his complaint "GOTO considered Harmful" (Dijk68), an important discussion followed among software professionals regarding structured programming. APL did not join in this discussion. As a result of that discussion, program languages implemented concepts to enable programmers to program in a structured way. APL however remained in splendid isolation, cherishing its beloved jump " $\rightarrow$ ".

This is very unfortunate because the tools you have influence the way you think. The essence of the discussion was not so much about better tools, but about thinking. Having jumps let you think in flow whereas structured concepts make you think in structure (blocks if you like) which is much better.

It was only in 1978 that Wiedman (Wie78), Reeves and Besemer (Ree78) started the discussion in APL. After their initiative, many authors followed with proposals to add control structures (see long list of references compiled by De Kerf (Ker86)) but APL vendors effectively ignored this. Finally, in 1994 Manugistics came with an implementation that copied the concepts as well as the keywords of conventional languages. A few years later in 1996, Dyalog APL followed suite and so did J; all of them copying the concepts and the keywords.

To my opinion, this is not good. Granted, these control structures have been essential improvements to hitherto unstructured languages and these concepts are worthwhile tools to structure a program. But why should APL slovenly copy these implementations lock, stock and barrel? After all, " $\text{COS}(X)$ ", " $\text{MIN}(X,Y)$ " and " $\text{CEIL}(X)$ " are nowadays also part of the standard repertoire of most languages (and rightfully so), but nevertheless APL programmers are much happier with " $2 \circ X$ ", " $X \downarrow Y$ " and " $\uparrow X$ ".

My point is that APL should copy the ideas of useful concepts (and possibly improve upon them), but not copy the way such concepts are implemented and certainly not a keyword implementation. Instead, APL should have chosen an APL-ish implementation using symbols instead of keywords and preferably one powerful symbol that does it all.

2. A Better Structure Control

APL2C has chosen such an APL-ish implementation for structure and structure control. APL2C is an APL implementation that is developed by T.Otto (Ott00). It is a fully functional APL2-type implementation. The name APL2C is chosen because one can produce C-source code from a workspace. This enables programmers to modify the C-code at will, add other modules if needed, and at the end compile the whole combination into an executable.

The implementation for structure control is based on a further development of my ideas as published in Bat82-Bat94. In APL2C the symbol that does all the structuring and its control is the del "v" which for that reason will be called *control*. The choice of this symbol agrees well with its original purpose to indicate start and end of function definition. As Willhofs argued, for a long time the only block available in APL was a function, so "v" seems a suitable symbol for blocks (Wil93). This does not clash with the function definition anymore because function definition is done by)EDIT or by double click (as is common in most Windowing implementations). Symbol *control* is used monadically to indicate the beginning of a block and it is used niladic to mark the end of a block.

Iteration

An example will show how this works. We begin to show iteration. The following code:

```
v 4+5
  Z←'HALLO'
  Z←'WORLD'
v
```

is a block that will be executed 9 times; first the expression at the right of the monadic *control* is executed and the resulting value 9 will instruct control to print out 9 times two lines with *HALLO* on the odd lines and *WORLD* on the even lines.

Condition

The previous example showed iteration, but condition is quite similar. For loops, the expression at the right of control is an integer indicating how many times to execute the block, for conditions it suffices when the integer is either one or zero. For example to do something only if *FLAG* is non-zero you could program:

```
v 0≠FLAG
  ::some interesting computation::
v
```

While

The examples above have shown how condition and iteration is implemented in APL2C. The while-construction is implemented in a similar way. After all, a while-construct is an iteration, but an iteration that is executed as long as the controlling expression is true. Therefore the implementation is identical to the fixed iteration, however, to tell APL to recheck the controlling expression a negative value is used.

For example to read input until the user enters an empty line:

```

▽ 0≠ρ,IN+,⌈
  :: some interesting computation::
▽

```

Another example that codes an iteration until the result does not change any more is:

```

OLD←1+NEW←INITIALVALUES a initially different values
                           for NEW and OLD
▽ OLD≠NEW
  OLD←ANALYSE NEW←OLD
▽

```

Alternatives

What about alternatives like IF-ELSE and SELECT? To program alternatives APL2C needs a vector for the right argument of *control*. This vector prescribes what to do for each alternative. The alternatives in the block are marked by a label-less colon ":". If the number of alternatives do not match the length of the vector, APL2C returns a *LENGTH ERROR*. An example will illustrate this:

```

▽ 2 3∈⌈NC'XXX'
  :Z+'VARIABLE'
  :Z+'FUNCTION'
▽

```

This will print *VARIABLE* if the ∇ -expression yields 1 0, it prints *FUNCTION* if the expression yields 0 1 and it will print nothing if the expression is 0 0. Just hypothetically, imagine that the ∇ -expression yielded "2 7". In that case APL2C would have printed 2 times the word *VARIABLE*, 7 times the word *FUNCTION* and after that (because of the negative number -2) APL2C would execute the ∇ -expression again.

Nesting

It is no problem to create nestings in APL2C. Just to wrap the whole idea up in one complicated (not so realistic) example, imagine the following code:

```

VCN←-2 3×COND1 acontrol1
:action1
  action2
:V10          acontrol2
  action3
  action4
▽
▽

```

If *CN* get the value -2 0 then *action1* and *action2* are done twice after which APL2C does the expression in *control1* again.

If *CN* get the value 0 3 then *action3* and *action4* are done, and done another 9 times to satisfy the value 10 in *control2*, and this is done another 2 times to satisfy the 3 in *control1*. As there was no negative number in *control1*, this ends the total block.

Iteration counter

In the previous examples, the iterations did not need a variable to keep track of the iterations. Sometimes you do not need one, but there are often occasions where it is convenient to know the iteration number. If so, you could of course explicitly program this, but it is more convenient if such a counter is available.

The iteration count can be found in Iteration Counter "+ZIC". This variable is a 2-row matrix with one row for the current iteration counter, and the second row for its terminal value. So within one single block, it is a one-column matrix. With nested blocks "ZIC[;1]" gives the current counter and the final value for the innermost block and the last column gives the counter and final value for the outer-most block.

For example:

```

V5 3 -8 0
: ~~~~~ a here ZIC is 2 1p(1-5),5
:V4      a here ZIC is still 2 1p(1-3),3
  ~~~~~ a but here ZIC is 2 2p(1-4),(1-3),4,3
  ▽
: ~~~~~ a here ZIC is 2 1p(1-8),-8
: ~~~~~ a alternative 4 is never executed
▽

```

Single-line shortcut

For small actions, a shorter alternative is available for iterations and conditions. This is the single-line alternative where *control* is used dyadically with controlling expression at the right and the action at the left. For example:

```
LEFT←3▽2×□NC'LEFT' ⌞ Only assignment ifor missing LEFT
□+ 'Thank you...'▽1000 ⌞ Thousand times thanks
ANALYSE INV-0×pIN+,□ ⌞ Analyse as long as input is given
```

Formatting

The examples that were presented not only showed the working of *control*, but also how APL2C formats the blocks. Each block is indented one space and the colon for alternatives is outdented so as to show to which *control* they belong.

3. Conclusion

You have seen how APL2C handles structure and its control. In this implementation all structured control concepts are blended together in one simple, but very powerful, construct *control*. This takes the essence of structured control, but implements it more in the spirit of APL. The result are programs where structure is better recognized than in current alternatives.

For a more complete example, the Vector website shows two programs that were structured to begin with. Both programs are shown in their original programming style as well as a remake using *control*. It shows how the programs get shorter and more comprehensible.

If you are interested in experiencing how this works you could download a demo version of APL2C. You can find more information as well as a full featured trial version with a time limit of half a year, at website: [//www.apl2c.com](http://www.apl2c.com).

4. Literature

- Bat82 Batenburg,F.H.D.van(1982): New control structures in APL. Quote Quad.13(2)15-20
- Bat85 Batenburg,F.H.D.van Prins,R.(1985): QL/APL: A leap back - or forward? Vector 2(2)115-117
- Bat88 Batenburg,F.H.D.van(1988): The Code-line: A "New" Structured Concept. Quote Quad.19(2)14-19

- Bat91 Batenburg,F.H.D.van (1991): L-one-two-three (L1:...L2:...L3:...) considered harmful. Proc. APL91 p.330-337
- Bat94 Batenburg,F.H.D.van (1994): Restriction riddance: more power to parentheses. Quote Quad 25(1)220-224.
- Dijk68 Dijkstra,E.W.(1968): GOTO statement considered harmful. ACM Comm. 11(3)147-148
- Ker86 Kerf,J.L.F. De (1986): APL and structured programming: an annotated biography. APLCAM 8(2)340-363
- Ott00 Otto,T.(2000): Oral presentation at APL conference 2000
- Ree78 Reeves,A.P. Besemer,J.(1978): Special control structures in APL. Quote Quad.9(2)23-31
- Wie78 Wiedman,C.(1978): Whither (wither?) control structures. Quote Quad 9(2)21-22
- Wil93 Willhoft,R. (1993): Structured APL: a proposal for block structured control flow in APL. Proc APL93 p.304-315
- F.H.D. van Batenburg
Theoretical Biology of Leiden University
Kaiserstraat 63
2311GP Leiden
The Netherlands
email: Batenburg@rulsfb.LeidenUniv.nl
<http://wwwbio.LeidenUniv.nl/~Batenburg/index.html>

J Rational Number Utilities & Verification of Knuth's Constants

by Gordon Sutcliffe

Overview

The utilities presented produce higher precision results than normal floating point. It is hoped that they, and their use to verify Knuth's constants (Ref 1), are of interest.

The utility routines are applicable to other calculations requiring up to about 50 decimal places. Given further effort, the utilities could be presented as a high precision package.

Practical Calculations with Rational Numbers

J provides rational arithmetic giving exact rational results. However, long calculations can lead to lengthy rational number representations which may require extensive storage and are slow to process. In these circumstances the theoretical infinite precision of rational numbers can be sacrificed by rounding values to an appropriate precision. In any case, any irrational values required, have to be rounded (approximated) in order to be held as finite length rational numbers.

For example, the repeated squaring of π (an 80 decimal place approximation to the well known mathematical constant), shows that the length of the rational representation of the results just about doubles at each squaring:

```
0j5": drl *: ^: (i.5) pi
drl: 79 79
drl: 157 157
drl: 314 313
drl: 628 625
drl: 1256 1249
3.14159 9.86960 97.40909 9488.53102 90032220.84293
```

Where the `drl` utility displays the numerator and denominator lengths of the rational number results. Continued squaring would eventually fill the available memory or be so slow that one is not prepared to wait for the result. Even using

the simple rational, 22r7, as an approximation to pi, creates rational values with thousands of digits after squaring 10 times.

The utilities presented round [intermediate] results to keep memory requirements and execution times within practical limits to achieve high, but not infinite, precision. The programmer and user then have the added responsibility of ensuring that the rounding is appropriate.

The Utilities

Maths Rational Utilities

The Maths utilities mimic similar built-in J functions, but with a restricted argument range:

- No complex arguments or results (applies to all the utility functions)
- Limitation of arguments to the range of J floating values (say $\pm 1.6e308$)
- Result values may be restricted to those near unity

Name	Description	J built-in function mimicked	Algorithm Type	Approximate Arg. Limits at 45dp	Arg. Type	
					x	y
ca	Cosine	monad 2&o.	as sa	as sa	[N]	I, Q, X
ea	Exponential	monad ^	Series	70x	[N]	I, Q, X
fa	Factorial	monad !	Series	_1< to <=6	[N]	I, Q, X
ge	Euler's constant, e	(none)	Series	not applicable	[N]	I, Q, X
pa	Pi	(o. 1)	J (o.)	not applicable	[N]	I, Q, X
la	Natural logarithm	monad ^.	Series	10^300x	[N]	I, Q, X
ra	(Square) Root	monad %:	J (%:)	10^80x	[N]	I, Q, X
sa	Sine	monad 1&o.	Series	+/- pi*10^3x	[N]	I, Q, X

pa and ra make use of J built-in o. and %: monadic functions which produce extended integer results as a special case.

Argument types are: N any numeric; I integer; Q rational and X extended integer. In all cases the result is a rational value. Where the optional x argument is present, it is an integer specifying the number of decimal places of the result of the function (the default is the value in global variable T). The right argument may not be a floating or complex type numeric.

Basic Rational Utilities

Name Valence Description

BN	Monad	Bernoulli numbers (requires extended integer argument for rational result)
cdp	Dyad	Number of common decimal places in two rational numbers. cdp is
Cdp	Dyad	limited to argument differences of magnitude less than about 5e17
dr1	Monad	Display rational length (numerator and denominator) and return rational
flg	Monad	Floor of base 10 logarithm of the rational argument (currently unused)
l2r	Monad	Literal representation (of an integer, decimal or floating value) to rational
rr	Ambiv- alent	Round rational. Reduces the length of a rational value where it can. No attempt is made to find the best approximation possible.

Notes:1) cdp Cdp, and dr1 are mainly for diagnostics

2) l2r enables rational numbers to be written as literal floating or decimal numbers

Constants for the Rational Utilities

These are rational constants required by the utilities, but some may be useful to the J programmer directly. They are given to 80 decimal places including 5 guard digits.

Name(s)	Description
e1	Base of natural logarithms, e
ln2, ln10	Natural logarithms of 2 10
lnu, ln x, ln y, ln z	Natural logarithms of 33r32 1025r1024 32769r32768 1048577r1048576
pi	Pi

The constants were produced using the J script itself, with the global control variable `ctr` set to zero. Thus, the script is not dependant on numerical values from any earlier work except in so far as such values are included within J itself.

Calculation of Knuth's Constants

Run Constant Table script

The call `RCT KC` produces a listing of most of the constants in Table 1, Appendix 1 of Knuth's book (Ref 1). It might be expected that these could be extended to more and more decimal places. However, 50 decimal places is about the limit with the current utilities.

The 31 constants are calculated in about 5 seconds on an IBM compatible PC. The time is the time to run `RCT KC` on a system with an 233MHz Pentium II processor and 64 MB RAM. The time does not include the precalculated constants in the source file accessed by `RCT`, and their calculation times would need to be taken into account for an objective timing.

Run Constant Table script Diagnostic

The `RCTD` function the displays number of common decimal places between pairs of results in order of increasing precision. It can be a useful diagnostic. A typical call is:

```
RCTD KC      NB. Execution time 34 seconds
```

Knuth's Constants Table Data

The `KC` table contains three fields for each of Knuth's constants (Ref 1):

- 1) Line No.;
- 2) Short description of constant;
- 3) J script to calculate the constant

The relation $(x^y) = e^{a y} * 1^a x$ is used in 3) to calculate the power of a number in terms of the available rational utilities, `ea` and `1a`.

Alternative J source has been used to calculate the constants in some cases. As an example, consider an alternative J script for pi:

```
0j45": *:fa _1r2
3.14159 26535 89793 23846 26433 83279 50288 41971 69133
```

When compared with Line 12 in the calculated constants, the difference is `_266e_45` within the 5 guard digits, giving increased confidence in the utility results.

Reference

- [1] Knuth, Donald E: *The Art of Computer Programming Vol 1*, First edition 1968; Addison-Wesley.

The J Source

The J source code is provided 'as is', and consists of about 300 lines (4 A4 pages of J10pt).

If problems are encountered, a reduction in the number of decimal places will often enable a result to be obtained. This may be achieved by reducing the value in:

- 1) the global variable T, the default precision, or
- 2) the left arguments of a utility, where a utility precision is set explicitly

Verification of Knuth's Constants (Ref 1)

The constants which have been recalculated are in Table 1, Appendix 1 of Knuth's book 'The Art of Computer Programming'. Of the 32 constants, one, zeta, was omitted due to problems finding a suitable algorithm. The remaining 31 results agree with Knuth's values (40 decimal places).

Although the results agree with Knuth's excellent work, the exercise was a very useful debugging exercise for the Rational Number utilities.

RCT KC

Nominal calculation precision, T, 45 decimal places. Results correct to 40 decimal places

LN	Constant	Time	Memory	J Code	D.Places:	10	20	30	40				
0)	Sqrt 2	0.007	50240	ra 2	1.41421	35623	73095	04880	16887	24209	69807	85696	71875
1)	Sqrt 3	0.007	63744	ra 3	1.73205	08075	68877	29352	74463	41505	87236	69428	05253
2)	Sqrt 5	0.006	42688	ra 5	2.23606	79774	99789	69640	91736	68731	27623	54406	18359
3)	Sqrt 10	0.005	39104	ra 10	3.16227	76601	68379	33199	88935	44432	71853	37195	55139
4)	Cube root 2	0.284	184256	ea (%&3) la 2	1.25992	10498	94873	16476	72106	07278	22835	05702	51466
5)	Cube root 3	0.360	153664	ea (%&3) la 3	1.44224	95703	07408	38232	16383	10780	10958	83918	69253
6)	Fourth root 2	0.013	50368	ra ra 2	1.18920	71150	02721	06671	74999	70560	47591	52929	72092
7)	ln 2	0.020	79680	la 2	0.69314	71805	59945	30941	72321	21458	17656	80755	00134
8)	ln 3	0.129	153088	la 3	1.09861	22886	68109	69139	52452	36922	52570	46474	90557
9)	ln 10	0.148	140608	la 10	2.30258	50929	94045	68401	79914	54684	36420	76011	01490
10)	1/ln 2	0.023	79680	% la 2	1.44269	50408	88963	40735	99246	81001	89213	74266	45955
11)	1/ln 10	0.149	140608	% la 10	0.43429	44819	03251	82765	11289	18916	60508	22943	97006
12)	pi	0.010	50048	pa 0	3.14159	26535	89793	23846	26433	83279	50288	41971	69399
13)	pi/180	0.005	67712	pi%180	0.01745	32925	19943	29576	92369	07684	88612	71344	28719
14)	1/pi	0.005	64064	% pi	0.31830	98861	83790	67153	77675	26745	02872	40689	19291
15)	pi squared	0.010	167552	*: pi	9.86960	44010	89358	61883	44909	99876	15113	53136	99407
16)	Sqrt pi, Gamma 1/2	0.007	43648	ra pi	1.77245	38509	05516	02729	81674	83341	14518	27975	49456
17)	Gamma 1/3	1.287	171712	G1r3=.fa _2r3	2.67893	85347	07747	63365	56929	40974	67764	41286	89529
18)	Gamma 2/3	0.026	147200	+:pi%G1r3*ra 3	1.35411	79394	26400	41694	52880	28154	51378	55193	27190
19)	e	0.019	82240	ea 1	2.71828	18284	59045	23536	02874	71352	66249	77572	47094
20)	1/e	0.018	82368	ea _1	0.36787	94411	71442	32159	55237	70161	46086	74458	11131
21)	e squared	0.029	130176	ea 2	7.38905	60989	30650	22723	04274	60575	00781	31803	15571
22)	Euler's const gamma	0.687	170112	EC=.ge 1e2	0.57721	56649	01532	86060	65120	90082	40243	10421	59334
23)	ln pi	0.170	156032	la pi	1.14472	98858	49400	17414	34273	51353	05871	16472	94812
24)	phi, Golden Ratio	0.012	63744	->: ra 5	1.61803	39887	49894	84820	45868	34365	63811	77203	09180
25)	e to power gamma	0.242	194816	ea EC	1.78107	24179	90197	98523	65041	03107	17954	91696	45209
26)	e to power pi/4	0.249	174080	ea (%&4) pa 0	2.19328	00507	38015	45655	97696	59278	73822	34616	37641
27)	sin 1	0.466	161280	si=.sa 1	0.84147	09848	07896	50665	25023	21630	29899	96225	63063
28)	cos 1	0.018	83136	ra 1-*:si	0.54030	23058	68139	71740	09366	07442	97660	37323	10417
29)	zeta 3, Sigma 1/n^3	0.000	2112		Not calculated								
30)	ln phi	0.259	148672	la ->: ra 5	0.48121	18250	59603	44749	77589	13424	36842	31351	84335

Appendix: Listing of J script

NB. File: ratnum9.ijs

Rational Utilities (J Ver 405d)

Wed 15/11/2000

ctr =: 1

NB. 0: Initialise constants; 1: Normal

diag =: 0

NB. 0: Normal; 1: Display loop counts in ea etc

dsp =: 1!2&2

NB. Display y. and result y.

T =: 45

NB. Default rounding decimal places

NB. Utilities for Rational Numbers

BN =: 3 : 0

NB. Bernoulli Numbers. Argument is an extended integer

NB. SYNTAX: BN rat_no_of_Bernoulli_nos_to_create (starting at BN0)

B=. ''

NB. Initial results vector

d=.1,<:y.)#0

NB. Initial denominator coefficients

r=.%<.@!>:i.y.

NB. Initial divisor coefficients

for. i.y. do.

NB. Loop for algebraic division

B=B, q=. d %&{. r

NB. Quotient is scalar q

d=.)d-q*r

NB. Update partial dividend (drop initial 0)

r=.)r

NB. Drop first/last items of d & r

end.

B = <.@!i.y.

NB. Bernoulli Numbers

)

NB. Bernoulli Numbers for k=0 1 2 (2) 50 ie. omitting bn having zero values

bnnz =: (~:&0 #]) BN 51x

NB. Bernoulli Nos. Non-Zero

drl =: 3 : 0 NB. Display Rational (and Extended) number component Lengths

NB. SYNTAX: drl scalar_rational_no

([[: dsp@('drl: '&,)@': '#;._2 @(&@'_'@{. }.]) @,&r'@:')"0 y.NB. Display N & D lengths

NB. y.

NB. Pass y. on as result

)

flg =: (3 : 0)"0 NB. Floor of (base 10) logarithm of rational

NB. SYNTAX: flg rat_no

if. y.>:1 do. _1 + # 0" = <. y.

NB. Log base 10 of y.

elseif. y.>0 do. (-@# + =&'1'@{. *. */@:=&'0'@{.}) 0" = >. % y.

NB. Log base 10 of y.

elseif. y.=0 do. x: _ return.

NB. Case for zero argument

elseif. do. 0j1 [dsp 'flg: ERROR: negative argument' return.

end.

)

12r =: 3 : 0 NB. Literal (J format float or integer) value to Rational number

NB. SYNTAX: 12rx literal_value

NB. y.= _ . is not handled

'A E'=. 2{. <:._2 y., 'e0e'

NB. Arg and Exp of equivalent float

'i f'=. 2{. #;._2(>A), '..'

NB. Integer and Fraction lengths

". ([#~ '&~:(>A),((0>."E)#0'), 'r1', (f-0<."E)#0' NB. Rational result

)

NB. x.>=0 Return 1 if two rationals in y. differ within specified dp, otherwise 0
 rdf = [: -,@(x/):@:=/ (j.@[" :])"0 NB. Rat.Diff; x: dps_acc; y: two rational nos

```
rr =: (3 : 0)"0 NB. Round Rational
T rr y. NB. Monad definition
:
NB. SYNTAX: [dec_or_sig_places] rr rat_no
if. 0=y. do. y. return. end. NB. Return rat 0 if zero
if. 0<x. do. 12r (j.x.) ": y. NB. Round to x. decimal places (x.<=0)
else. 12r (j.0>.-x.+>:f1g|y.)": y. NB. Round to -x. sig figs
end. NB. %&ln10 60 1a is better than 10^.
```

NB. Constants: lnu to lnz, e 1, ln 2, ln 10 and pi to 80 dps (last place(s) are in doubt)
 NB. (ea lnu) to (ea lnz) have exact rational representations of 33r32 to 1048577r1048576
 lnu =:12r'0.03077165866675368837102820759677216409169673995889035634986199530109980728337209'
 lnz =:12r'0.00097608597305545889596082490801718667261183433378453623775859827440037212402588'
 lny =:12r'0.00003051711247318637856906951416899468124304989470989364165971885272442917955964'
 ln2 =:12r'0.00000095367386165918823390841551496333614360314807097930285811225469343381836078'
 e1 =:12r'2.71828182845904523536028747135266249775724709369995957496696762772407663035354759'
 ln2 =:12r'0.69314718055994530941723212145817656807550013436025525412068000949339362196969472'
 ln10=:12r'2.30258509299404568401799145468436420760110148862877297603332790096757260967735248'
 pi =:12r'3.14159265358979323846264338327950288419716939937510582097494459230781640628620900'

```
cdp =: -(10&^.)@(x:~;_1&.%)+:@|@- NB. Common dps in pair of rational numbers
Cdp =: -%&ln10@{50&1a}+:@|@- NB. Common dps in pair of rational numbers
```

NB. Operations with Rational numbers

```
ca =: (3 : 0)"0
T ca y. NB. Monad definition
:
x.&sa y.+~:pi NB. Cosine; ? accuracy for small |y.
)
```

```
ea =: (3 : 0)"0 NB. Rational Exponential Approximation
T ea y. NB. Monad definition
:
```

```
NB. SYNTAX: [dps_accuracy] ea exp_rat_arg
y.=.(x:y.)-m=.ctr*+.0.5+y. NB. Scale y. m times by 1
y.=.(x:y.)-lnu* mu=.ctr*+.0.5+y.%lnu NB. Scale y. mu times by 1r32 approx
y.=.(x:y.)-lnx* mx=.ctr*+.0.5+y.%lnx NB. Scale y. mx times by c. 1r1024
y.=.(x:y.)-lny* my=.ctr*+.0.5+y.%lny NB. Scale y. my times by c. 1r32768
NB. y.=.(x:y.)-lnz* mz=.ctr*+.0.5+y.%lnz NB. Scale y. mz times by c. 1r1048576
```

```
NB. dsp m, mu, mx, my, mz
PS =. 0r1 NB. Initialise accumulator, Previous Sum
r =. 0 NB. Initial r of series
t =. 1 NB. Initial term of series
while. x. rdf PS, R=.PS + t do. NB. Test Previous Sum against Current Sum
r =. >:r NB. Increment term number
t =. x. rr t*y.%r NB. Update t to next term
PS=. x. rr R NB. Partial result
```

```

end.
if. diag do. dsp 'ea terms: ', ":r end. NB. Diagnostic
R =. R * 33r32^mu NB. Reverse scaling 33r32
R =. R * 1025r1024^mx NB. Reverse scaling 1025r1024
R =. R * 32769r32768^my NB. Reverse scaling 32769r32768
NB. R =. R * 1048577r1048576^mz NB. Reverse scaling 1048577r1048576
fi=: x.&rr@%e1 :.(x.&rr@%e1) NB. Define obverse function
x. rr fi ^: m R NB. Reverse scaling of result
)

la =: (3 : 0)"0 NB. Rational Natural logarithm Approximation
T la y. NB. Monad definition
;
NB. SYNTAX: [dps_accuracy] la ln_rat_arg
if. diag =. 0=y. do. dsp 'la terms: 0' end.
if. 0=y. do. __ return. end.
y. =. y.* 2x^ - m=.ctr<.0.5+-%&ln2^2 x: y. NB. Scale y. m times by 2
y. =. y.* 33r32^ - mu=.ctr<.0.5+-%&(x^:_1 lnu)^2 x: y.NB. Scale y. mu times by 1.01
y. =. y.* 1025r1024^ - mx=.ctr<.0.5+-%&(x^:_1 lnx)^2 x: y.NB. Scale y. mx times by 1.01
y. =. x. rr y.* 32769r32768^ - my=.ctr<.0.5+-%&(x^:_1 lny)^2 x: y.NB. Scale y. my times b
y 1.01
NB. dsp mu,mx,my,mz
x=.(<: % >: )y. NB. x =. (y.-1)/(y.+1)
R =. 0 NB. Initial accumulator
r =. 1 NB. Initial term no.
i =. x NB.
x2=. x. rr *:x NB.
while. x. rdf R , R1 =. x. rr R + i % r do.
i =. x.rr i*x2 NB.
r =. r+2 NB.
R =. R1 NB.
end.
if. diag do. dsp 'la terms: ', ": -:r+1 end. NB. Diagnostic
NB. R=. x. rr (mh*lnh) + +:R
R=. (mu*lnu) + +:R
R=. (mx*lnx) + R
R=. (my*lny) + R
NB. R=. (mz*lnz) + R
x. rr (m*ln2) + R NB. Reverse scaling
)

ra =: (3 : 0)"0 NB. Rational Approximation to square_root (Newton's Method)
T ra y. NB. Monad definition
;
NB. SYNTAX: [dps_accuracy] ra Root_arg
(10^x:x.) %~ <.@%: y.*10^x+:x. NB. Root as Roger Hui's pi
)

sa =: (3 : 0)"0 NB. Rational Sine Approximation
T sa y. NB. Monad definition
;
NB. SYNTAX: [dps_accuracy] sa sin_arg
a =. -&(+:pi) ^: (m=.<.0.5+ -:pi%~ y.) y. NB. Scale y. m times by 2pi

```

```

ps =. *a                                NB. Post calculation result multiplier
a =. | a                                NB. Modulus of a
if. a>:-pi do. a=.pi-a end.             NB. Supplementary angle test
if. s=.a>pi%4 do. a=.a--:-pi end. NB. Set s for sin: 0 or cos: 1
":m, s, a =. x. rr a                   NB. Truncate a, diagnostic
t =. s{a, 1x                            NB. Initial term: sin: angle; cos: 1
i =. - *: a                             NB. - angle squared
R =. 0 * a                              NB. Clear accumulator
r =. -s                                  NB. Initial term no., r, cos: 0; sin: 1
while. x. rdf R, R1 =. x. rr R+t do.
  t =. x. rr t*i % (<:r)*r=.r+2         NB. Update r and term
  R =. R1                                NB. Update R (? should truncate here also)
  if. 2>10|r do. R=.x. rr R [ t=.x. rr t end.
end.
if. diag do. dsp 'sa terms: ', ":<.:r+1 end. NB. Diagnostic
ps*R                                     NB. Correct result sign if necessary
)
```

NB. Calculation of pi

```

pa =: (3 : 0)*0                        NB. Rational Pi Approximation
T pa y.                                NB. Monad definition
:
NB. SYNTAX dps_accuracy pa ''
n %~ <.@a. n=.10^x: x.                NB. Based on Roger Hui's Code
)
```

NB. Calculation of Gamma constant (Approximately 0.577)

```

ge =: (3 : 0)*0                        NB. Knuth's algorithm for Euler's gamma
T ge y.                                NB. Monad definition
:
NB. SYNTAX: dps_accuracy ga n
n=. x: y.                              NB. Use n=1000 for upto 80 decimal places
R=.0r1                                  NB. Accumulator for sum of reciprocals
c =. fc =. 5                            NB. Full count for inner loop
'nc lc'=.(>.@%&c ; c&|&.<:) n          NB. Number_of_Cycles, Last_Count
for_j. i.nc do.                          NB. Outer loop
  if. j=nc-1 do. c=.lc end.             NB. Adjust count for last cycle
  R=.x.rr R+x.&rr@+/%(1+j*fc)+i.c       NB. Inner loop
end.
k =: +>:i.15                            NB. Bernoulli numbers for positive even k
x. rr R - +/(x. 1a n), % (2*n), n (-@%&(15{.2 }.bnnz)@] * ^) k
)
```

NB. Knuth's Constants, Constant Table

```

KC=: ,: '0) Sqrt 2';                   'ra 2'
KC=:KC, '1) Sqrt 3';                   'ra 3'
KC=:KC, '2) Sqrt 5';                   'ra 5'
KC=:KC, '3) Sqrt 10';                  'ra 10'
KC=:KC, '4) Cube root 2';              'ea (%63) 1a 2'
KC=:KC, '5) Cube root 3';              'ea (%63) 1a 3'
```

```

KC=:KC, '6) Fourth root 2';      'ra ra 2'
KC=:KC, '7) ln 2';                'la 2'
KC=:KC, '8) ln 3';                'la 3'
KC=:KC, '9) ln 10';               'la 10'
KC=:KC, '10) 1/ln 2';             '% la 2'
KC=:KC, '11) 1/ln 10';           '% la 10'
KC=:KC, '12) pi';                 'pa 0'
KC=:KC, '13) pi/180';             'pi%180 '
KC=:KC, '14) 1/pi';               '% pi '
KC=:KC, '15) pi squared';         '*: pi '
KC=:KC, '16) Sqrt pi, Gamma 1/2'; 'ra pi '
KC=:KC, '17) Gamma 1/3';          'G1r3=.fa _2r3'
KC=:KC, '18) Gamma 2/3';          '+:pi%G1r3=ra 3'
KC=:KC, '19) e';                  'ea 1'
KC=:KC, '20) 1/e';                'ea _1'
KC=:KC, '21) e squared';          'ea 2'
KC=:KC, '22) Euler's const gamma'; 'EC=.ge 1e2'
KC=:KC, '23) ln pi';              'la pi '
KC=:KC, '24) phi, Golden Ratio';  '->: ra 5'
KC=:KC, '25) e to power gamma';   'ea EC'
KC=:KC, '26) e to power pi/4';    'ea (%&4) pa 0'
KC=:KC, '27) sin 1';              's1=.sa 1'
KC=:KC, '28) cos 1';              'ra 1~:s1'
KC=:KC, '29) zeta 3, Sigma 1/n^3'; ''Not calculated''
KC=:KC, '30) ln phi';             'la ->: ra 5'
KC=:KC, '31) 1/ln phi';           '% la ->: ra 5'

```

```

s5 =: 3 : 0
if. 'e.y. do. (&'.>@{. , ,@(_5&((,&' ')\))>@(:) @(<:_2)&,&' y.
else. y.
end.
)

```

RCT =: 3 : 0 NB. Run Constants Table, eg KC

T RCT y. NB. Monad definition

NB. SYNTAX: RCT KC

```

t=. 'Nominal calculation precision, T, ', ('T=:x.),' decimal places. '
dsp t, 'Results correct to ', ('T-5),' decimal places', LF
V =: (10# ' '), 'D.Places:', 7}, (10&#@(' ')) , [ , '0"_)0 (<.T%10){.'123456789'
dsp 'LN Constant Time Memory J Code', V
for_i. _2[ \ y. do. NB. Allow single item of shape 2
t. 6!:2 's=. 7!:2 'r=. ".c=.>{:i'' NB. Time & memory to execute J code
if. 2~:3!:0 r do. v=.s5 (3+j.~T)": r else. c=.'' [ v=.r end.
dsp (23{.>{:i}, (6j3":t), (7":s), ' ', (16{.c), v
end.
0 Q$'
)

```

RCTD =: 3 : 0 NB. Guide to no. of decimal places correct in result

(5*>:i.10) RCTD y. NB. Monad definition; 34 sec runtime

NB. SYNTAX: T_vals RCTD CT NB. Run Constant Table Diagnostic

```

dsp 'Guesstimate of number of decimal places correct (has flaws)', LF
R =: (0, #y._2[\,y.)$'' NB. Allow single item of shape 2
for_T. x. do.
  T =: T NB. Set global T
  R1 =. 0$''
  for_i. y. do.
    t=. 6!:2 's=. 7!:2 ''r=.'.c=.>1{i'' NB. Time & memory to execute J code
    if. 2~:3!:0 r do. v=.s5 ((5+T)j.T)": r else. c=.'' [ v=.r end.
    $ R1 =. R1, <r
  end.
  R =: R, R1
end.
dsp 'LN Constant T=', 3}. 5":x.
(>{."1 y.). 5j1":. (2&(cdp/\)"1@:> :: (0:/)"1) |:R
)

tc=: {.&bnnz *&}.&}. $&1 _1 % i. * <:@i. NB. Coeffs for Gamma (tc N)
ts=: ($&1@-&2@[ # ]@tc@[) % [: */\ ] , $&1 @-&2@[ ].@# *:@] NB. Exp. terms (N ts n)

fa =: (3 : 0)"0 NB. Factorial y. approximation (Knuth p115)
T fa y. NB. Monad definition
:
NT=.51x NB. Arbitrary number of term in series
'y. z' =. (>:y.),(<:# y.).45x NB. Initialise z
N=.x:(<:@+:@(/) % =&0@{:) x. >- 10^, NT ts x^:_1 z NB. Determine No of terms +2 ???
n=. y+(z-<.y=>:y.) NB. 25 <= n < 26. Term 41 <= 10^51

NB. m is ln of correction divisor, +/fract+integers
m=. -(1&| x.&rr@ 1a@|@(_30&(x.&rr@*/\))@:+ <.@<: }. i.@>:@(x"_)y
k=.+:>:i.<.-:N+2 NB. Values of k for each term
C=. ({&bnnz@>@-: % [ * <:)k NB. Modify bn coeffs

R=.x.&rr@+/ m, C % */\ n,(<.-:N-1)#*:n NB. Divide coeffs by powers of n and sum
x.&rr{ra +: x.&rr pi*n) * x.&rr ea R + n*1a x.&rr n%1 NB. Root and exponential terms
)

```

Index to Advertisers

APL Borealis Inc	37
Dyadic Systems Ltd	10
Renaissance Data (Booklist)	100
Soliton	2
Strand Software	6
Vector Back Numbers	5

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2000/2001 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-973 0036 acam@tesco.net	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Corso Vercelli 54 20145 - Milano Italy
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
Education	Dr Ian Clark earthspot@aol.com Dave Phillips 100117.2254@compuserve.com	c/o Vector Production
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths, Bob Hoekstra (01483-771028), Jon Sandles (01904-612882)	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk
Denmark
Tel: +45 70 26 13 26
Fax: +45 70 26 13 25
Email: info@insight.dk
Web: www.insight.dk

APL2000
Rapid Application Development
6610 Rockledge Drive
Suite 502
Bethesda MD 20817
USA
Email: sales@apl2000.com
Web: www.apl2000.com

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: +31 347 342 337